



智能计算系统

第三章 深度学习

北京邮电大学人工智能学院，何召锋

北京邮电大学人工智能学院，徐振博

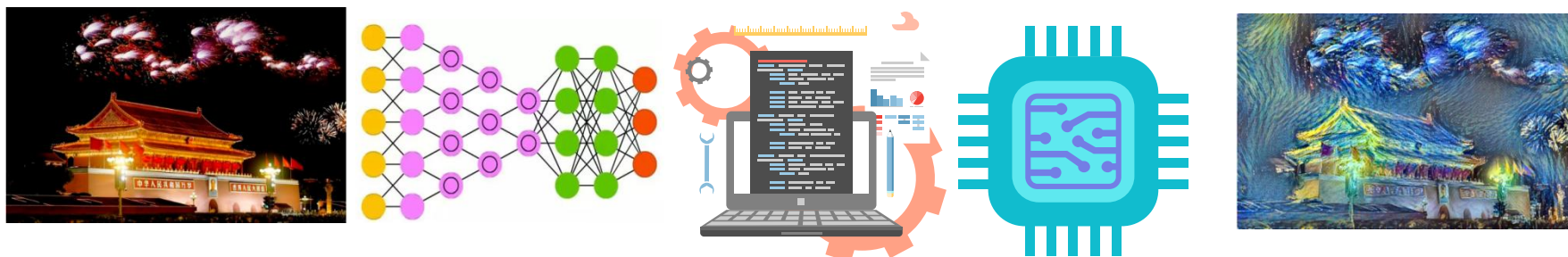
北京邮电大学人工智能学院，项刘宇

中国科学院计算技术研究所，李威

第二章回顾

- ▶ 从机器学习到神经网络
 - ▶ 线性回归→感知机
- ▶ 神经网络训练
 - ▶ 前向+反向，损失函数+梯度下降
- ▶ 神经网络设计原则
 - ▶ 网络的拓扑结构
 - ▶ 激活函数
 - ▶ 损失函数
- ▶ 过拟合与正则化
 - ▶ L1, L2, 稀疏化, Bagging, Dropout
- ▶ 交叉验证
- ▶ 小结

Driving Example



神经网络基础



上一章学习了神经网络的基本知识，多层感知机的正反向计算过程，以及基础优化方法。

深度学习



本章通过分析经典深度学习算法，**了解其设计思路和方法**，学习将基础神经网络应用到实际场景，并逐步优化实现应用效果的过程。

目标：让机器更好的理解和服务人类

方法：使计算机来模拟人的某些思维过程和智能行为的学科

>70%

人获得的输入是什么？

图像信息

任务：理解图像内容

方法：卷积神经网络

眼、耳、手、鼻、舌



序列信息

任务：理解语音/文字/视频

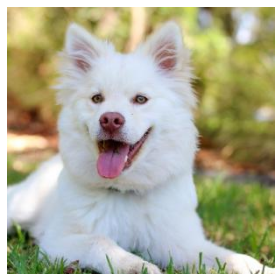
方法：循环神经网络

提纲

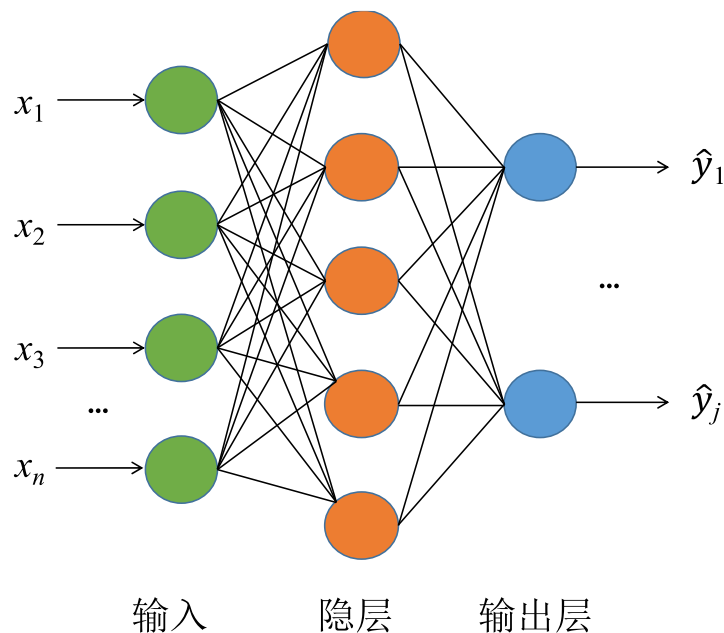
- ▶ 适合图像处理的卷积神经网络
- ▶ 基于CNN的图像分类算法
- ▶ 基于CNN的图像检测算法
- ▶ 近年来目标检测算法简介
- ~~▶ 序列模型：循环神经网络~~
- ~~▶ 序列模型：长短期记忆模型~~
- ~~▶ Transformer方法及应用~~
- ~~▶ 生成对抗网络GAN、Diffusion模型~~
- ▶ Driving Example
- ▶ 小结

一个例子

► 计算机视觉



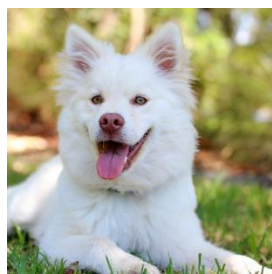
输入图像



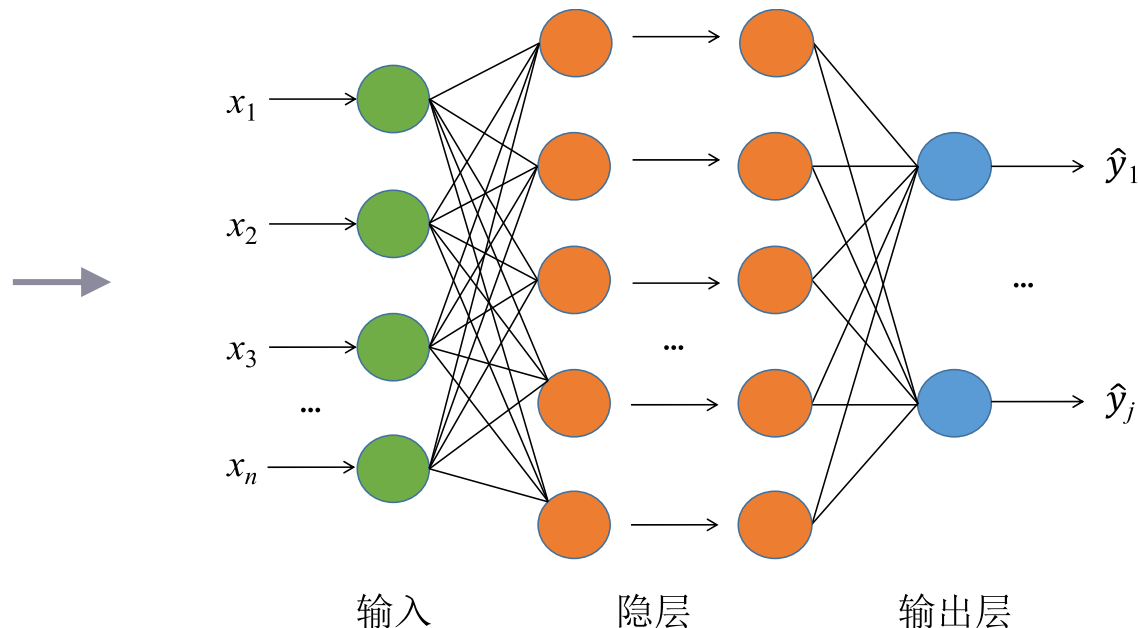
- 输入图像大小为 32×32 ，输入数据量为 $32 \times 32 \times 3 = 3072$
- 隐层神经元个数为 100，第一层权值数量为 $3072 \times 100 = 307200$

一个例子

- 实际场景中，往往需要更大的输入图像以及更深的网络结构。

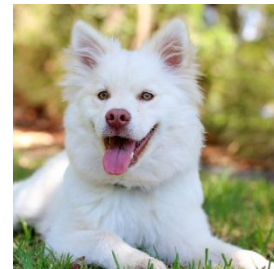


输入图像



- 输入图像大小为 1024x1024，第一层隐层神经元个数为 1000
- 第一层权重数量级为 10^9 ，过多的参数会导致过拟合
- 如何降低权重的数量，又能够去支持很大的输入图像跟很深的网络结构？

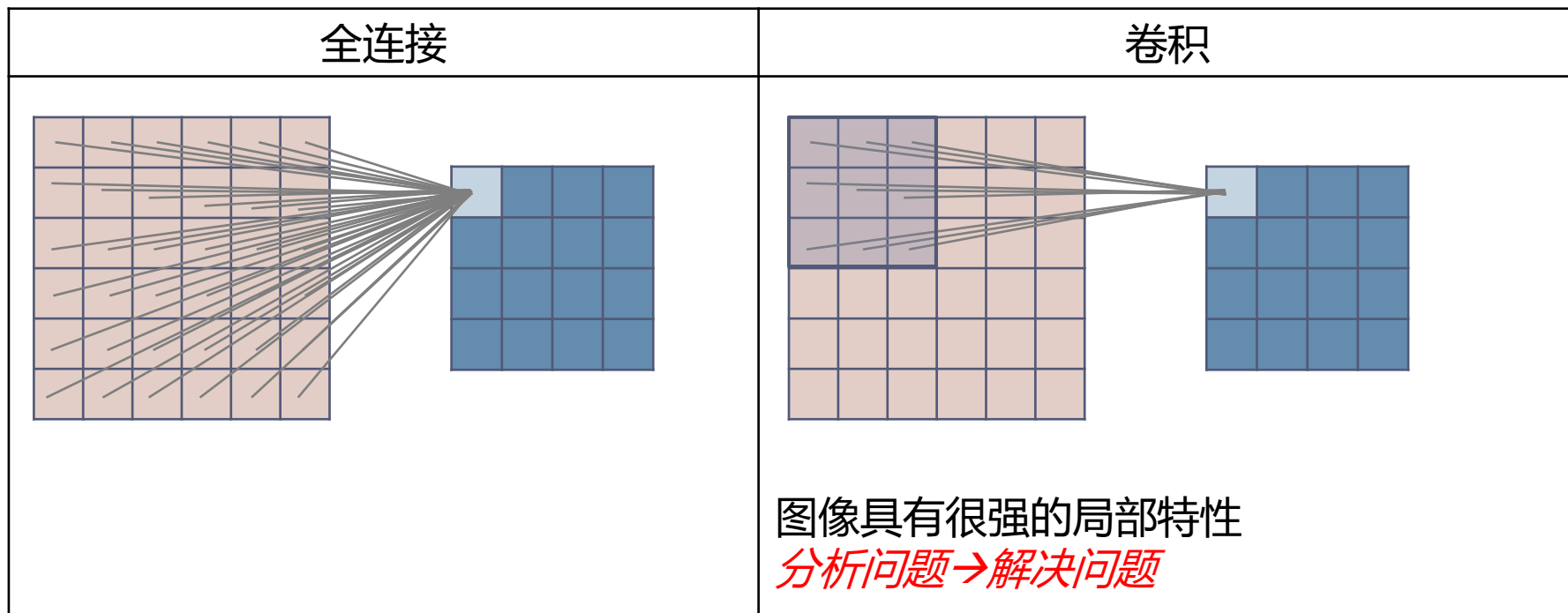
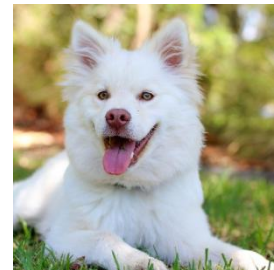
卷积神经网络 (CNN)



	全连接	卷积
局部连接		图像具有很强的局部特性
权重共享	所有神经元之间的连接都使用不同权重。	输出层神经元共用同一组权重，进一步减少权重数量。 注：权重，其实就是卷积模板
权重数量	$w_i \times h_i \times w_o \times h_o$	$f \times f$

卷积神经网络 (CNN)

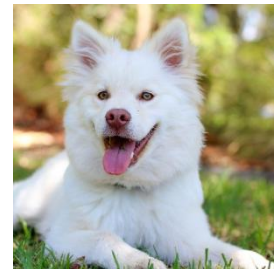
局部连接



- ◆ 全连接网络每一层之间都是全连接的。
- ◆ 计算机视觉问题具有很强的局部特性：一个像素跟它周围的旁边几个像素有很强联系，但是离它非常远的像素之间他们联系的可能性就非常弱，所以不需要去做这种全连接的神经网络。
- ◆ 只需要在局部去做连接，权重就会减少。

卷积神经网络 (CNN)

权重共享



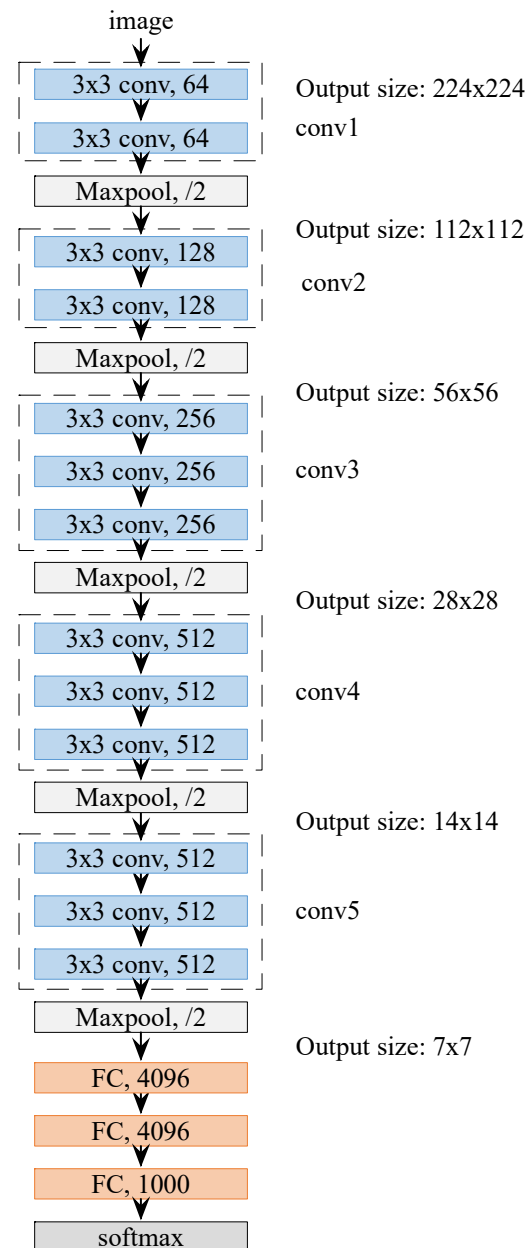
	全连接	卷积
权重共享	所有神经元之间的连接都使用不同权重。	输出层神经元共用同一组权重，进一步减少权重数量。 注：权重，其实就是卷积模板
权重数量	$w_i \times h_i \times w_o \times h_o$	$f \times f$

- ◆ 全连接的神经网络每一个权重都是独立的，都是不可复用的。
- ◆ 卷积神经网络中，权重又称为卷积模板，卷积模板用于表达一种图象的特征。在图象的不同位置去找特征，可以用同样的卷积模板的。比如找三角形的特征需要的卷积模板，在图象的不同位置，它是有共性的，是统一的。所以完全可以在图象的不同位置，共用同样的一种权重，这样可以进一步减少我们权重的数量。
- ◆ **引申思考：**传统模式识别方法通常需要手工设计特征提取算子（比如sobel算子，gabor算子，OM算子等等）。而在卷积神经网络中，不再需要手工设计，而是用计算机自动的搜索最优算子。**这是传统模式识别算法与人工神经网络的完美结合，是卷积神经网络成功的最核心原因之一。**

CNN组成

► VGG16

- 卷积层 (conv)
 - 一组卷积模板，抽特征，不同位置共享
- 池化层 (max pool)
 - 图像尺寸变小
- 全连接层 (FC)
 - 用于分类
- Softmax
 - 转化为分类概率



卷积层

- 卷积层如何检测特征（卷积模板==特征抽取）

➤ 卷积神经网络的两个重要特征：局部连接、权重共享

可有效减少权重参数，避免过拟合，为增加卷积层数提供可能。

- 检测复杂边缘

将卷积模板权重作为参数，
在训练中学习。



w0	w1	w2
w3	w4	w5
w6	w7	w8

filter/kernel

卷积层

- 卷积层如何检测特征

- 检测垂直边缘

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

 *

1	0	-1
1	0	-1
1	0	-1

 =

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

- 检测对角线边缘

10	10	10	10	10	0
10	10	10	10	0	0
10	10	10	0	0	0
10	10	0	0	0	0
10	0	0	0	0	0
0	0	0	0	0	0

 *

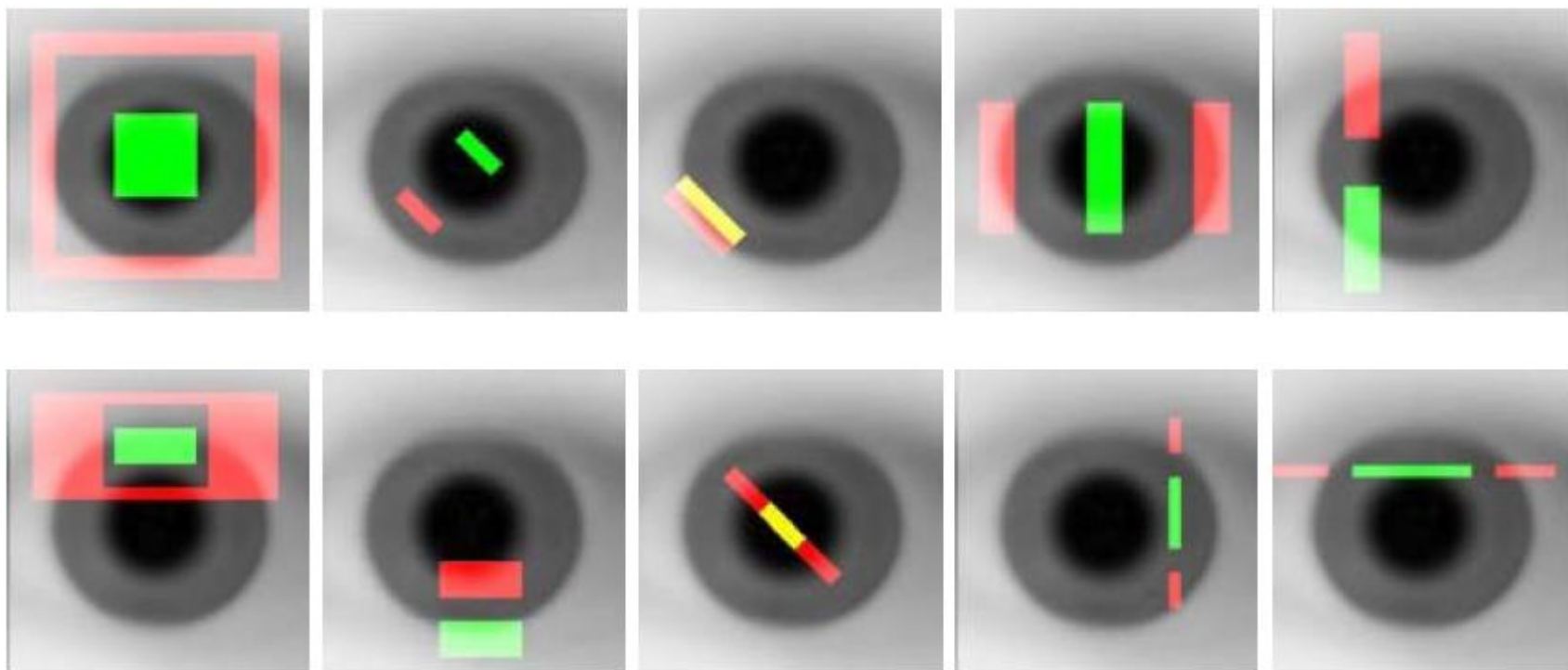
1	1	0
1	0	-1
0	-1	-1

 =

0	10	30	30
10	30	30	10
30	30	10	0
30	10	0	0

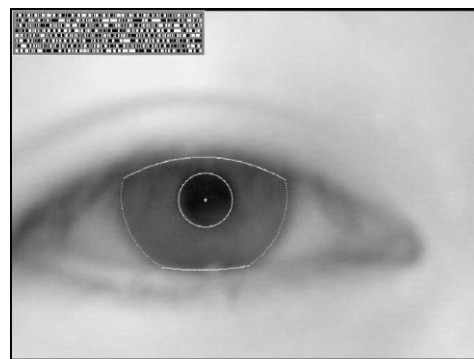
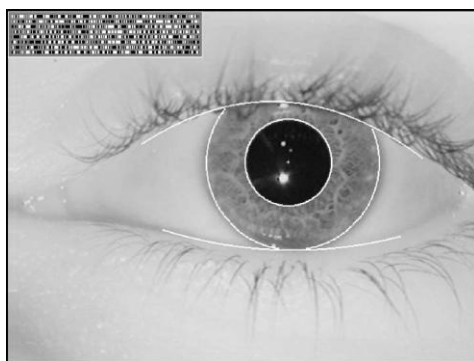
传统模式识别方法

- 目标检测



传统模式识别方法

- 质量判断



-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	+3	+3	+3	+3	-1	-1
-1	-1	+3	+3	+3	+3	-1	-1
-1	-1	+3	+3	+3	+3	-1	-1
-1	-1	+3	+3	+3	+3	-1	-1
-1	-1	-1	-1	-1	-1	-1	-1
-1	-1	-1	-1	-1	-1	-1	-1

The (8×8) convolution kernel for fast focus assessment.

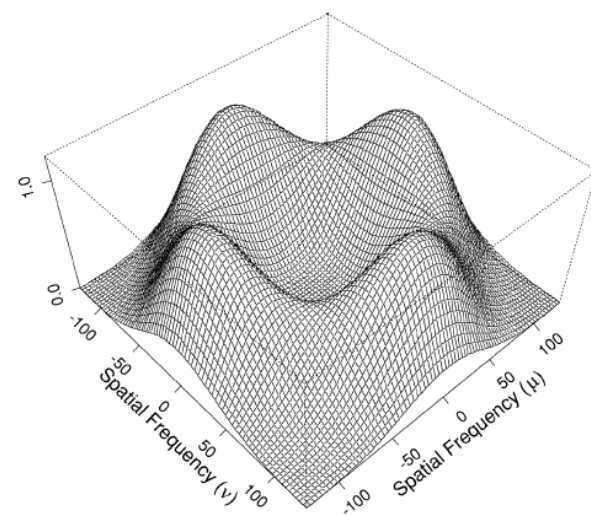
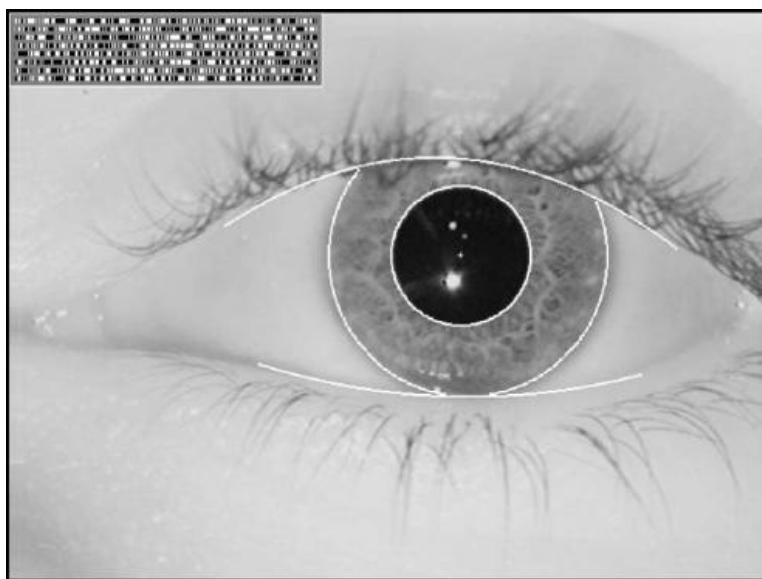


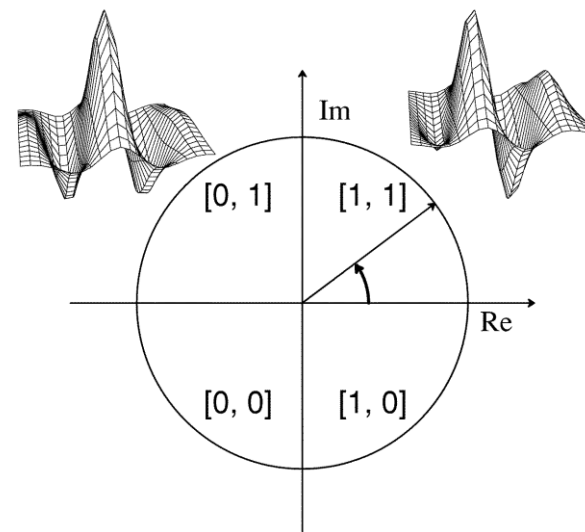
Fig. 12. The 2-D Fourier power spectrum of the convolution kernel used for rapid focus assessment.

传统模式识别方法

- 特征抽取和分类Gabor



Phase-Quadrant Demodulation Code



$$h_{\{Re, Im\}} = \text{sgn}_{\{Re, Im\}} \int_{\rho} \int_{\phi} I(\rho, \phi) e^{-i\omega(\theta_0 - \phi)} \cdot e^{-(r_0 - \rho)^2 / \alpha^2} e^{-(\theta_0 - \phi)^2 / \beta^2} \rho d\rho d\phi$$

传统模式识别方法

- 特征抽取和分类OM (Ordinal Measures)

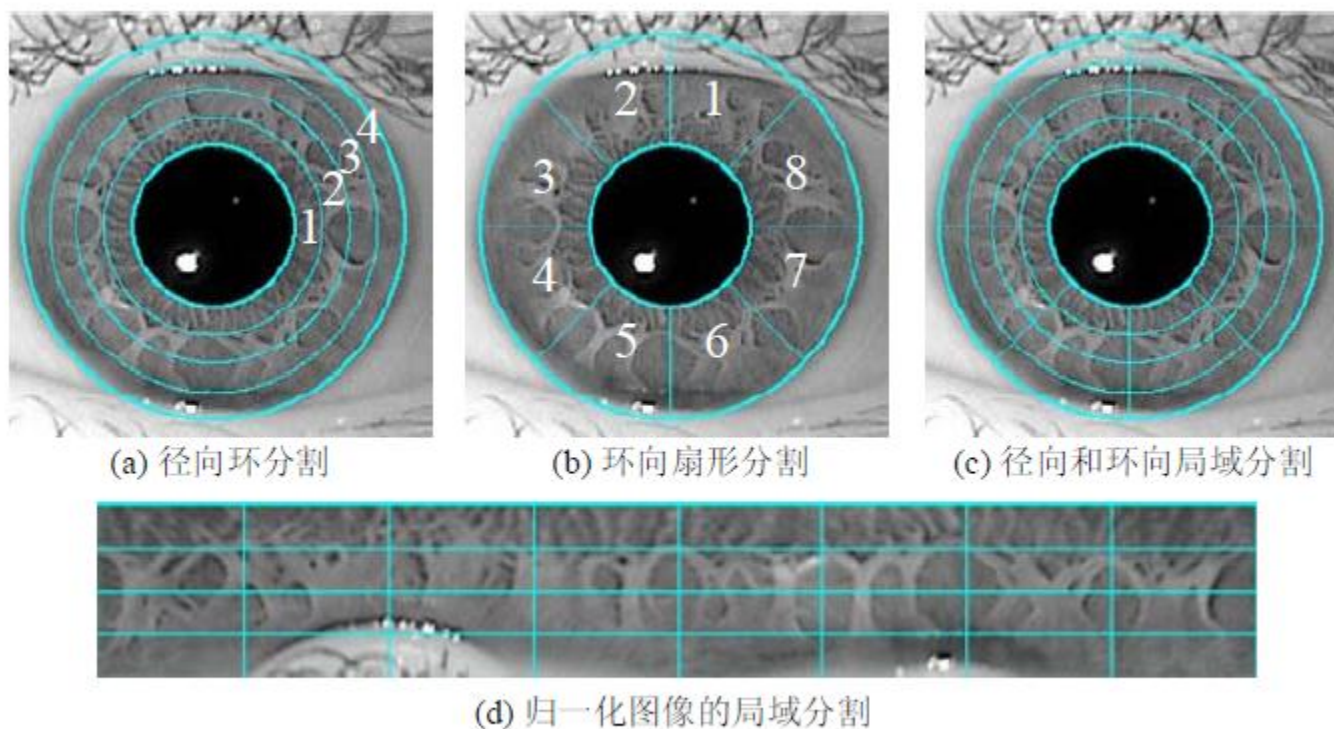


图 7-3 基于虹膜纹理径向延展性和环向相关性的虹膜图像区域分割

传统模式识别方法

- 特征抽取和分类Gabor

Adaboost
特征选择

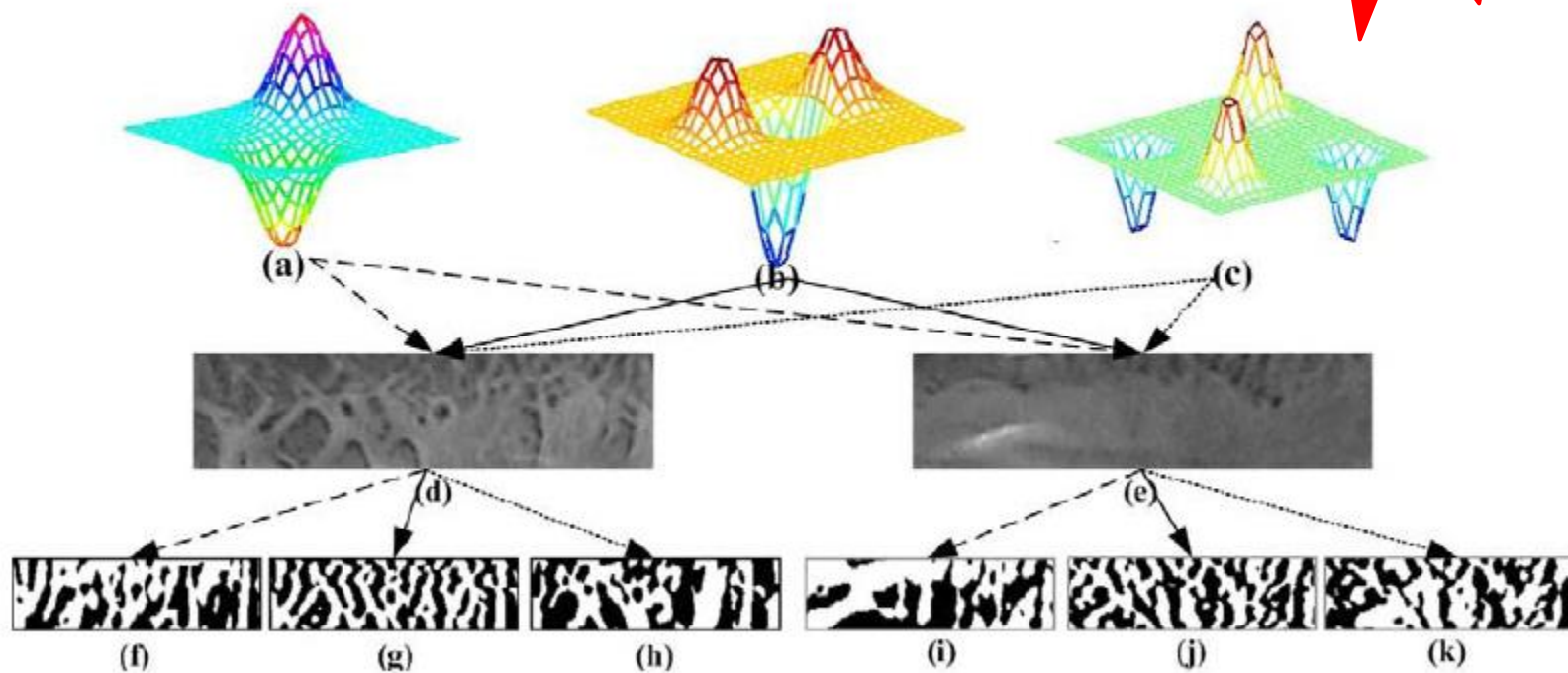


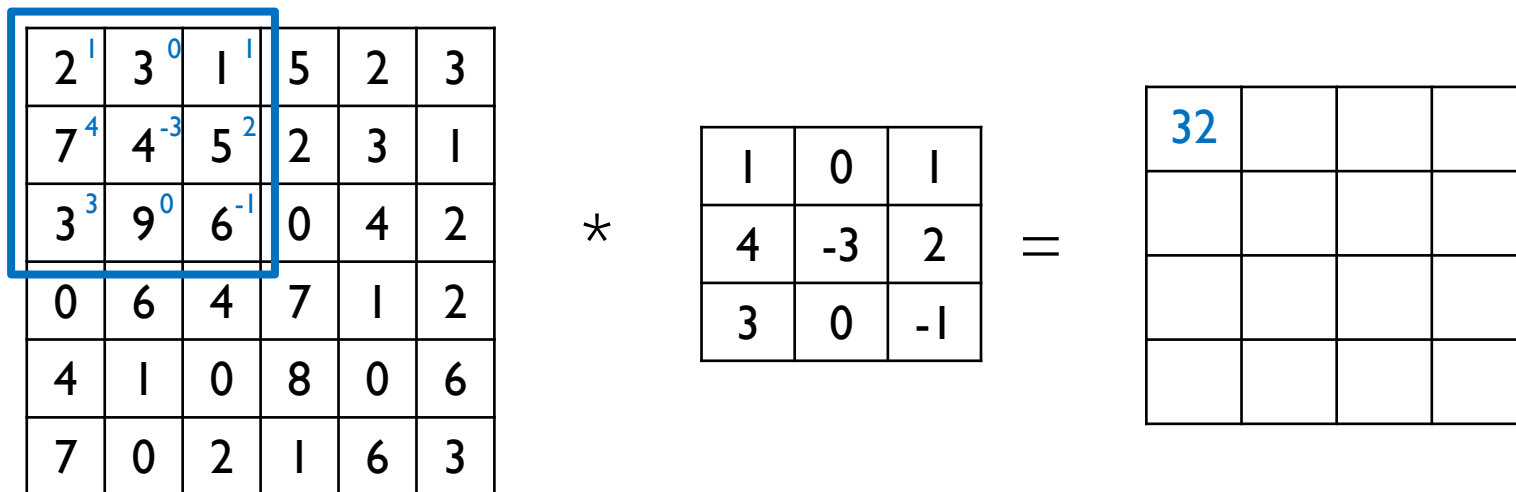
图 7-9 Gaussian-OM定序编码示例

卷积层

- 卷积运算

▶ 数学: $y(n) = \sum_{i=-\infty}^{\infty} x(i)h(n-i) = x(n) * h(n)$ ("*"表示卷积)

▶ 神经网络: 实际为计算矩阵内积(相关系数);



卷积层

2	3 ¹	1 ⁰	5 ¹	2	3
7	4 ⁴	5 ⁻³	2 ²	3	1
3	9 ³	6 ⁰	0 ⁻¹	4	2
0	6	4	7	1	2
4	1	0	8	0	6
7	0	2	1	6	3

*

1	0	1
4	-3	2
3	0	-1

=

32	40		

2	3	1	5	2	3
7 ¹	4 ⁰	5 ¹	2	3	1
3 ⁴	9 ⁻³	6 ²	0	4	2
0 ³	6 ⁰	4 ⁻¹	7	1	2
4	1	0	8	0	6
7	0	2	1	6	3

*

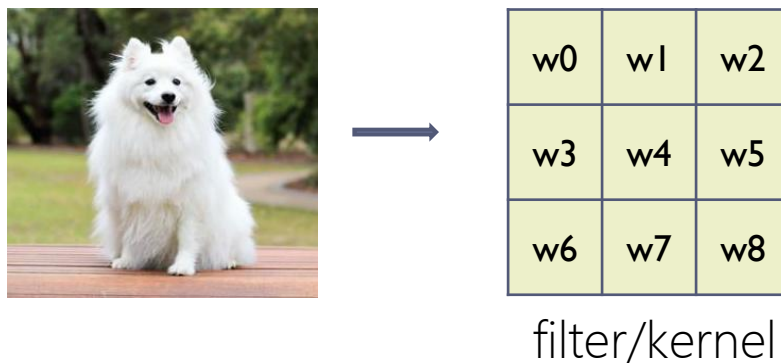
1	0	1
4	-3	2
3	0	-1

=

32	40	37	7
5			

卷积层

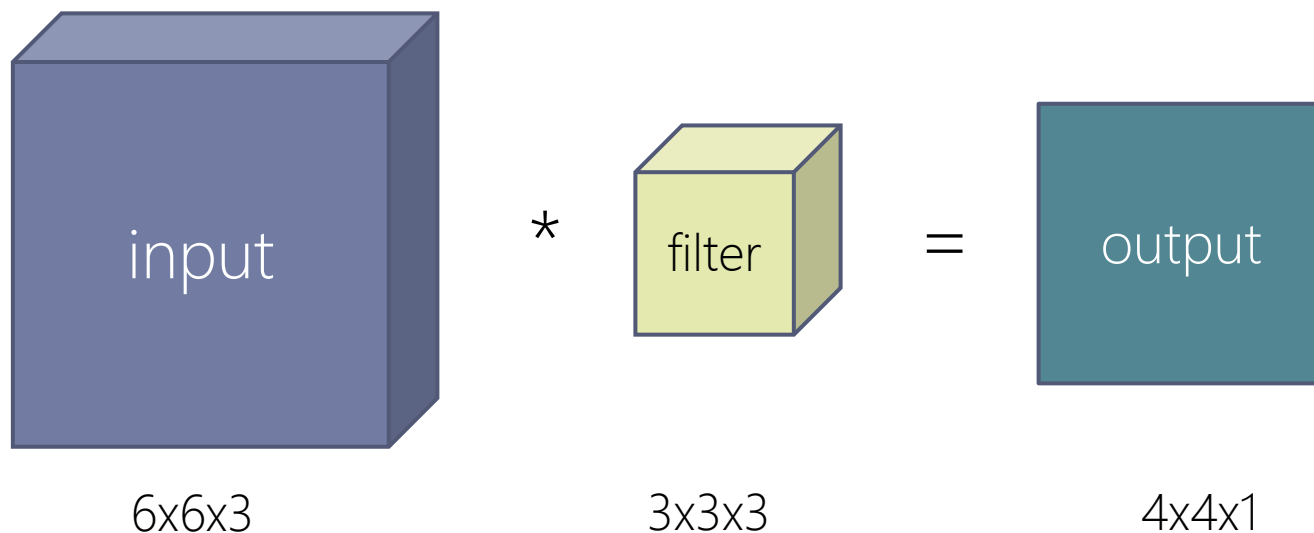
- 卷积层如何检测特征



- 卷积模板==特征抽取

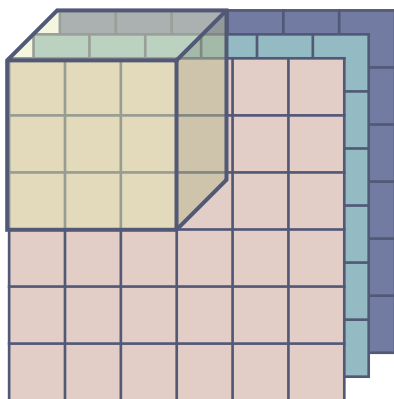
卷积层

- 多输入特征图单输出特征图卷积运算



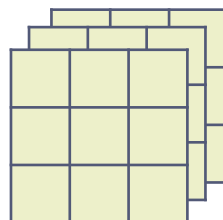
如何算？

卷积层



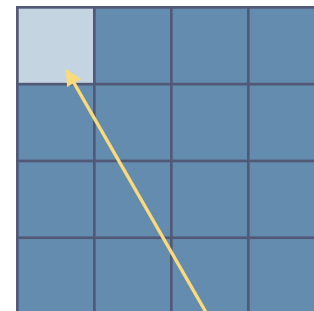
6x6x3

*



3x3x3

=



4x4x1

$C = 0$

0	0	0
0	2	2
0	1	2

$C = 1$

0	0	0
0	0	2
0	1	2

$C = 2$

0	0	0
0	1	1
0	0	2

*

-1	1	1
-1	1	-1
1	-1	1

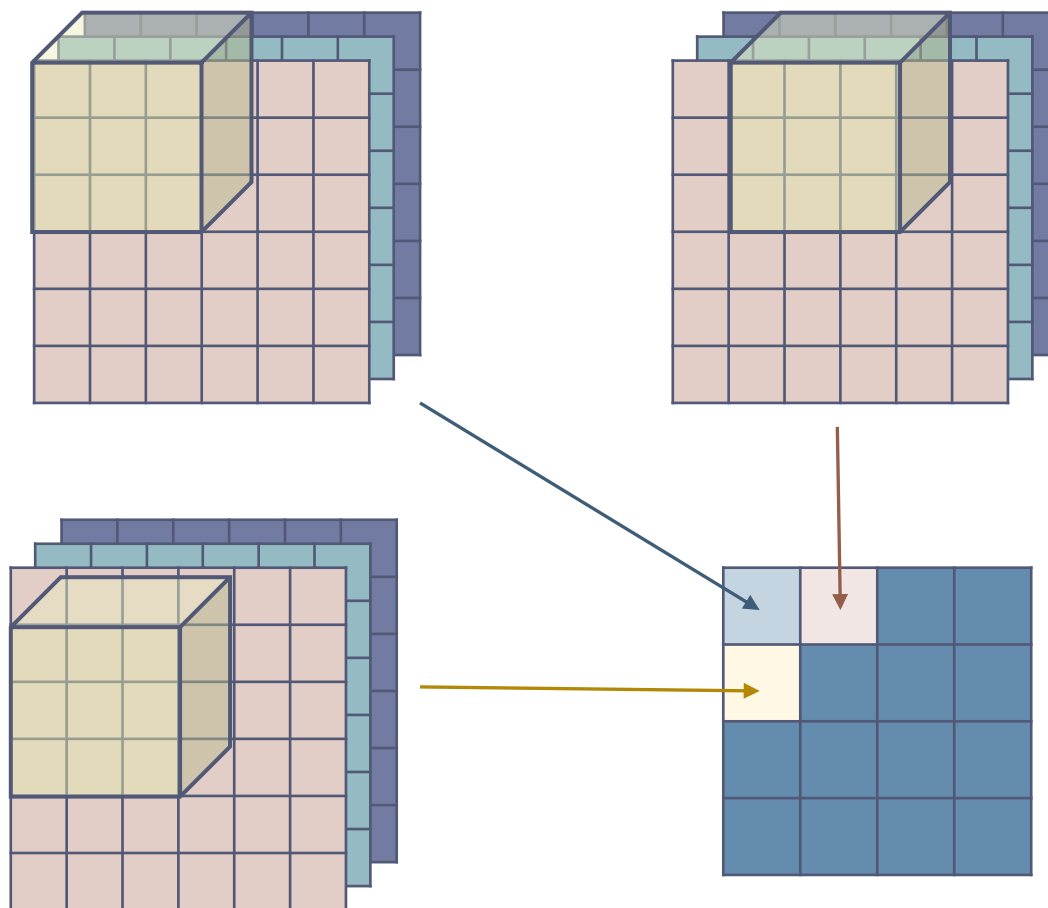
1	-1	-1
-1	0	-1
-1	0	1

1	-1	-1
-1	-1	0
-1	1	1

=

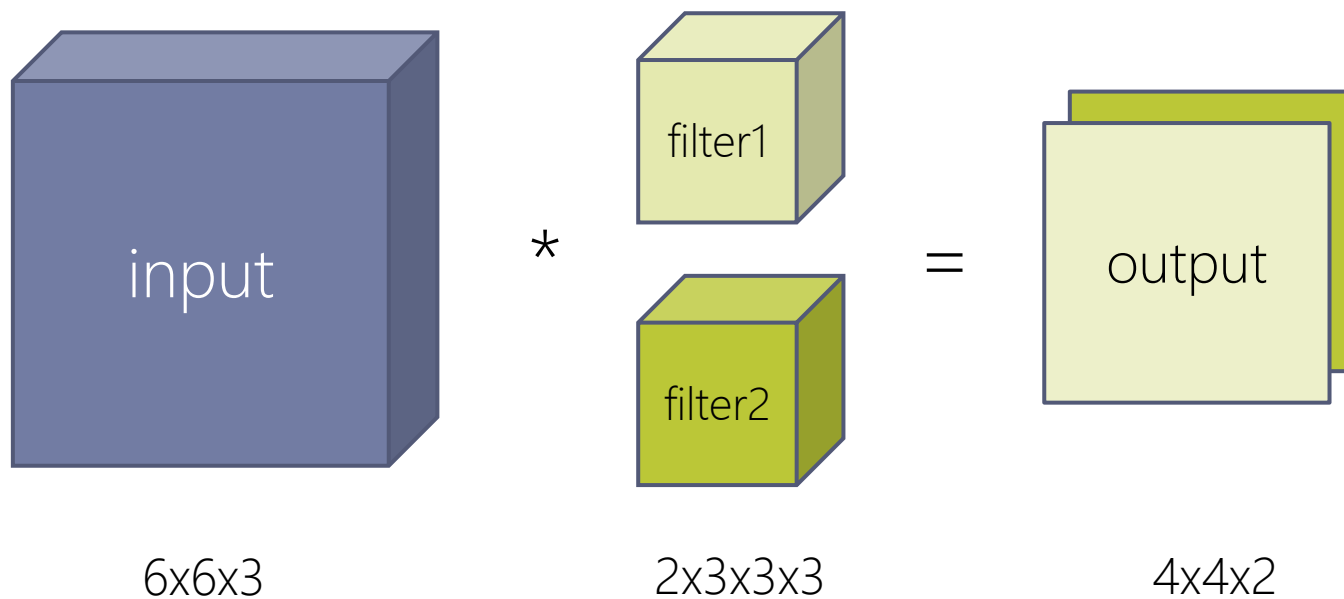
$$\begin{aligned}
 &2 - 2 - 1 + 2 \\
 &+ \\
 &0 - 2 + 0 + 2 \\
 &+ \\
 &(-1) + 0 + 0 + 2 \\
 &= 2
 \end{aligned}$$

卷积层



卷积层

- 多输入特征图多输出特征图卷积运算



- 不同的过滤器可检测不同特征。

卷积层

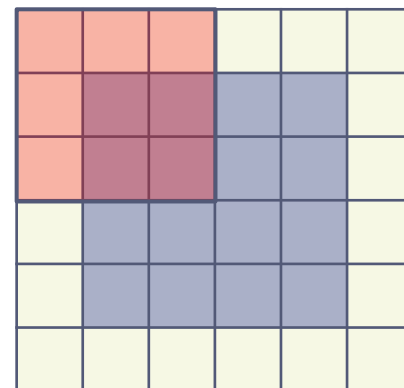
- 边界扩充(padding)

- 扩大输入图像/特征图的尺寸并填充像素
- 防止深度网络中图像被动持续减小
- 强化图像边缘信息

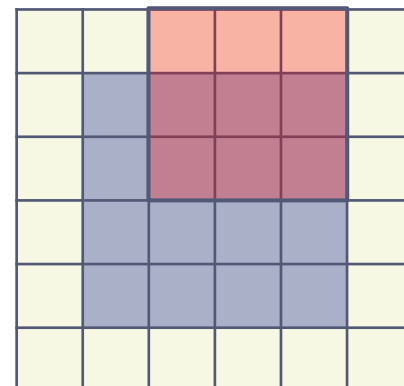
- 卷积步长(stride)

- 滑动滤波器时每次移动的像素点个数
- 与pad共同确定输出图像尺寸

$input = 4 \times 4$, $filter = 3 \times 3$, $pad = 1$



↓ $stride = 2$



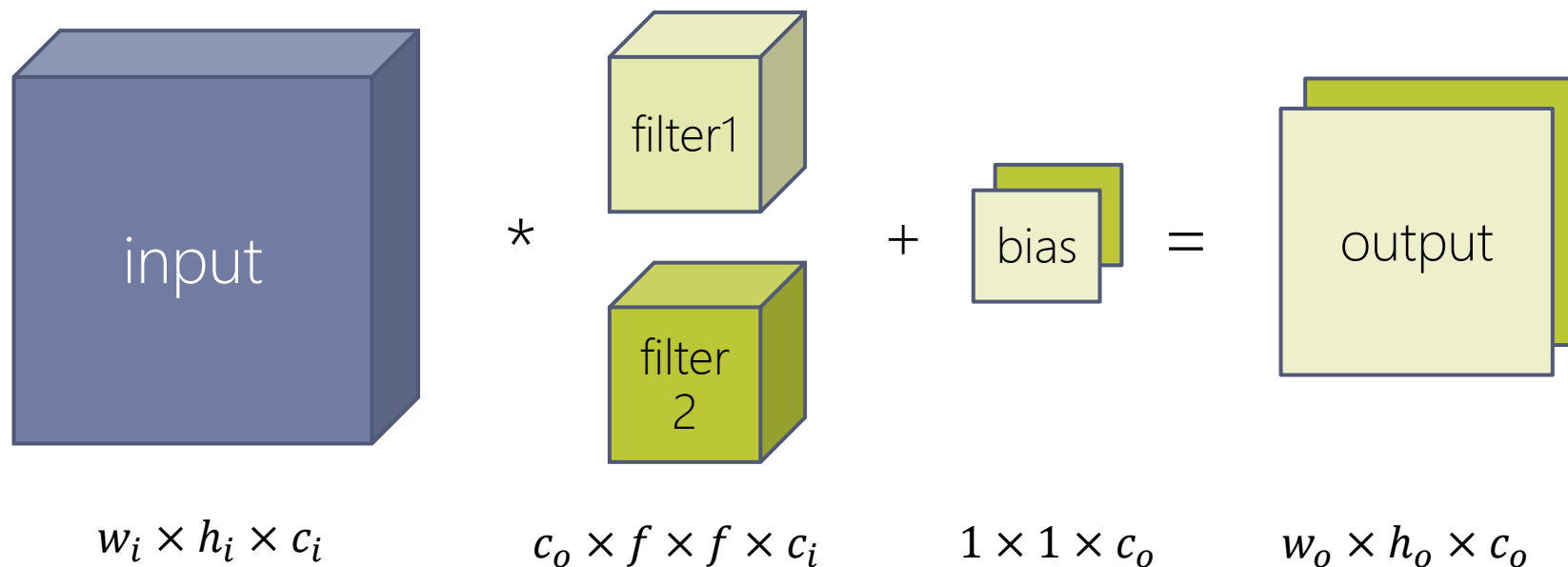
$$input = w_i \times h_i, \quad filter = f \times f, \quad pad = p, \quad stride = s$$

$$output = \left\lfloor \frac{w_i + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{h_i + 2p - f}{s} + 1 \right\rfloor$$

卷积层

有没有人研究Ci层之间参数共享，退化为一个深度为1的卷积核？

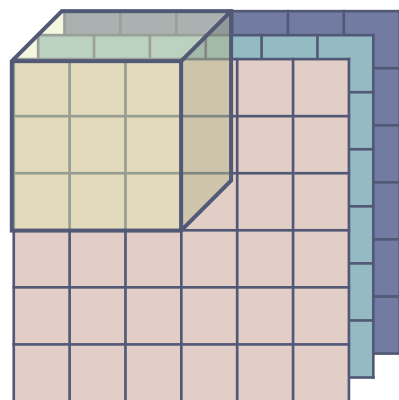
- 总结：卷积层参数



- stride
- pad
- $\text{output} = \left\lfloor \frac{w_i + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{h_i + 2p - f}{s} + 1 \right\rfloor$
- filter: 可训练
- bias: 可训练, 使分类器偏离激活函数原点, 更灵活;
- activation

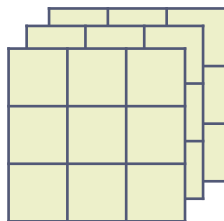
卷积层

在卷积运算中，输入特征图的通道数和卷积核的通道数必须一致



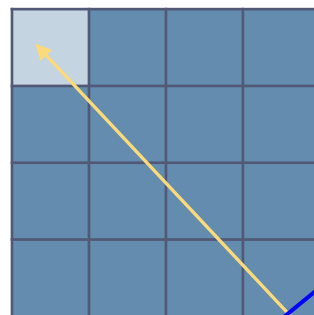
6x6x3

*



3x3x3

=



4x4x1

有没有人研究Ci层之间参数共享，退化为一个深度为1的卷积核？

C = 0

0	0	0
0	2	2
0	1	2

C = 1

0	0	0
0	0	2
0	1	2

C = 2

0	0	0
0	1	1
0	0	2

*

-1	1	1
-1	1	-1
1	-1	1

1	-1	-1
-1	0	-1
-1	0	1

1	-1	-1
-1	-1	0
-1	1	1

$$\begin{aligned}
 & 2 - 2 - 1 + 2 \\
 & + \\
 & 0 - 2 + 0 + 2 \\
 & + \\
 & (-1) + 0 + 0 + 2 \\
 & = 2
 \end{aligned}$$

Tensorflow中的padding

- valid: no padding

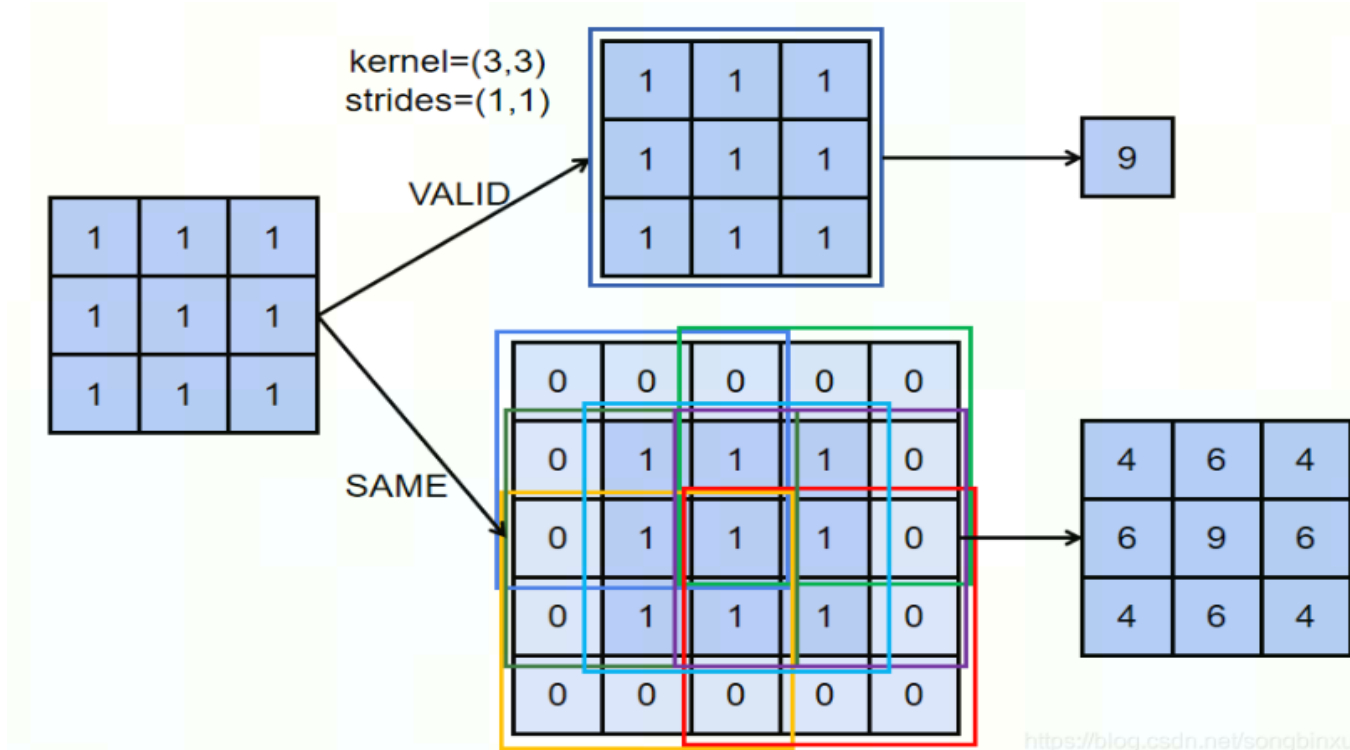
$$\text{output} = \left\lfloor \frac{w_i - f + 1}{s} \right\rfloor \times \left\lfloor \frac{h_i - f + 1}{s} \right\rfloor$$

- same: results in padding evenly to the left/right or up/down of the input such that output has the same height/width dimension as the input

$$\text{output} = \left\lceil \frac{w_i}{s} \right\rceil \times \left\lceil \frac{h_i}{s} \right\rceil$$

Tensorflow中的padding

- 输入x是一个3x3的矩阵，卷积核为3x3，两个维度的strides=1



- $$\left\lfloor \frac{w_i + 2p - f}{s} + 1 \right\rfloor = \left\lfloor \frac{w_i}{s} \right\rfloor, \rightarrow 2p = 2$$

池化层

- **Pooling**: 用来主动减小图象的尺寸的, 从而控制减少这种参数的数量和计算量
 - Max Pooling / Avg Pooling / L2 Pooling
 - 主动减小图片尺寸, 从而减少参数的数量和计算量, 控制过拟合;
 - 不引入额外参数;

2	3	1	5	2	3
7	4	5	2	3	1
3	9	6	0	4	2
0	6	4	7	1	2
4	1	0	8	0	6
7	0	2	1	6	3

Max pooling
→
 $filter = 2 \times 2, pad = 0, stride = 2$

7	5	3
9	6	4
7	8	6

Max pooling 可保留特征最大值, 提高提取特征的鲁棒性。

全连接层

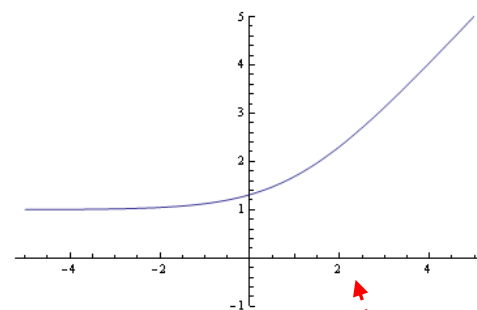
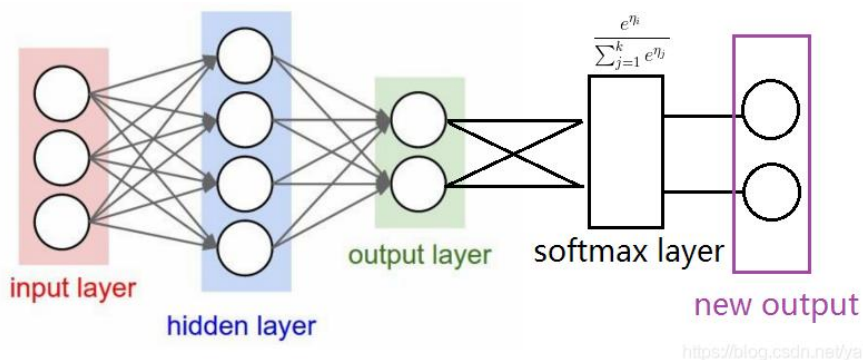
- Fully Connect: 分类

- 卷积层和池化层构成特征提取器，全连接层则为分类器；
- 将特征提取得到的高维特征图映射成一维特征向量，该特征向量包含所有特征信息，可转化为各个类别的概率。

- Softmax: 对输出进行归一化、概率化

- 通常作为网络的最后一层，对输出进行归一化，输出分类概率；
- **凸显其中最大的值**并抑制远低于最大值的其他分量；
- Softmax层输入、输出数据规模相同；
- 公式: $f(z_j) = e^{z_j} / \sum_{i=0}^n e^{z_i}$

为何需要Softmax?



- softmax层只是对神经网络的输出结果进行了一次换算，将结果用概率的形式表现？
得分结果是数值型，不便直接用作分类，而soft之后的概率分布可直观用作分类。

训练过程：

输入层经过hidden层之后，得到计算值；

然后softmax计算后，得到概率分布；

接着通过交叉熵cross entropy计算损失函数；

随后用链式法则，反向传播更新网络权重。

多次迭代得到分类的预测结果。

优点：softmax的求导非常简单，反向更新的梯度就是正确分类 y_i 的softmax值减去真实标签label (<https://blog.csdn.net/u014453898/article/details/108435173>)

为什么用softmax，而不直接用标准化将确定范围的值转换到区间[0,1]当中呢？

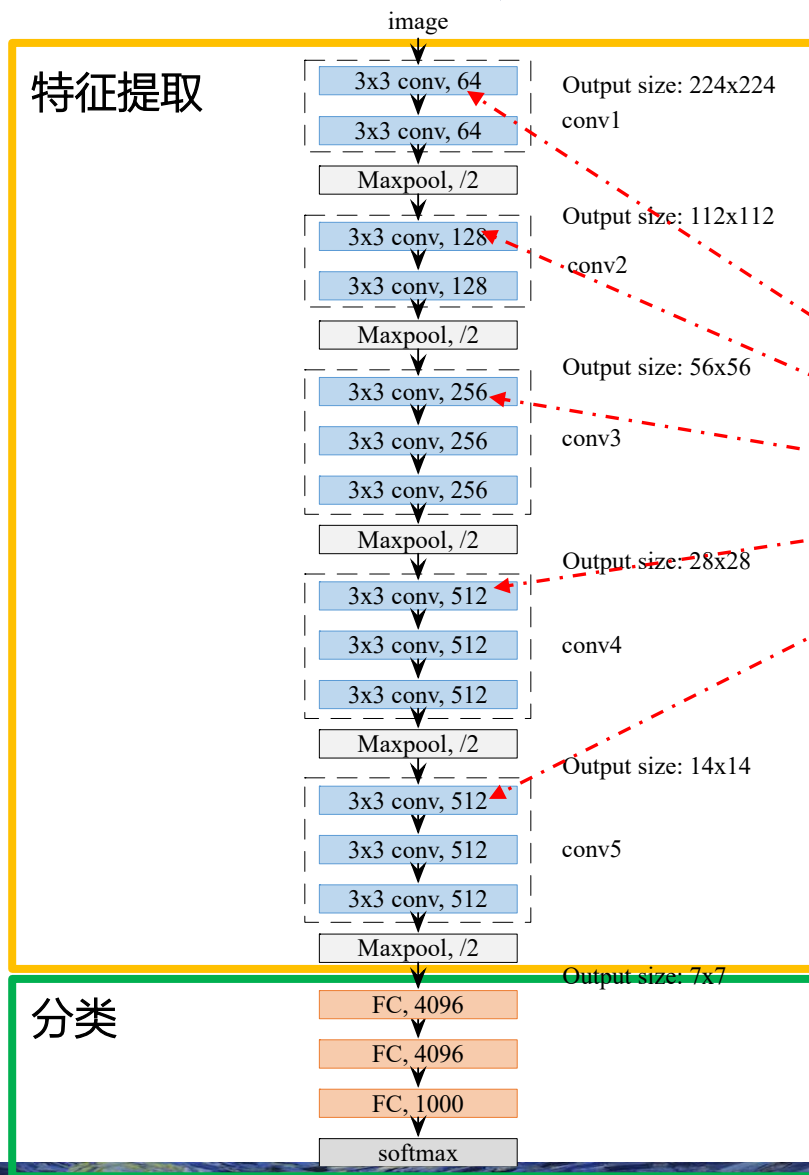
•如果计算结果里面有很大的值（正、负都有），通过normalization会有部分分数会变得非常小。Softmax相较于normalization能够更好的区分不同类别的计算结果（结果区别比较小）

Softmax及其求导过程

- ▶ <https://blog.csdn.net/u014453898/article/details/108435173>

各层如何排布组成一个网络?

- VGG16

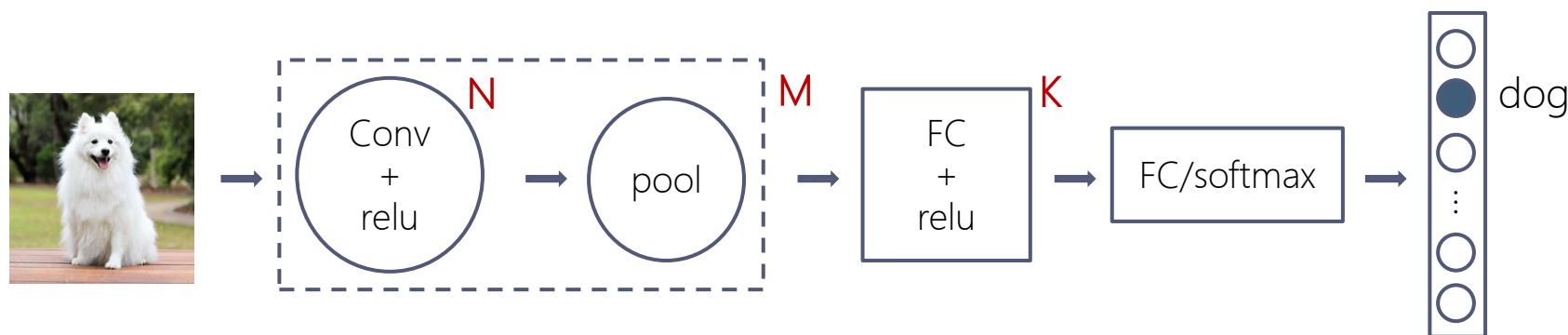


问题：这些数是怎么来的？

通道数，做卷积的时候，用多少个卷积核，得到的输出通道数就是多少，接着做池化不改变通道数，只改变h和w

卷积神经网络结构

- 层排列规律



- 常见卷积神经网络由卷积层（激活）、池化层和全连接层构成；
- 各层的常见排列方式如图所示，其中N、M、K为重复次数；
- 例如：N=3，M=1，K=2 情况下的网络结构为：

input → conv(relu) → conv(relu) → conv(relu) → pool → FC(relu) → FC(relu) → FC → output

- 其中卷积和池化部分可包含分支和连接结构，将在具体网络分析中介绍。

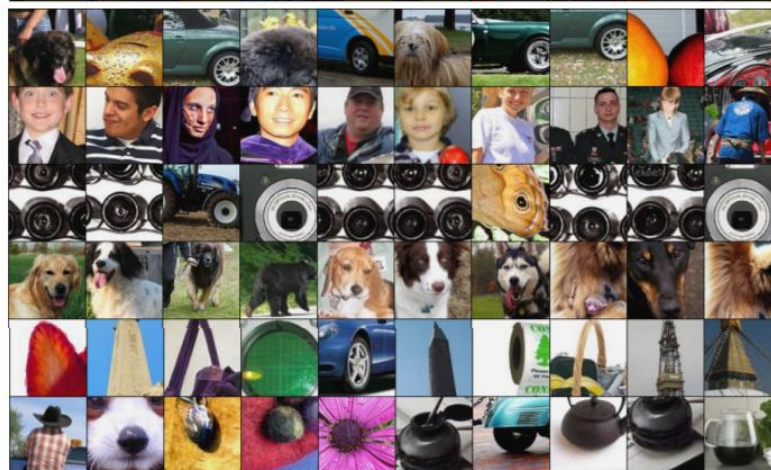
浅层学习局部特征，深层学习整体特征

- 神经网络可视化:

conv6



conv9



Springenberg, J. T.; Dosovitskiy, A.; Brox, T. & Riedmiller, M. Striving for simplicity: the all convolutional net ICML, 2015, 1-12

卷积神经网络结构

- 为何选择“深”而非“广”的网络结构
 - 即使只有一层隐层，只要有足够的神经元，神经网络理论上可以拟合任意连续函数。为什么还要使用深层网络结构？

- 两方面原因：

- ▣ 深度网络可从局部到整体“理解图像”

学习复杂特征时（例如人脸识别），浅层的卷积层感受野小，学习到局部特征，深层的卷积层感受野大，学习到整体特征。

- ▣ 深度网络可减少权重数量

用多个小卷积替代一个大卷积，在获得更多样特征的同时所需权重数量也更少。

提纲

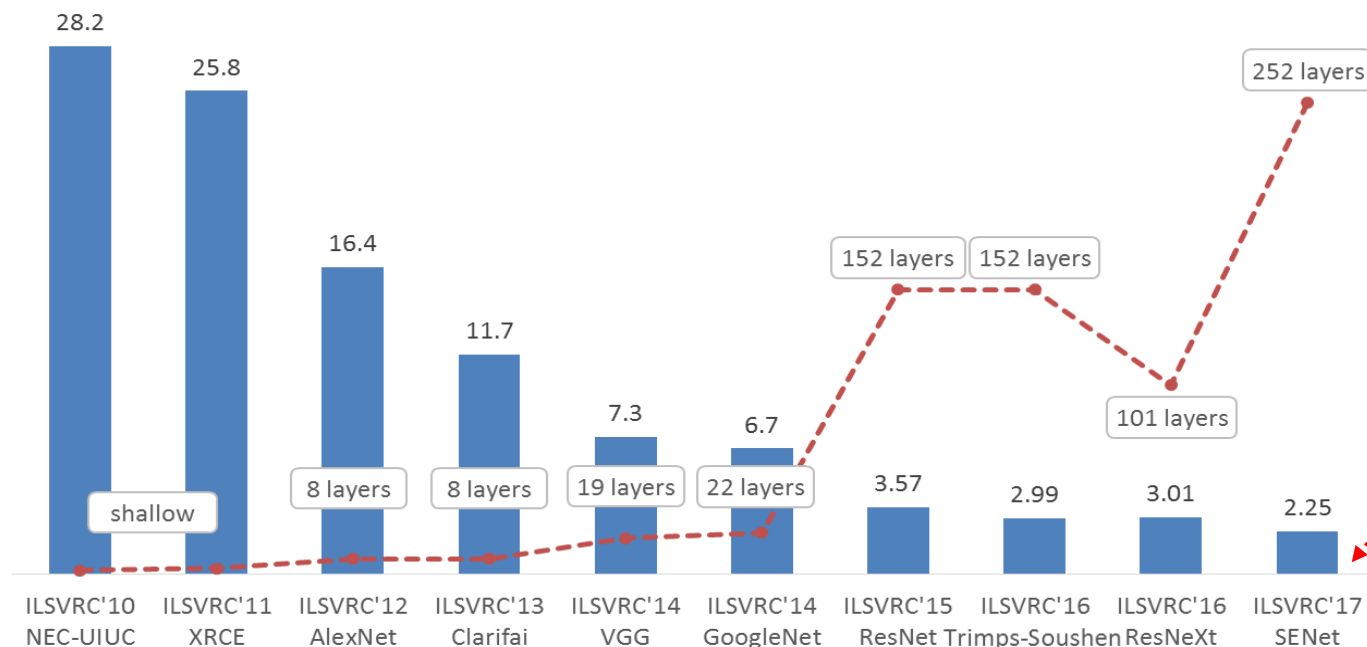
- ▶ 适合图像处理的卷积神经网络
- ▶ 基于CNN的图像分类算法
- ▶ 基于CNN的图像检测算法
- ▶ 近年来目标检测算法简介
- ▶ 序列模型：循环神经网络
- ▶ 序列模型：长短期记忆模型
- ▶ Transformer方法及应用
- ▶ 生成对抗网络GAN、Diffusion模型
- ▶ Driving Example
- ▶ 小结

基于CNN的图像分类算法

- 对**卷积神经网络**的研究可追溯至日本学者福岛邦彦提出的neocognition模型。在其1979 和1980年发表的论文中，福岛参照生物的视觉皮层（visual cortex）设计了以“neocognition”命名的神经网络。
- AlexNet使用卷积神经网络解决图像分类问题，在ILSVRC2012中取得获胜并大大提高了state-of-art的准确率。自此卷积神经网络在图像分类领域获得快速发展。
 - ImageNet大规模视觉识别比赛
 - ImageNet Large Scale Visual Recognition Competition

ImageNet大规模视觉识别比赛

- ImageNet Large Scale Visual Visual Recognition Competition, ILSVRC
- 图像分类领域最有影响力的学术竞赛之一，提供1000种分类的有标签样本
 - 由斯坦福大学李飞飞教授主导，包含了超过1400万张全尺寸的有标记图片
- TOP5的误差

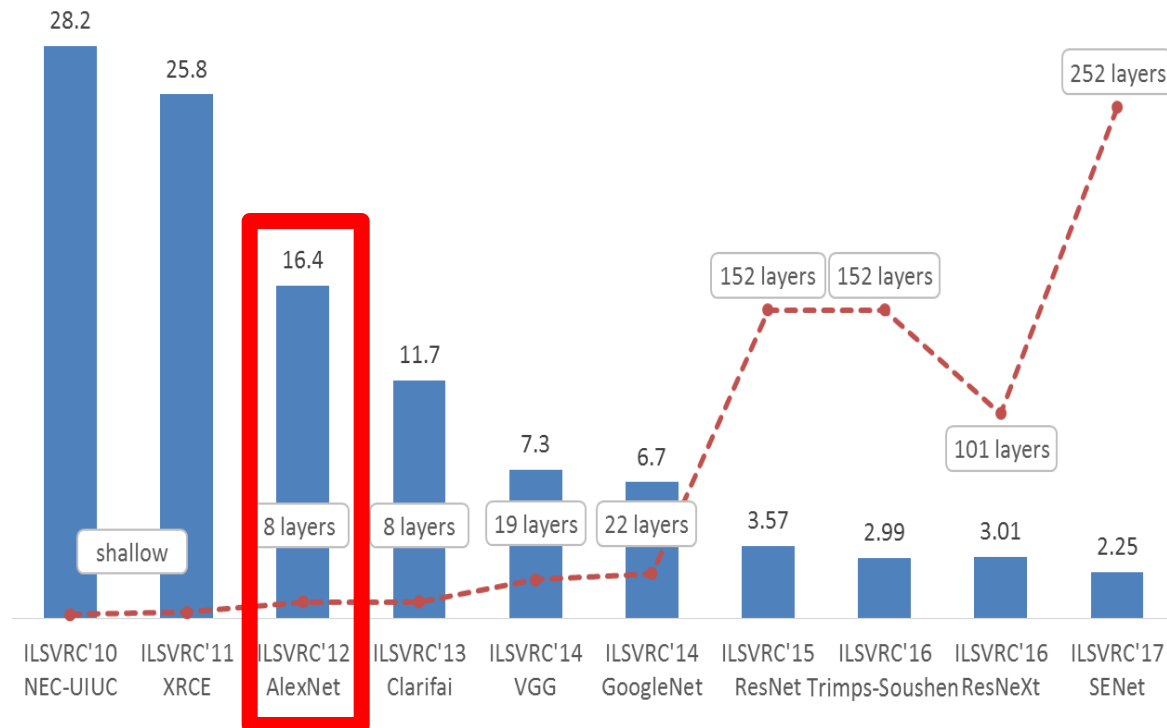


正式
结束

ImageNet图像分类大赛中历年的Top-5错误率

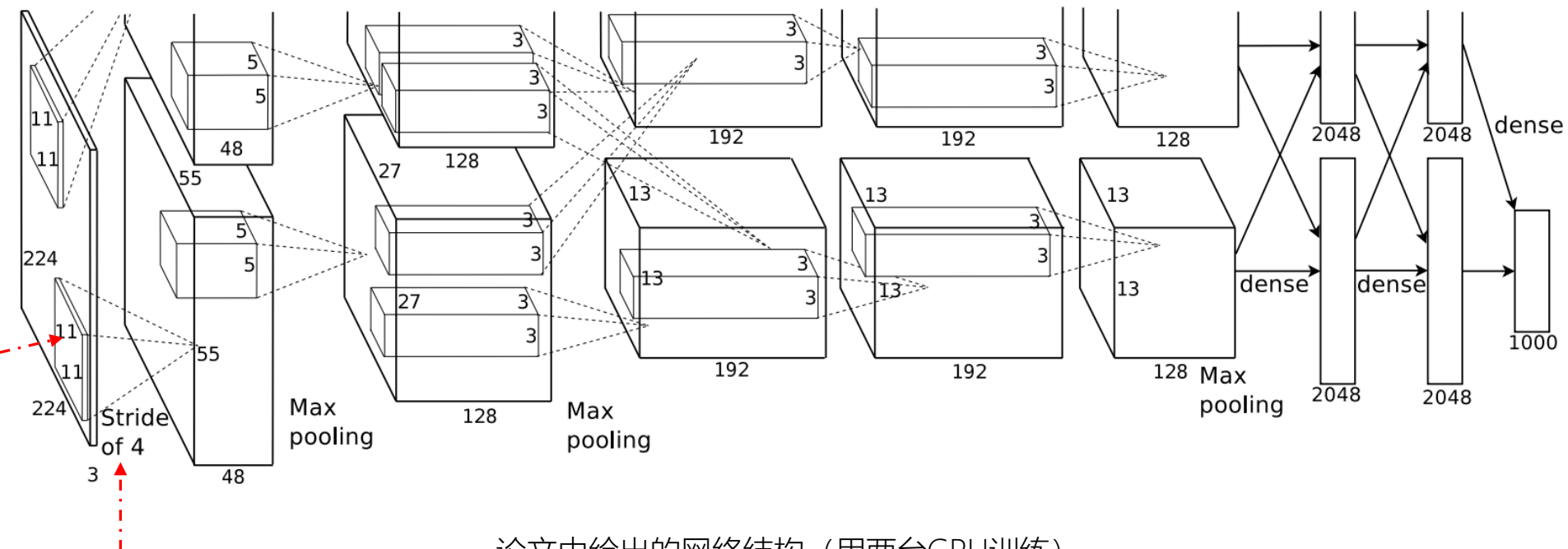
基于CNN的图像分类算法

- AlexNet
- VGG
- Inception 系列
- ResNet



AlexNet

- Paper: ImageNet Classification with Deep Convolutional Neural Networks (2012)
- Author: Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton
- Test: error rate on ImageNet, top1: 37.5%, top5: 16.4%



论文中给出的网络结构（用两台GPU训练）

AlexNet

What's New? AlexNet有哪些窍门

层号	层名称	输入大小	卷积核或池化窗口大小	输入通道数	输出通道数	步长
1	Conv	224 × 224	11 × 11	3	96	4
	ReLU	55 × 55	—	96	96	1
	LRN	55 × 55	—	96	96	1
	MaxPool	55 × 55	3 × 3	96	96	2
2	Conv	27 × 27	5 × 5	96	256	1
	ReLU	27 × 27	—	256	256	1
	LRN	27 × 27	—	256	256	1
	MaxPool	27 × 27	3 × 3	256	256	2
3	Conv	13 × 13	3 × 3	256	384	1
	ReLU	13 × 13	—	384	384	1
4	Conv	13 × 13	3 × 3	384	384	1
	ReLU	13 × 13	—	384	384	1
5	Conv	13 × 13	3 × 3	384	256	1
	ReLU	13 × 13	—	256	256	1
	MaxPool	13 × 13	3 × 3	256	256	2
6	FC	—	—	9216	4096	1
	ReLU	—	—	4096	4096	1
	Dropout	—	—	—	—	—
7	FC	—	—	4096	4096	1
	ReLU	—	—	4096	4096	1
	Dropout	—	—	—	—	—
8	FC	—	—	4096	1000	1
	softmax	—	—	1000	1000	1

ReLU激活函数：训练中收敛速度更快；

LRN局部归一化：提升较大响应，抑制较小响应；

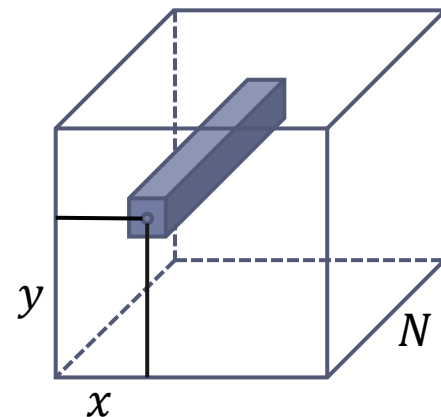
MaxPool：避免特征被平均池化模糊，提升特征鲁棒性；

Dropout：随机舍弃部分隐层节点，避免过拟合；

AlexNet

- Local Response Normalization (LRN)

- 局部响应归一化;



- 例如：有3个特征图，一个特征图是提对角线特征，一个特征图是提三角形特征，一个特征图是提正方形特征。
 - 在同一个位置上，第一个特征图里面就体现了它是不是参与到了对角线里面。在第二个特征图里面体现了它是不是参与到了三角形里面。第三个特征图里面体现了它有没有参与到一个正方形里面。
 - LRN就是对它做一个归一化，就是想看一下该位置到底是参与到了哪一个特征里面，参与的比较多，参与的比较多的就强化。（“嫌贫爱富”）
 - 局部归一化的动机：在神经生物学有一个概念叫做“侧抑制”，指的是被激活的神经元抑制相邻神经元。归一化 (normalization) 的目的是“抑制”，局部响应归一化就是借鉴“侧抑制”的思想来实现局部抑制。

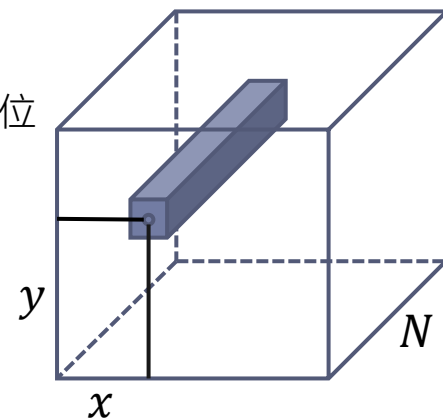
AlexNet

- Local Response Normalization (LRN)

- 局部响应归一化;

- LRN的输入是卷积层的输出特征图经过ReLU激活函数后的输出。
- 假设LRN的输入是N个特征图，LRN要对输入的n个相邻特征图上相同位置的点进行归一化处理，得到第i个输出特征图上位置 (r, c) 的值

$$b_{x,y}^i = a_{x,y}^i / \left(k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} (a_{x,y}^j)^2 \right)^\beta$$

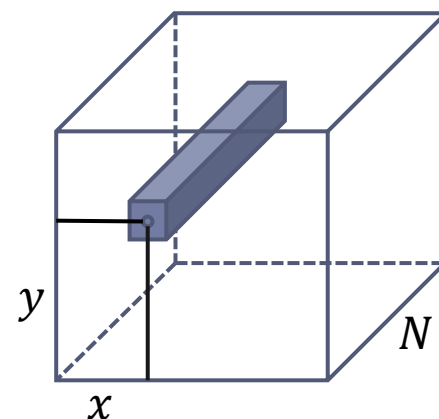


- LRN层模仿生物神经系统的侧抑制机制，对局部神经元的活动创建竞争机制，使得响应比较大的值相对更大，提高模型泛化能力。
 - 对图像的每个“位置”，提升高响应特征，抑制低响应特征；
 - 减少高激活神经元数量，提高训练速度，抑制过拟合；

AlexNet

- Local Response Normalization (LRN)
- 局部响应归一化;

$$b_{x,y}^i = a_{x,y}^i / \left(k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} (a_{x,y}^j)^2 \right)^\beta$$



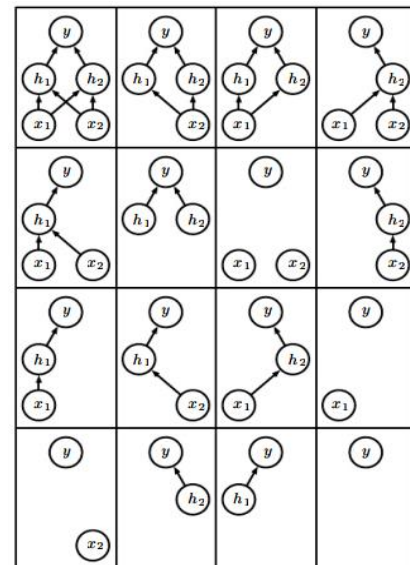
理论上, LRN可以把最显著的特征突出出来, 提升训练速度, 但是, 被后来研究者发现无明显效果, 故现在很少使用。

1. 一个图像里面的一个位置, 可能同时参与到了多个特征里面。LRN可能把三角形特征强化了, 正方形特征给丢掉了, 可能就适得其反。
2. 每一个特征值的分布可能是不一样的, 不同特征的量纲可能是不一样的。直接将不同特征进行归一化, 可能也未必合适。

AlexNet

- Dropout

- 随机丢弃部分神经元，**忽略一部分的特征检测器**
- 具体做法：在模型训练过程中，以一定概率随机地舍弃某些隐层神经元。在反向传播更新权重时，不更新与该神经元相关的权重；
- 但是与被舍弃神经元相关的权重得保留下来（只是暂时不更新），另一批样本输入时继续使用与该神经元相关的权重；
- **防止训练数据中复杂的共同适应，抑制过拟合；**
 - 在训练过程中会产生不同的训练模型(网络结构)，不同的训练模型也会产生不同的的计算结果。**最终的训练结果是不同模型的平均输出。**
 - 减弱了神经元节点间的联合，**降低了网络对单个神经元的依赖**，从而增强了泛化能力，解决了共同适应问题（即一个特征检测器需要依赖其他几个特定特征检测器）。



AlexNet

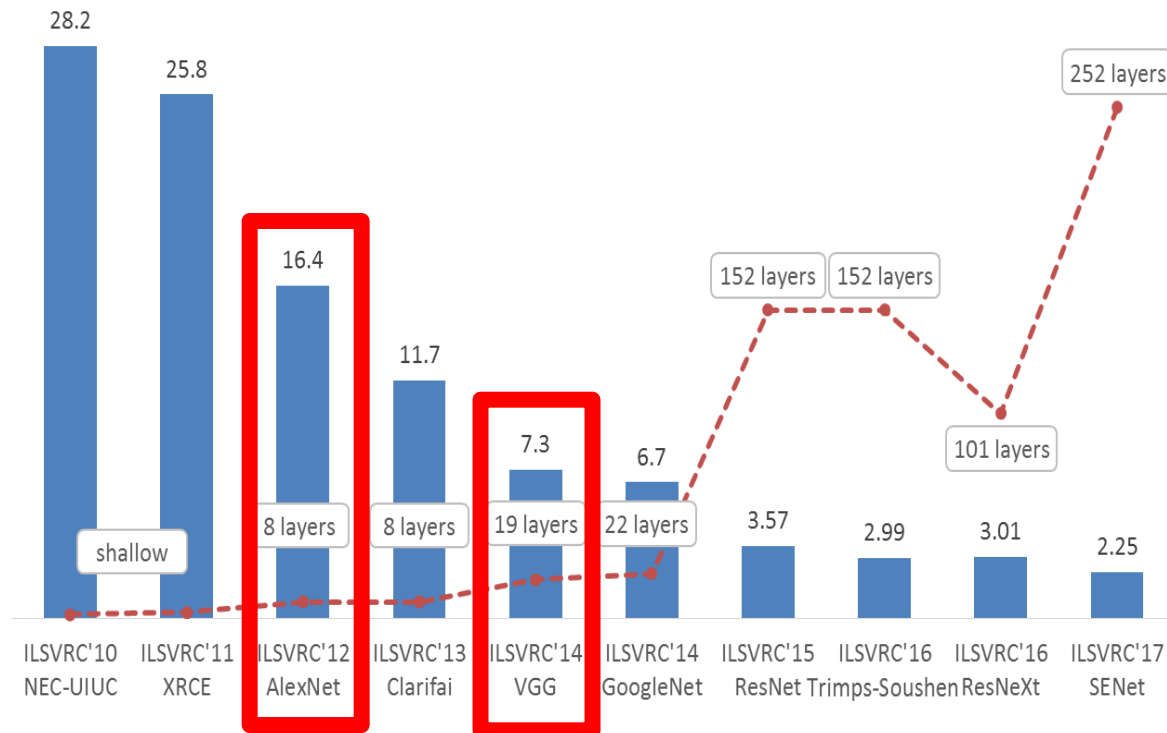
- AlexNet 成功的原因
 - 使用多个卷积层，有效提取图像特征；
 - ReLU帮助提高训练速度；
 - MaxPooling提高特征鲁棒性；
 - Dropout、数据增强扩大训练集，防止过拟合。
- ▶ ILSVRC, Top5从25%→16%，具有开创性

使用更多卷积层是否能进一步提升效果？



基于CNN的图像分类算法

- AlexNet
- VGG
- Inception 系列
- ResNet



使用更多卷积层是否能进一步提升效果？



VGG

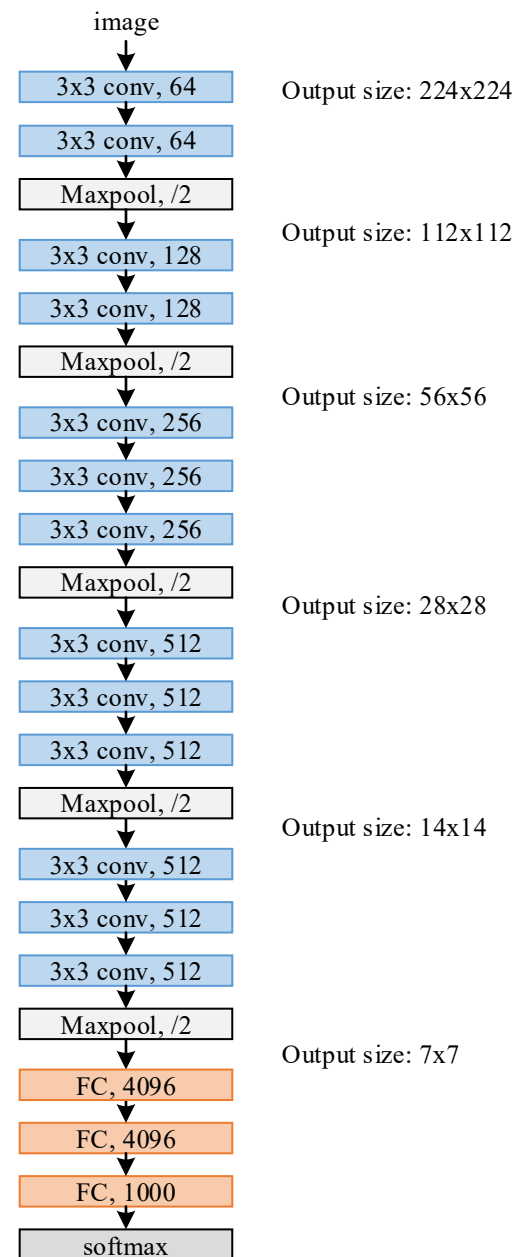
- Paper: **Very Deep Convolutional Networks** for Large-Scale Image Recognition (2014)
- Author: K. Simonyan, A. Zisserman
- Test: error rate on ImageNet, top1: 24.7%, top5: 7.5%

- **层数增加后的难度**

- 层数多了，有可能梯度爆炸，但更多的时候你可能会碰到**梯度消失**：在离损失函数很近的那一两层里面比较容易训练。但是16层、19层神经网络中输出比较远的层的权重，偏导梯度会很小，可能训了半天都训不动。

- **VGG16的核心思路**

- 从8层跳到16层可能训不动，但是可以一步一步来，先训一个比较浅的神经网络，比如先训一个11层的神经网络，如果11层的神经网络训好了，再以它为初始条件我去训一个16层或者是19层的神经网络。



VGG网络结构和训练思路

■ 由简单到复杂的网络结构

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

有没有VGG21、
22、30

训练VGG11 (A)

收敛后的前4个卷积层和后3个全连接层的权重作为更深神经网络的前4个卷积层和后3个全连接层权重的初始值，其余层的权重随机初始化；

训练更深神经网络

VGG网络结构和训练思路

● 实验结果

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512

作者尝试了不同的训练参数

Table 3: ConvNet performance at a single test scale.

ConvNet config. (Table 1)	smallest image side		top-1 val. error (%)	top-5 val. error (%)
	train (S)	test (Q)		
A	256	256	29.6	10.4
A-LRN	256	256	29.7	10.5
B	256	256	28.7	9.9
C	256	256	28.1	9.4
	384	384	28.1	9.3
	[256;512]	384	27.3	8.8
D	256	256	27.0	8.8
	384	384	26.8	8.7
	[256;512]	384	25.6	8.1
E	256	256	27.3	9.0
	384	384	26.9	8.7
	[256;512]	384	25.5	8.0

- A/A-LRN：加 LRN 准确率无明显提升；
- A/B/D/E：层数越多准确率越高；
- C/D：conv3x3 比 conv1x1 得到的准确率高；
- Multi-scale training 可明显提高准确率；

多尺度训练

- ImageNet数据集
 - 图像尺寸不固定、长宽比不固定
 - 如何训练模型？
- 数据增强的常用方法：Scale Jittering尺度扰动
 - 将原始图像的短边缩放到固定大小 S ，再随机裁切成 224×224 的图像
- Multi-scale:
 - 把 S 推广到一个区间 $[S_{\min}, S_{\max}]$ ，每次随机从区间选择一个值作为当前的 S 值
- 训练时的尺寸扰动（训练图像大小 $S \in [256, 512]$ ）得到的结果好于固定尺寸，这证实了通过尺度扰动进行的训练集增强确实有助于捕获多尺度图像统计

多尺度评估

- 训练阶段可以有多个尺度，测试阶段也可以有多个尺度，最后输出多个尺度的结果平均值
- 整体上，使用多尺度评估的错误率要小于单纯使用多尺度训练的情况

ConvNet config. (Table 1)	smallest image side		top-1 val. error (%)	top-5 val. error (%)
	train (S)	test (Q)		
B	256	224,256,288	28.2	9.6
C	256	224,256,288	27.7	9.2
	384	352,384,416	27.8	9.2
	[256; 512]	256,384,512	26.3	8.2
D	256	224,256,288	26.6	8.6
	384	352,384,416	26.5	8.6
	[256; 512]	256,384,512	24.8	7.5
E	256	224,256,288	26.9	8.7
	384	352,384,416	26.7	8.6
	[256; 512]	256,384,512	24.8	7.5

稠密评估

- 稠密评估：VGG中的稠密评估（Dense Evaluation）是指在测试时，对每个类别进行多次评估，以获得更准确的分类结果。
- 具体做法：全连接层替换为卷积层（第一FC层转换到 7×7 卷积层，最后两个FC层转换到 1×1 卷积层），最后得出一个预测的score map，再对结果求平均。

ConvNet config. (Table 1)	Evaluation method	top-1 val. error (%)	top-5 val. error (%)
D	dense	24.8	7.5
	multi-crop	24.6	7.5
	multi-crop & dense	24.4	7.2
E	dense	24.8	7.5
	multi-crop	24.6	7.4
	multi-crop & dense	24.4	7.1

- 多尺度优于稠密评估
- 两种方法互补
- 测试时，使用多尺度（multi-crop）与稠密评估(dense)结合的方法

VGG卷积-池化结构

● 规整的卷积-池化结构

• AlexNet: $11 \times 11, 5 \times 5$, stride=4.....

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input				$224 \times 224 \times 3$	
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool				$112 \times 112 \times 64$	
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool				$56 \times 56 \times 128$	
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool				$28 \times 28 \times 256$	
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool				$14 \times 14 \times 512$	
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool				$7 \times 7 \times 512$	

■ Conv

- 所有卷积层filter大小/stride/pad相同;
- filter=3*3, stride=1, pad=SAME;
 - pad=SAME: 补pad至输出图像大小等于输入图像大小
 - pad=VALID: pad=0;

■ Maxpool

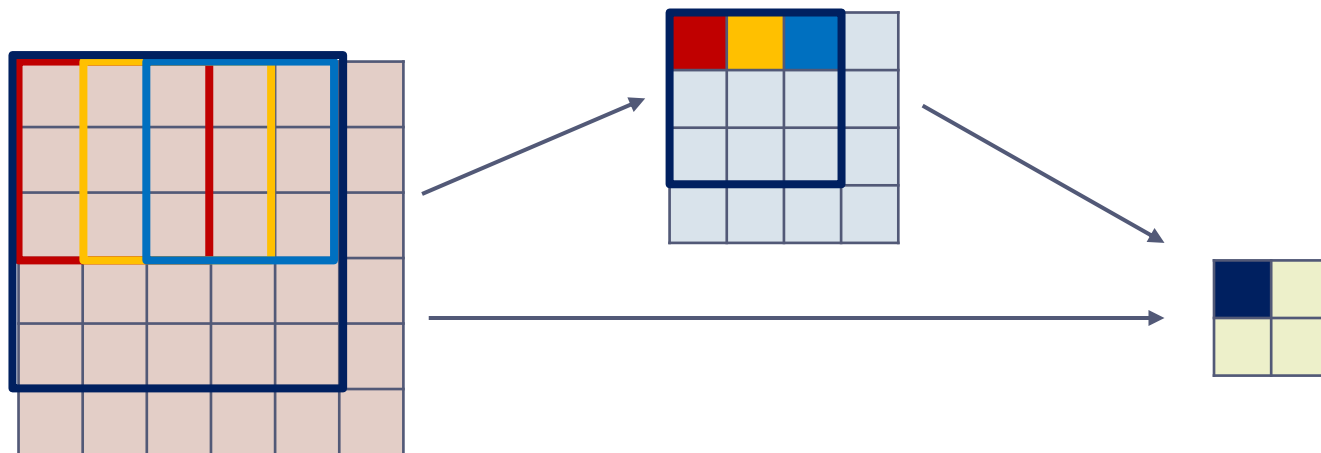
- 所有池化层filter大小/stride/pad相同;
- filter=2*2, stride=2, pad=0;

卷积层: 负责数据体深度变换
(控制特征图数量)

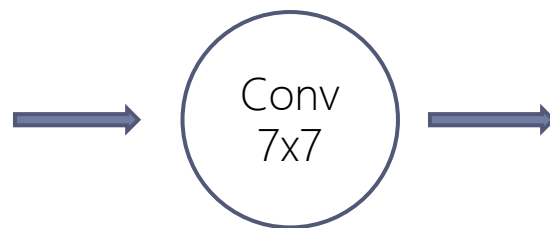
池化层: 负责数据体长宽变换
(控制特征图大小)

VGG卷积-池化结构

- 规整的<卷积-池化>结构
 - 多层小卷积比单层大卷积效果好;
 - 实验：对 VGG13(上表B)，使用5x5conv代替两层3x3conv，进行训练和测试；
 - 结果： 5x5conv网络比两个3x3conv网络top-1准确率低7%。
 - 原因：一个5x5conv和两个3x3conv的感受野大小相同；
 - 感受野 (Receptive Field)：卷积神经网络每一层输出的特征图上的像素点在输入图片上映射的区域大小。通俗讲，特征图上的一个点对应输入图上的区域。
 - 每个卷积层加入ReLU，两层3x3conv 决策函数的区分能力更强



相同感受野，多层网络权值更少



$$weight_count_1 = 7 \times 7 = 49$$



$$weight_count_2 = 3 \times 3 \times 3 = 27$$

VGG

- VGG成功的原因

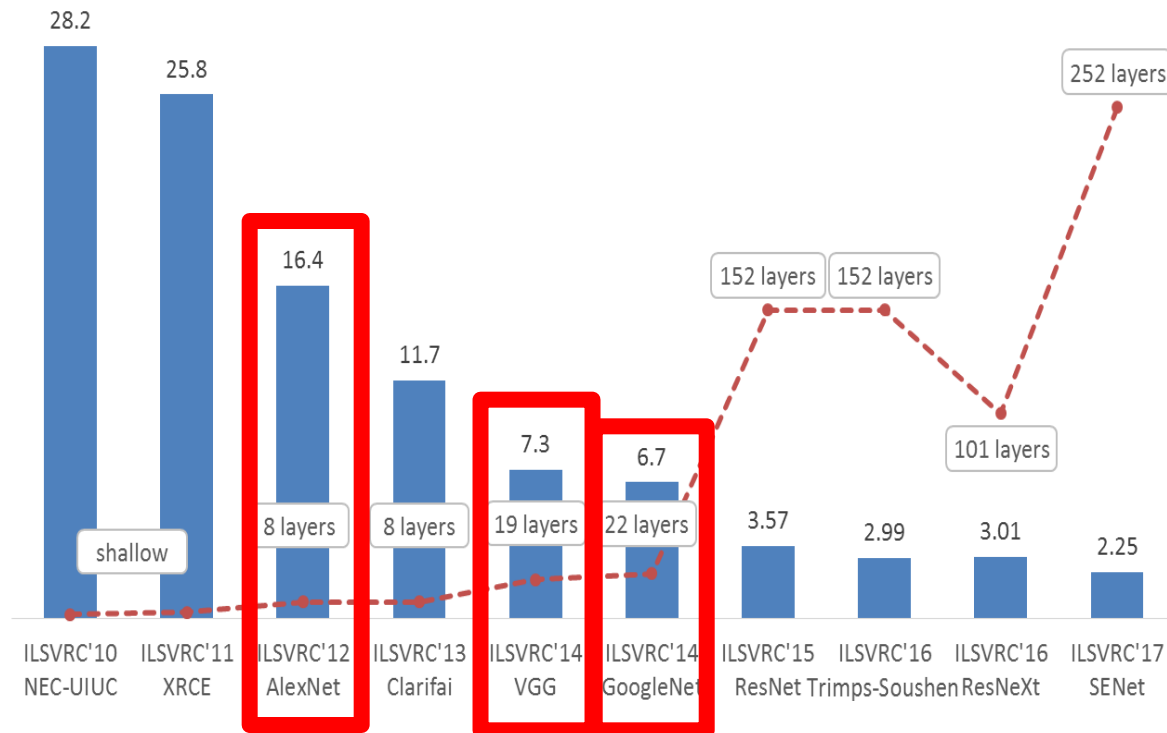
- 部分网络层参数的预初始化，提高训练收敛速度
- 更深的卷积神经网络，更多的卷积层和非线性激活函数，提升分类准确率
- 使用规则的多层小卷积替代大卷积，减少参数数量，提高训练收敛速度
- 多尺度训练和多尺度评估

卷积核还能不能更小？网络还能不能更深？



基于CNN的图像分类算法

- AlexNet
- VGG
- Inception 系列
- ResNet



AlexNet、VGG的不足

- ▶ 1.参数太多，容易过拟合，若训练数据集有限；
- ▶ 2.网络越大计算复杂度越大，难以应用；
- ▶ 3.网络越深，梯度越往后穿越容易消失（梯度弥散），难以优化模型。

Inception

Inception这个系列也是一个非常重要承上启下的工作，它上面承接了像VGG，后面也对于像Resnet这样的技术应该说是起到了很好的这种启发性的作用。

- Inception-v2, Inception-v3: Szegedy C, Vanhoucke V, Ioffe S, et al. Rethinking the Inception Architecture for Computer Vision[C]// CVPR, 2016:2818-2826.
- Inception-v4: Szegedy C, Ioffe S, Vanhoucke V, et al. Inception-v4, Peeking Inside the Black-Box of Deep Neural Networks on Learning, AAAI'2017.

更小的卷积核，
支持更深的网络

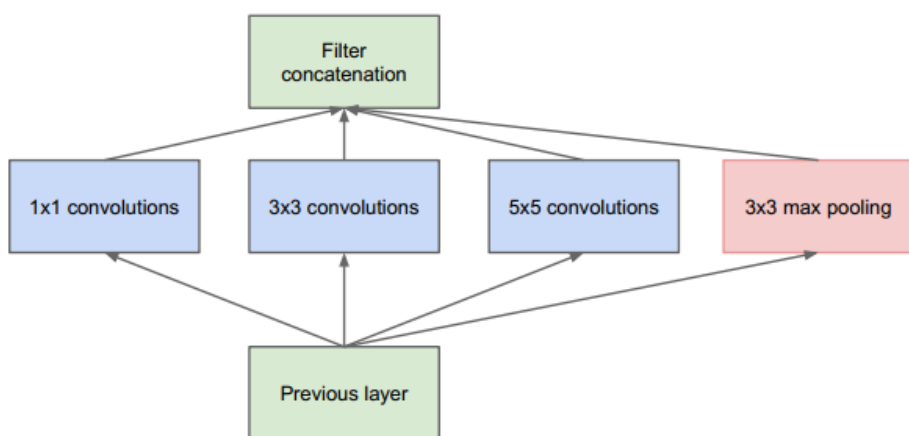
较好地缓解掉
梯度消失问题给

的卷积核，能够用更深的网络去获得更好的效果

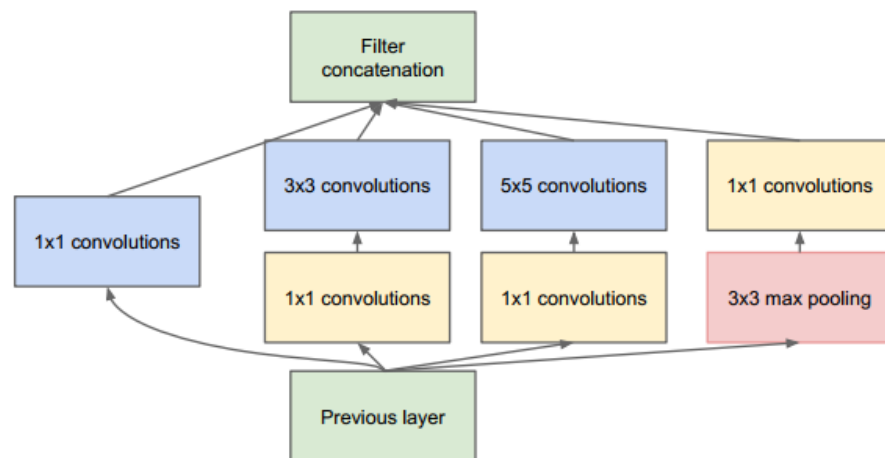
网络	主要创新	Top5错误率	网络层数
GoogLeNet	提出inception结构	6.67%	22
BN-Inception	提出 Batch Normalization, 用3x3代替5x5	4.82%	—
Inception-v3	将一个二维卷积拆成两个一维卷积，辅助分类器的全连接层做BN	3.5%	42
Inception-v4	inception模块化, 结合ResNet的跳转结构	3.08%	—

Inception-v1

- Inception 模块



(a) Inception module, naïve version



(b) Inception module with dimension reductions

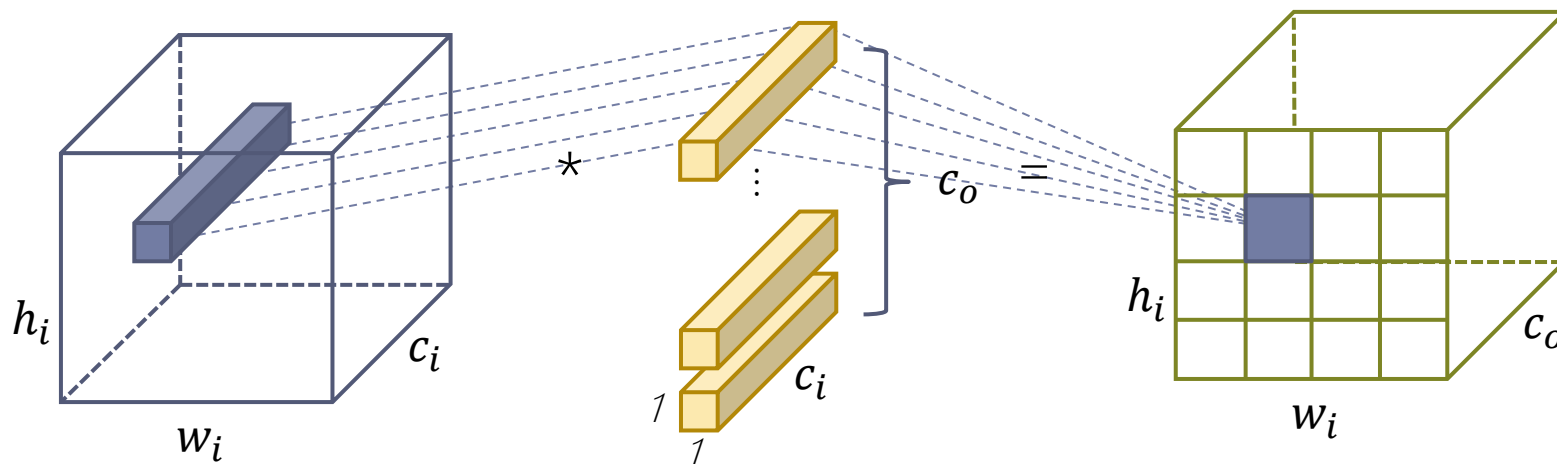
- Naïve version: 叠加多种尺寸的卷积层和池化层, 获得不同尺度的特征, 提高网络对不同尺寸特征的适应性; (缺点: 计算量大)
- Dimension reductions: a) 使用1x1的卷积层来缩减维度(减小channel), 形成“瓶颈层”, 减少参数, 降低计算量。 b) 在 1×1 卷积后加入ReLU, 增添非线性, 提高泛化能力。

Inception-v1



- 1x1卷积

- 计算：把输入特征图上相同位置的点做全连接处理，计算出输出特征图上同样位置的一个点。
- 作用：跨通道（跨不同特征图信息）聚合，进一步可以起到降维（或者升维）的作用，减少参数。

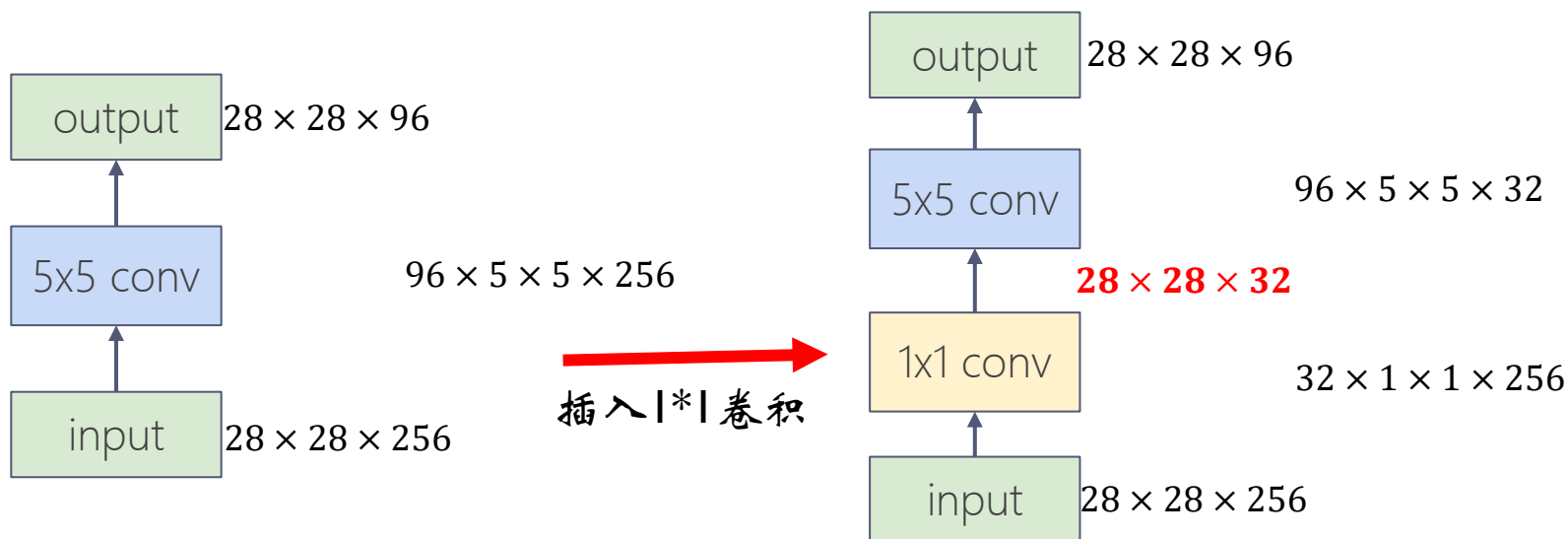


- 相当于在输入和输出之间做了一个特征上的全连接，提取得到非线性特征；
- 同时，当 $c_o < c_i$ 时，维度降低，参数减少；当 $c_o > c_i$ 时，维度升维
- 扩展：1x1卷积的思想：Network in Network：参数29M（Alexnet参数230M）

Inception-v1

- 1x1卷积

- 使用1x1卷积，形成“瓶颈层”，可有效减少计算量和参数数量；



乘法次数:

$$28 \times 28 \times 96 \times 5 \times 5 \times 256 \approx 4.8 \times 10^8$$

参数数量:

$$96 \times 5 \times 5 \times 256 \approx 6.1 \times 10^5$$

乘法次数:

$$28 \times 28 \times 32 \times 256 + 28 \times 28 \times 96 \times 5 \times 5 \times 32 \approx 6.7 \times 10^7$$

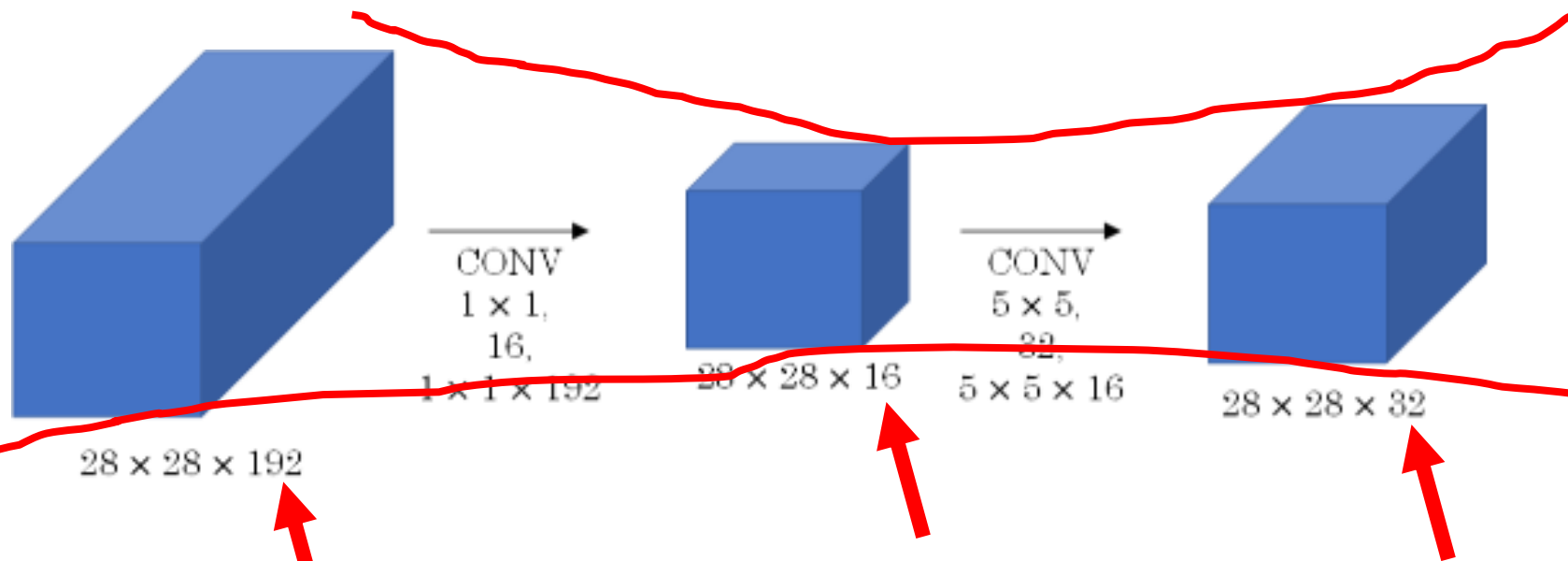
参数数量:

$$32 \times 256 + 96 \times 5 \times 5 \times 32 \approx 8.5 \times 10^4$$

Inception-v1

- 1x1卷积

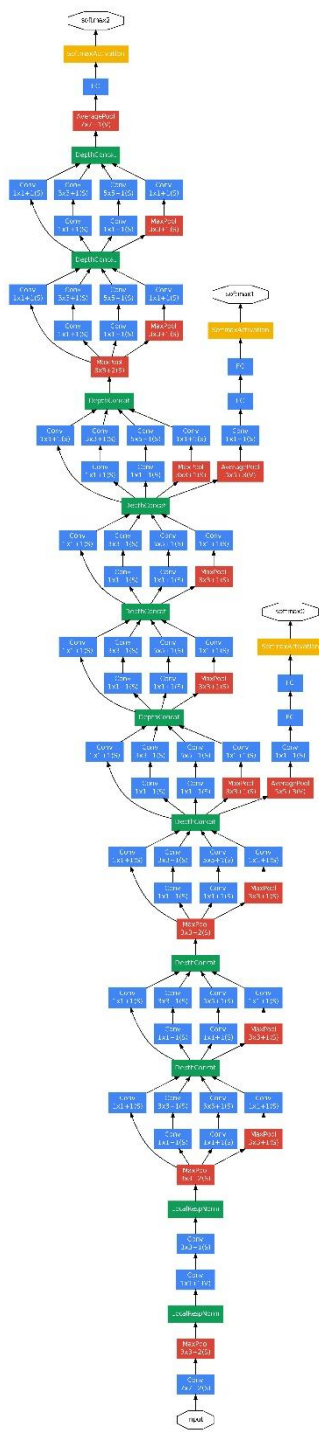
- 瓶颈层使用的是1*1的卷积神经网络。之所以称之为瓶颈层，是因为长得比较像一个瓶颈。
- 这种先降维度后升维度的瓶颈型结构十分经典，能够利用深层次提高抽象能力，同时节约计算量和参数的数量



Inception-v1

- GoogLeNet网络结构

- 多个Inception模块叠在+辅助分类网络+全连接层Softmax层



type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

Table 1: GoogLeNet incarnation of the Inception architecture

Inception-v1

● Inception 模块

在GoogLeNet第三层的inception module, 采用不同尺度的卷积核来处理问题。第二层到第三层的四个分支分别为:

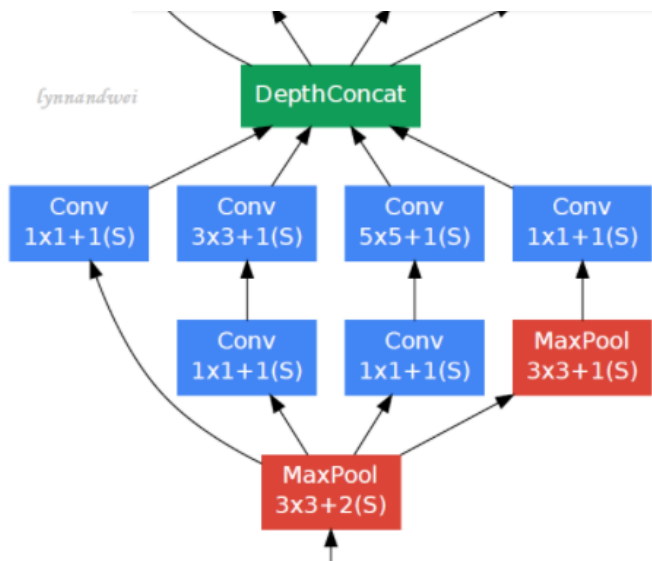
(1) 64个1x1的卷积核, 进行ReLU计算后得到28X28X64.

(2) 96个1X1的卷积核作为3x3卷积核之前的reduce, 变成28x28x96, 进行ReLU计算后, 再进行128个3x3的卷积, pad为1, 变为28x28x128.

(3) 16个1x1的卷积核作为5x5卷积核之前的reduce, 变为28x28x16, 进行ReLU计算后, 再进行32个5x5的卷积, pad为2, 变为28x28x32.

(4) pool层, 3x3的卷积核, pad为1, 输出为28x28x192, 进行32个1x1的卷积, 变为28x28x32.

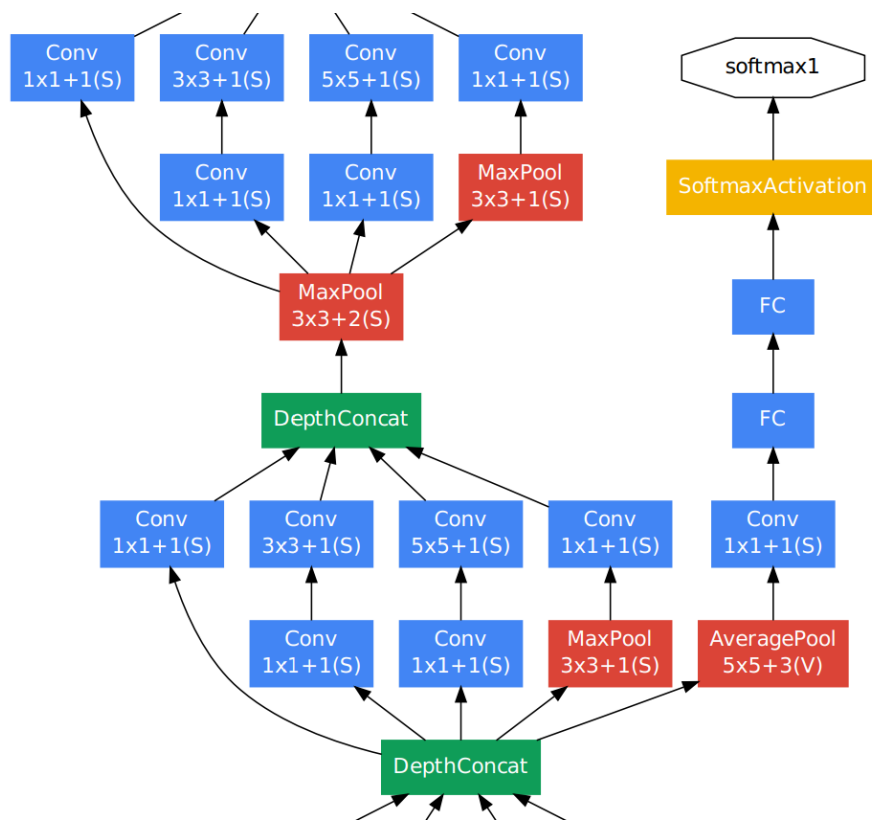
最后将四个结果进行拼接, 输出为28x28x256



type	patch size/ stride	output size	depth	#1x1	#3x3 reduce	#3x3	#5x5 reduce	#5x5	pool proj	params	ops
convolution	7x7/2	112x112x64	1							2.7K	34M
max pool	3x3/2	56x56x64	0								
convolution	3x3/1	56x56x192	2		64	192				112K	360M
max pool	3x3/2	28x28x192	0								
inception (3a)		28x28x256	2	64	96	128	16	32	32	159K	128M

Inception-v1

- Softmax 辅助分类网络



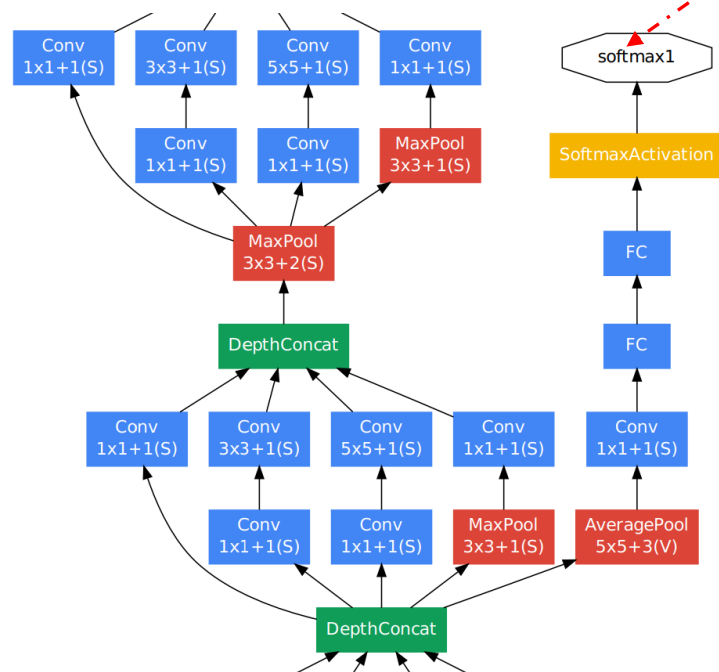
- 训练时，让中间某一层的输出经过softmax得到分类结果，并按较小的权重加到最终分类结果中。

Inception-v1

监工
月考
节点评估
中期考核



• Softmax 辅助分类网络



➤ 深层网络训练的难点是接近输入层的权重，导数非常小，很难训。

➤ Softmax辅助分类网络相当于是一个观察层，在这个旁路出来一个结果，这个结果会以比较小的权重加到最终分类结果。

➤ 辅助分类网络能够非常快地通过一个旁路的结果，对比较接近输入层的权重进行调整。防止多层神经网络训练过程中梯度消失。让模型去适应更深的网络。

➤ 推断时，softmax辅助分类网络会被去掉。

BN-Inception

- 使用 BatchNorm, 并在每个卷积层之后、激活函数之前插入 BN 层, 具体来讲:
 - BN-Inception在训练时同时考虑一批样本。
 - 在每个卷积层之后、激活函数之前插入一个特殊的**跨样本**的BN层, 用多个样本做归一化, 将输入归一化到加了参数的标准正态分布上。
 - **有效避免梯度爆炸或者梯度消失, 进而可以训练出很深的神经网络。**

采用BN的原因

- 首先，随着神经网络层数的增多，在训练过程中，各层的参数都在变化，因此每一层输入的分布都在变化，其分布会逐渐偏移，即内部协方差偏移。输入分布通常会向非线性激活函数的两端偏移，即靠近饱和区域，因此导致反向传播时梯度消失。
- 其次，神经网络训练的这个过程很大程度上就是为了学习样本数据的分布情况。那么一旦我们的训练数据跟测试数据的分布很不一样的话，网络泛化的能力、适应测试数据的能力就大大下降。
- 同时，如果我们每一批的训练数据，它的分布都不一样的话，网络在每次迭代的时候都要去学习和适应各种不同的分布，这就大大地降低整个网络训练的速度跟效益。
- 所以需要对训练数据、测试数据进行归一化，这就是Batch Normalization 一个核心的出发点。

BN-Inception

● BatchNorm

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

■ Normalize

- 将激活层的输入调整为标准正态分布（均值为0，方差为1）；激活函数的取值就会靠近中间区域。
- 从而激活层输入分布在激活函数敏感部分，输入有小变化就能使损失函数有较大的反应，避免梯度消失，加快训练速度。

■ 缩放和偏移 (Scale and shift)

- 标准化后的输入使得网络的表达能力下降；（因为都是正态分布了）
- 为保持网络的表达能力，增加两个可训练参数 γ 和 β 。动态适应各层具体的情况。

BN-Inception

● Batch和mini-batch

● Batch 批处理

- 一次处理一批数据，然后梯度下降。计算量大，速度慢。

● 随机梯度下降：

- 每次随机去1个样本，随机梯度下降。速度快但是收敛性差。

● Mini-batch（折中手段）

- 将数据分成若干批，按批来更新参数（小批）；小批数据，不会跑偏、计算量可控

● Batch size 批大小

- Mini-batch中的数据量

- Yann LeCun: Training with large minibatches is bad for your health. More importantly, it's bad for your test error. Friends don't let friends use minibatches larger than 32

BN-Inception

- BatchNorm 效果

- BN 可替代 LRN / Dropout / L2 Normalization;
- 可提高收敛速度、训练速度;
- 可选择更高的学习率, 方便调参;

Batch Normalization

基本上成了深度学习算法训练中必不可少的一种技术

Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
BN-Baseline	$13.3 \cdot 10^6$	72.7%
BN-x5	$2.1 \cdot 10^6$	73.0%
BN-x30	$2.7 \cdot 10^6$	74.8%
BN-x5-Sigmoid		69.8%

- -x5 表示学习率设为 inception 初始学习率的5倍。

Figure 3. For Inception and the batch-normalized variants, the number of training steps required to reach the maximum accuracy of Inception (72.2%), and the maximum accuracy achieved by the network.

Inception-v3

- Factorization (因式分解) 思想

- 将 3x3 卷积拆分成 1x3 和 3x1 卷积，感受野相同，但是更加高效；
- 减少参数数量，同时通过非对称的卷积结构拆分增加特征多样性（可以得到矩形的卷积结构）；
- 浅层用分解效果不好，一般在中间层，特征图尺寸12-20左右，效果好

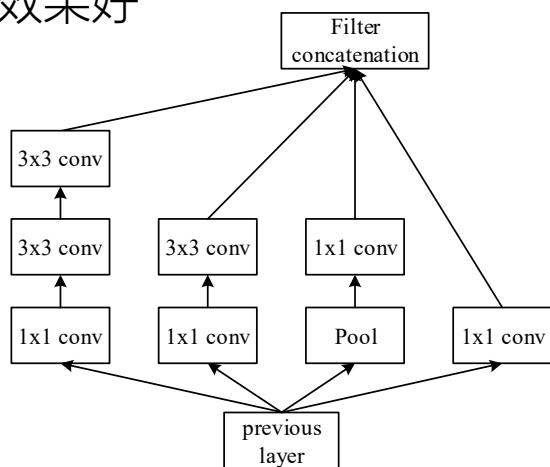
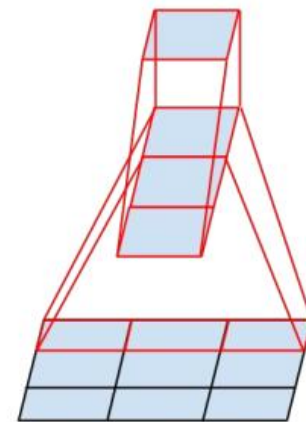


Figure 5

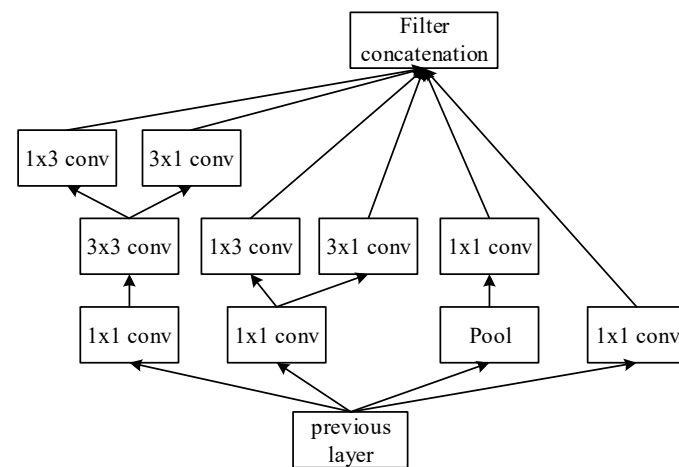


Figure 7

Inception-v3

- 网络结构

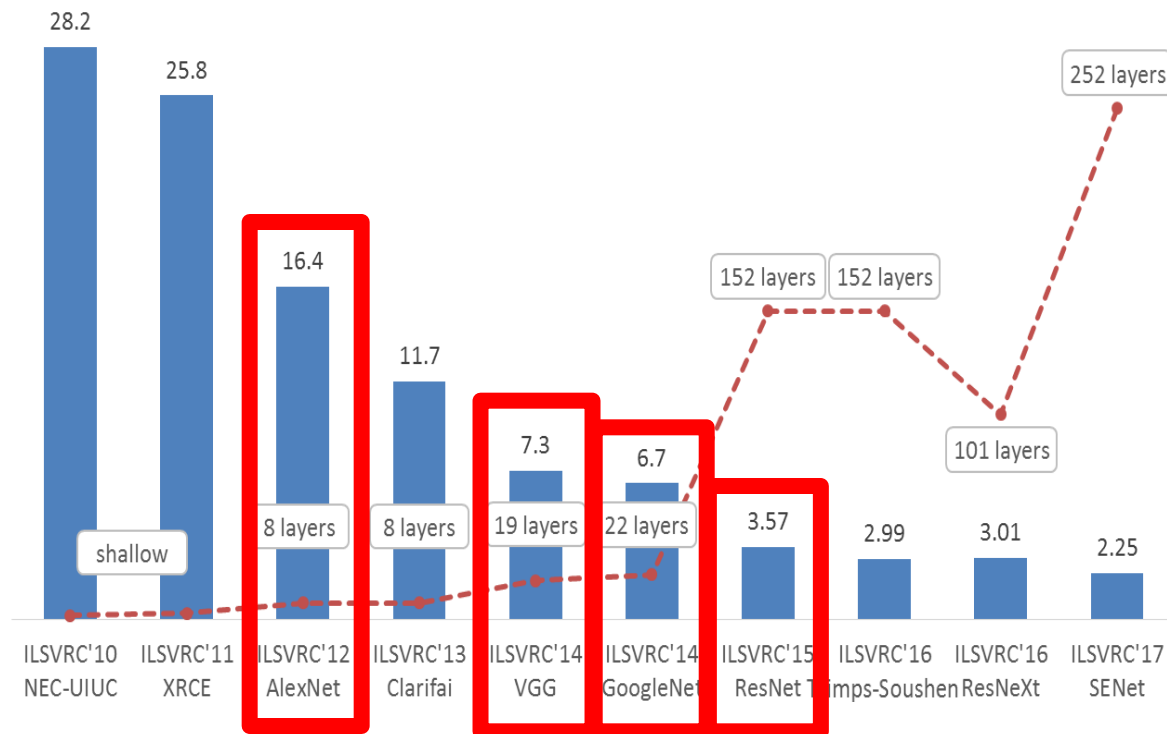
- 卷积核原来越小：GoogLeNet中7x7卷积拆分成3x3卷积，3x3又拆成了1x3、3x1
- 卷积层和辅助分类器的全连接层做BN

type	patch size/stride or remarks	input size
conv	$3 \times 3 / 2$	$299 \times 299 \times 3$
conv	$3 \times 3 / 1$	$149 \times 149 \times 32$
conv padded	$3 \times 3 / 1$	$147 \times 147 \times 32$
pool	$3 \times 3 / 2$	$147 \times 147 \times 64$
conv	$3 \times 3 / 1$	$73 \times 73 \times 64$
conv	$3 \times 3 / 2$	$71 \times 71 \times 80$
conv	$3 \times 3 / 1$	$35 \times 35 \times 192$
3×Inception	As in figure 5	$35 \times 35 \times 288$
5×Inception	As in figure 6	$17 \times 17 \times 768$
2×Inception	As in figure 7	$8 \times 8 \times 1280$
pool	8×8	$8 \times 8 \times 2048$
linear	logits	$1 \times 1 \times 2048$
softmax	classifier	$1 \times 1 \times 1000$

Inception系列把神经网络的深度从VGG的16层、19层又进一步拓展到22层甚至更多，而且准确度有提升

基于CNN的图像分类算法

- AlexNet
- VGG
- Inception 系列
- ResNet



ResNet

- Paper: Deep Residual Learning for Image Recognition (2015)
- Author: Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun
- Test: error rate on ImageNet, top5: 3.57% (resnet152)

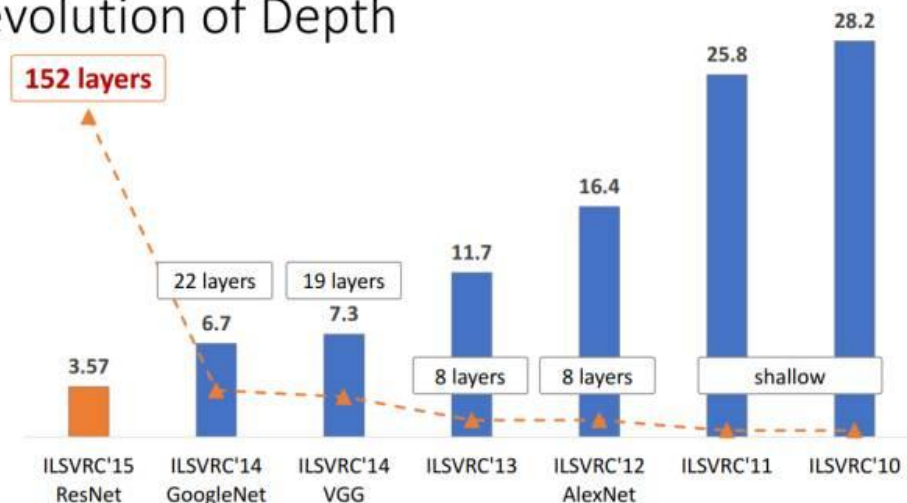
微软研究院华人科学家

ResNets @ ILSVRC & COCO 2015 Competitions

• 1st places in all five main tracks

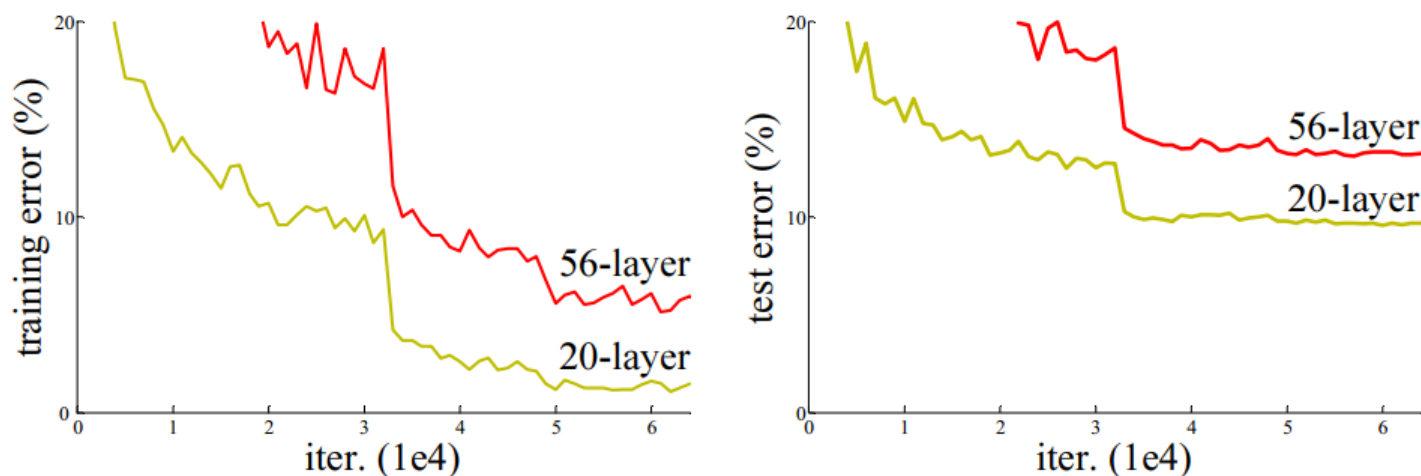
- ImageNet Classification: “Ultra-deep” 152-layer nets
- ImageNet Detection: 16% better than 2nd
- ImageNet Localization: 27% better than 2nd
- COCO Detection: 11% better than 2nd
- COCO Segmentation: 12% better than 2nd

Revolution of Depth



ResNet

- 问题：卷积层堆积就能提升图像分类准确率吗？
- 实验：分别用20层和56层卷积神经网络在cifar10数据集上进行训练和测试，发现更深的网络错误率更高，在ImageNet数据集上也同样如此。

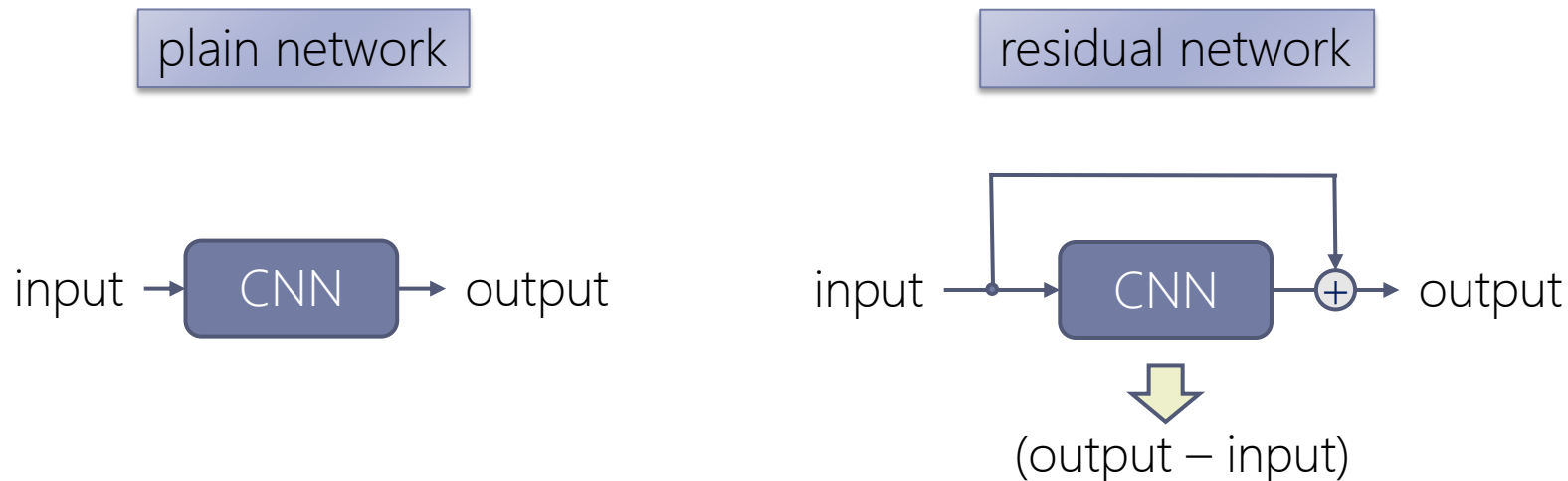


- 原因：梯度消失？ No, 使用BatchNorm可有效缓解梯度消失；
过拟合？ No, 更深的网络在**训练集**上的误差同样更高；

神经网络退化：收敛到极值点而非最优值，无法进一步训练，误差大。

ResNet

- 什么是“残差”

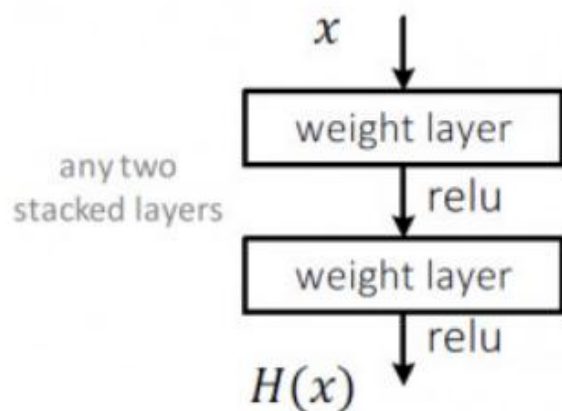


- Plain network(普通网络): 直接用多项式拟合输出;
- Residual network(残差网络): 增加了从输入到输出的直联, 卷积拟合的是输出与输入的差 (残差)。
- **??? : 输入和输出为什么可以去做减法呢?**
- 由于输入和输出都做了BN, 都符合正态分布, 因此输入和输出可以做减法。
- **优点: 在解附近时权重的反应更灵敏, 更容易学习获得最优解。**

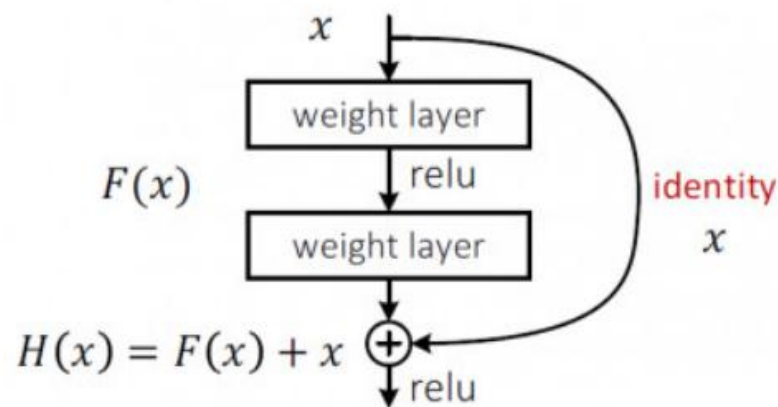
ResNet

- 残差块

- Plain net



- Residual net



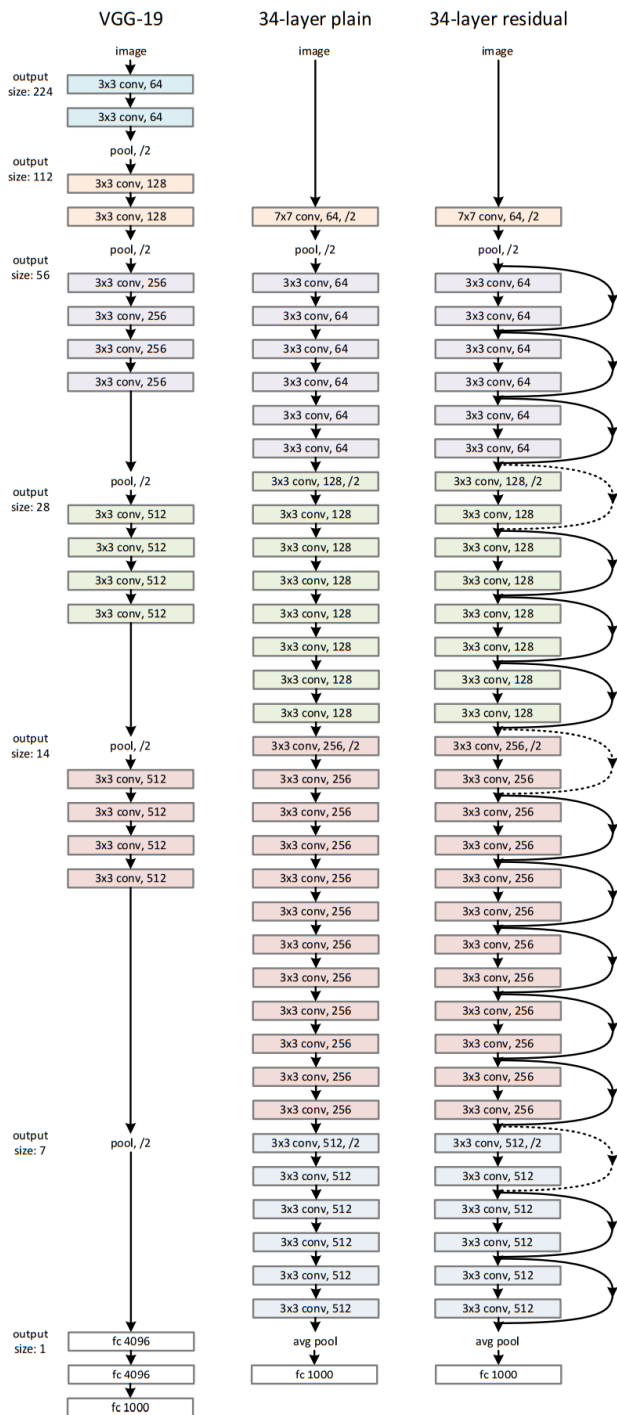
- $H(x) = F(x) + x$

- 残差网络在训练时更容易收敛

$H(x)$ 偏导至少有一个1。表明残差网络可以无损的传播梯度，不会导致梯度消失，所以残差网络的训练会更容易。

ResNet

- 残差块如何解决神经网络退化问题?
 - AlexNet等深度学习网络学习 $y=f(x)$,比较困难, 层次增加后, 导致神经网络退化;
 - ResNet核心是让较深的网络拥有恒等映射的能力, 在加深网络的时候, 保证深层网络和浅层网络持平
 - 学习目标: $F(x) = H(x) - x$, 让 $F(x)$ 逼近0
 - 只需要让残差函数 $f(x)=0$, 而学习 $f(x)=0$ 比直接学习 $y=f(x)$ 要简单多
 - 简单地讲, 因为网络权重的初始化一般偏向于0, 前向传播的计算往往是类似于 $y = g(wx + b)$ 的函数, 当权重 w 很小的时候 $(wx+b)$ 理论上也会很小, 进而网络层学习残差 $f(x)=0$ 的更新参数能够更快收敛。



● ResNet网络结构：将残差块应用到普通网络

■ 第一步：改造VGG得到plain-network

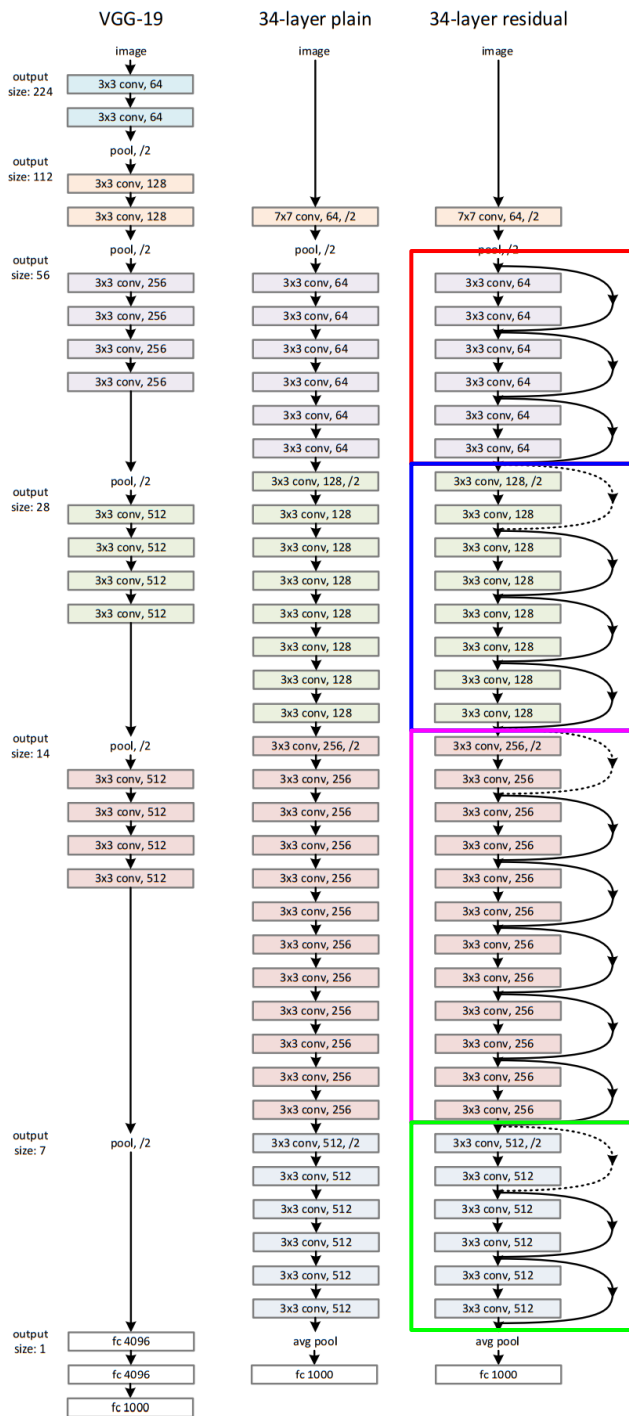
- plain-network：无跳转连接的普通网络；
- 基本全部由卷积层构成：filter=3*3, stride=1, pad=SAME；
- 特征图尺寸的减小由 stride=2 的卷积层完成；
- 原则：若特征图尺寸不变，则特征数量也不变；若特征图尺寸减半，则特征图数量翻倍，保持网络层的复杂度

■ 第二步：增加跳转连接得到 resnet

- 实线：特征图尺寸和特征数量不变，直接相连；
- 虚线：特征图尺寸减半，特征图数量翻倍；
- 特征图数量翻倍（虚线），输入输出的维度不一致，输入到输出的直联有两种方法：

a. 恒等映射：不够的特征补0(不引入额外参数)

b. 采用新的映射（**projection shortcut**）：特征数量翻倍的1x1卷积做映射，卷积的权重经过学习得到，会引入额外参数；



残差块的数量

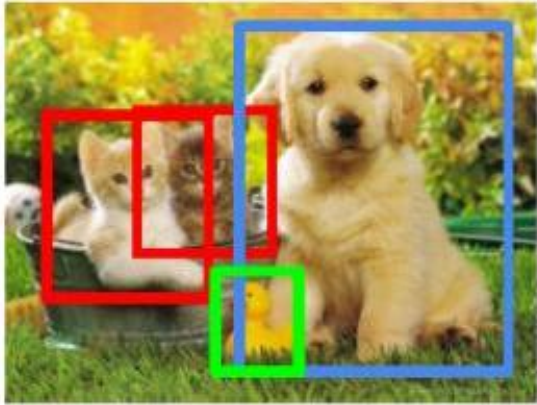
layer name	output size	18-layer	34-layer	50-layer	101-layer
conv1	112×112			7×7, 64, stride 2	
				3×3 max pool, stride 2	
conv2_x	56×56	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1			average pool, 1000-d fc, softmax	
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9

提纲

- ▶ 适合图像处理的卷积神经网络
- ▶ 基于CNN的图像分类算法
- ▶ 基于CNN的图像检测算法
- ▶ 近年来目标检测算法简介
- ▶ 序列模型：循环神经网络
- ▶ 序列模型：长短期记忆模型
- ▶ Transformer方法及应用
- ▶ 生成对抗网络GAN、Diffusion模型
- ▶ Driving Example
- ▶ 小结

图像检测算法

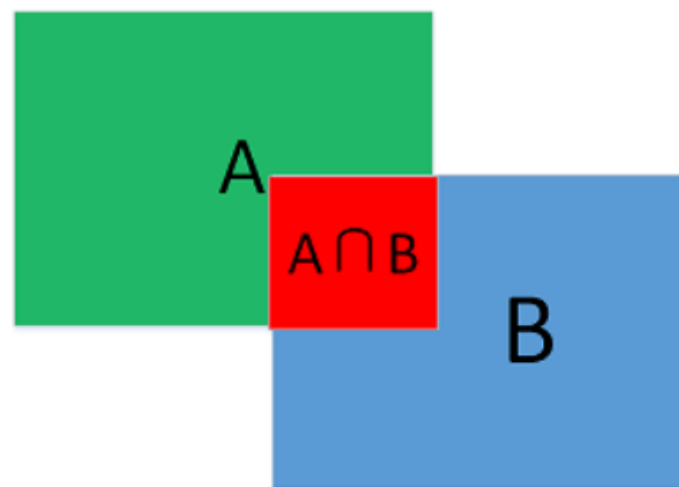
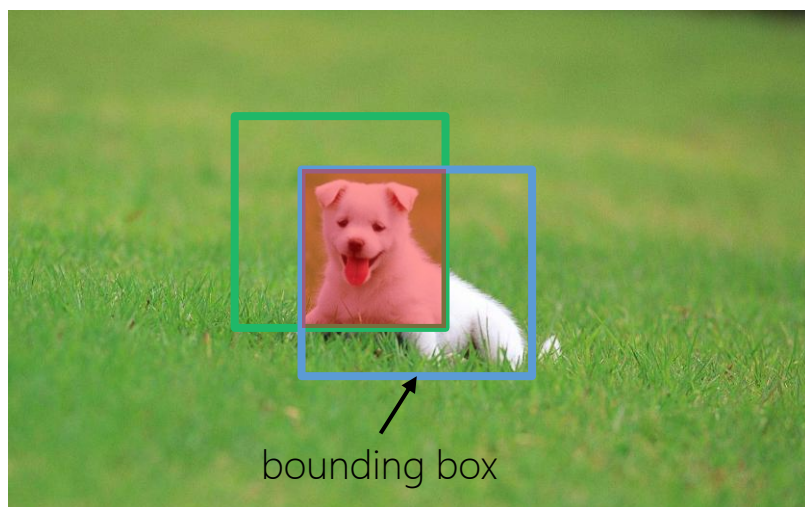
我们不仅要知道图象里面有猫，我们还知道这个猫在什么地方？

	分类	定位+分类	物体检测
图示	 CAT	 CAT	 CAT, DOG, DUCK
输入	single and big object	single and big object	multi and small object
输出	label	label & bounding box	multi label & bounding box
评价	precision(top1/top5)	IoU(交并比)	mAP(Mean Average Precision)

评测指标——IoU

- IoU(交并比)

- 用于衡量定位准确度，一般 $\text{IoU} \geq 0.5$ 可认为定位成功 (true detection);



$$IOU = \frac{A \cap B}{A \cup B}$$

评测指标——mAP

- mAP(mean Average Precision平均精度均值)
 - 在计算机视觉领域，用于衡量模型在测试集上检测精度的优劣程度；
 - 综合考虑检测结果的召回率/查全率和精度/查准率，mAP值越高表示检测结果越好；
- mAP计算原理

- ▶ 召回率/查全率(recall)：检测出的N个样本里选对的k个正样本占总的M个正样本的比例 k/M ；
- ▶ 精度/查准率(precision)：检测出的N个样本里选对的k个正样本比例 k/N ；
- ▶ **检测出的样本数N越多，召回率越高，精度越低；**

召回率 $\text{Recall} = k/M = TP/(TP+FN)$

精度 $\text{Precision} = k/N = TP/(TP+FP)$

测试结果	实际情况	
	正例(Positive)	反例(Negative)
正例(Positve)	真正例(True positive, TP)	假正例(False positve, FP)
反例(Negative)	假反例(False negative, FN)	真反例(True positive, TP)

评测指标——mAP

- 假设一个图像检测任务，共有5种类别，有100张图像作为测试集；
- 假设对于类A，100张测试图像中共有25个事先人为将类别标记为A的框；
- 假设算法对100张测试图像共检测出20个分类为A的候选框；

编号	置信度	标签
1	0.35	0
2	0.15	0
3	0.92	1
4	0.03	0
5	0.24	1
6	0.10	0
7	0.78	1
8	0.01	0
9	0.47	0
10	0.09	1
11	0.69	0
12	0.43	0
13	0.26	0
14	0.35	1
15	0.11	0
16	0.07	0
17	0.45	0
18	0.16	1
19	0.32	0
20	0.52	1

编号	置信度	标签
3	0.92	1
7	0.78	1
11	0.69	0
20	0.52	1
9	0.47	0
17	0.45	0
12	0.43	0
1	0.35	0
14	0.35	1
19	0.32	0
13	0.26	0
5	0.24	1
18	0.16	1
2	0.15	0
15	0.11	0
6	0.10	0
10	0.09	1
16	0.07	0
4	0.03	0
8	0.01	0

评测指标——mAP

例如：

confidence_threshold=0.5时：

第3, 7, 11, 20号框(score>0.5)被认为是positive，实际只有3, 7, 20号框(label=1)是true positive，那么此时 precision=3/4，又因为总共应该有25个类别为A的框，那么recall=3/25

confidence_threshold=0.2时：

共有12个框(score>0.2)被认为是positive，实际只有5个框是true positive，此时 precision=5/12，recall=5/25

阈值越小，选中样本越多，精度越低，召回率越高

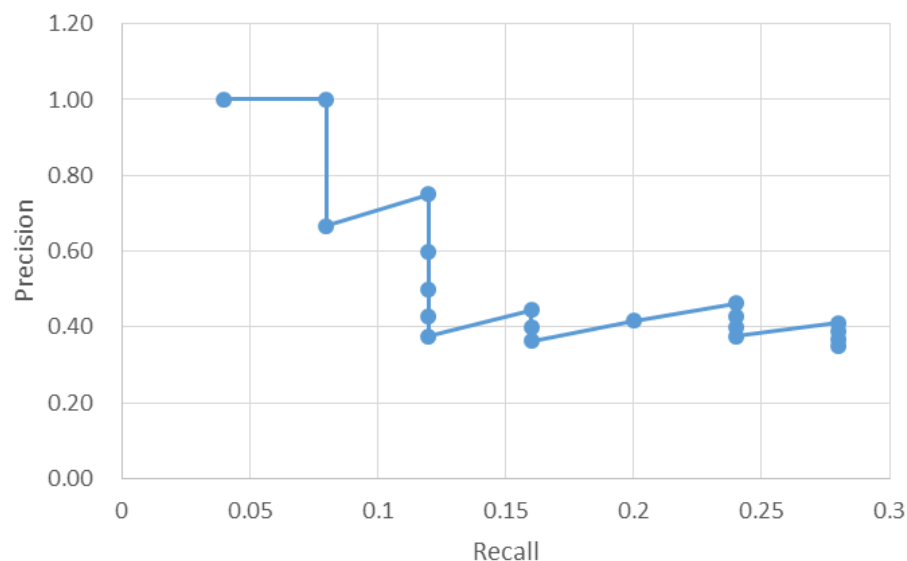
编号	置信度	标签
3	0.92	1
7	0.78	1
11	0.69	0
20	0.52	1
9	0.47	0
17	0.45	0
12	0.43	0
1	0.35	0
14	0.35	1
19	0.32	0
13	0.26	0
5	0.24	1
18	0.16	1
2	0.15	0
15	0.11	0
6	0.10	0
10	0.09	1
16	0.07	0
4	0.03	0
8	0.01	0

评测指标——mAP

编号	置信度	标签	k/M recall	k/N precision
3	0.92	1	1/25	1/1
7	0.78	1	2/25	2/2
11	0.69	0	2/25	2/3
20	0.52	1	3/25	3/4
9	0.47	0	3/25	3/5
17	0.45	0	3/25	3/6
12	0.43	0	3/25	3/7
1	0.35	0	3/25	3/8
14	0.35	1	4/25	4/9
19	0.32	0	4/25	4/10
13	0.26	0	4/25	4/11
5	0.24	1	5/25	5/12
18	0.16	1	6/25	6/13
2	0.15	0	6/25	6/14
15	0.11	0	6/25	6/15
6	0.10	0	6/25	6/16
10	0.09	1	7/25	7/17
16	0.07	0	7/25	7/18
4	0.03	0	7/25	7/19
8	0.01	0	7/25	7/20

precision, recall不是一个绝对的东西, 而是相对threshold而改变

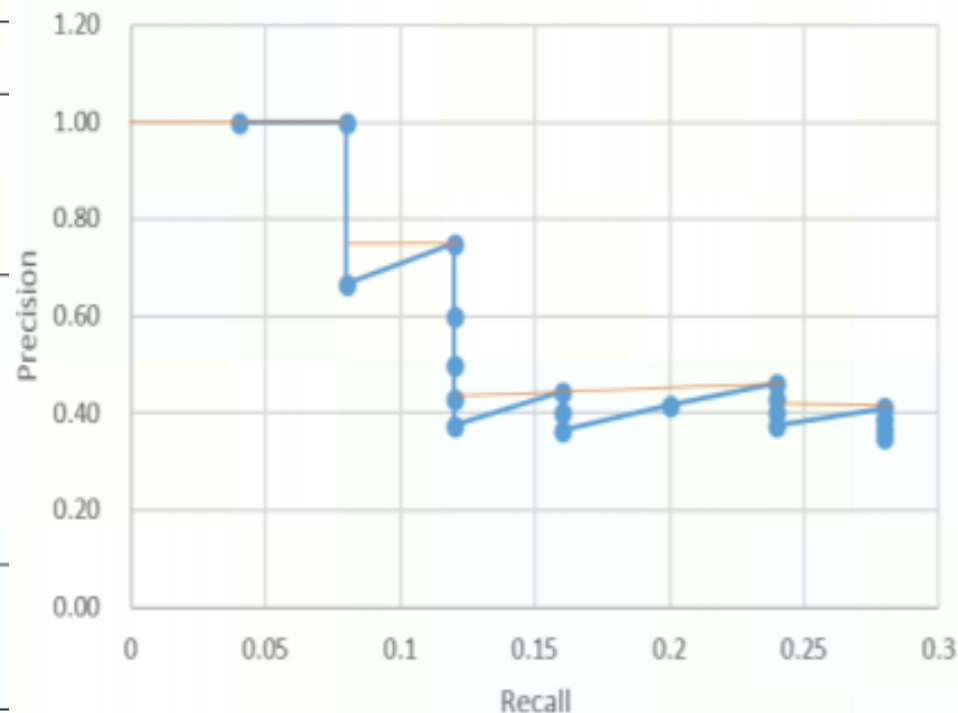
对于每个threshold, 都有 (precision, recall) 对, 得到precision和recall之间的曲线关系



- 类A的AP值计算方法(VOC2012): 对每个recall值 r , 取 $\tilde{r} \geq r$ 时的最大的 precision 求平均;
- 例如: recall=4/25时, 取precision=6/13; recall=6/25时, 取precision=6/13;

评测指标——mAP

编号	置信度	标签	Recall (r)	Precision	$\forall r \geq r$ 时的最大 Precision
3	0.92	1	1/25	1	1
7	0.78	1	2/25	1	1
11	0.69	0		2/3	
20	0.52	1	3/25	3/4	3/4
9	0.47	0		3/5	
17	0.45	0		3/6	
12	0.43	0		3/7	
1	0.35	0		3/8	
14	0.35	1	4/25	4/9	6/13
19	0.32	0		4/10	
13	0.26	0		4/11	
5	0.24	1	5/25	5/12	
18	0.16	1	6/25	6/13	
2	0.15	0		6/14	
15	0.11	0		6/15	
6	0.10	0		6/16	
10	0.09	1	7/25	7/17	7/17
16	0.07	0		7/18	
4	0.03	0		7/19	
8	0.01	0		7/20	



- AP:** 类A的AP值计算方法(VOC2012): 对每个recall值 r , 取 $\hat{r} \geq r$ 时的最大的 precision, 作为召回率 r 对应的精度, 然后计算精度-召回率曲线的面积作为平均精度AP;
- recall=4/25时, 取precision=6/13; recall=6/25时, 取precision=6/13;
- AP (类A) = $(1+1+3/4+6/13+6/13+6/13+7/17) / 25 = 0.1819$
- mAP:** C种类别的检测平均精度均值 $mAP = (\sum_{c=1}^C AP_c) / C$

评测指标——mAP

有人自己动手
写过mAP吗？

基于CNN的图像检测算法

- R-CNN系列
- YOLO
- SSD

目前，基于深度学习的目标检测算法大致分为两类：

- 1.两阶段 (two-stage) 算法：基于候选区域方法，先产生边界框，再做CNN分类(R-CNN系列)
- 2.一阶段 (one-stage) 算法：对输入图像直接处理，同时输出定位及其类别 (YOLO系列)

R-CNN系列

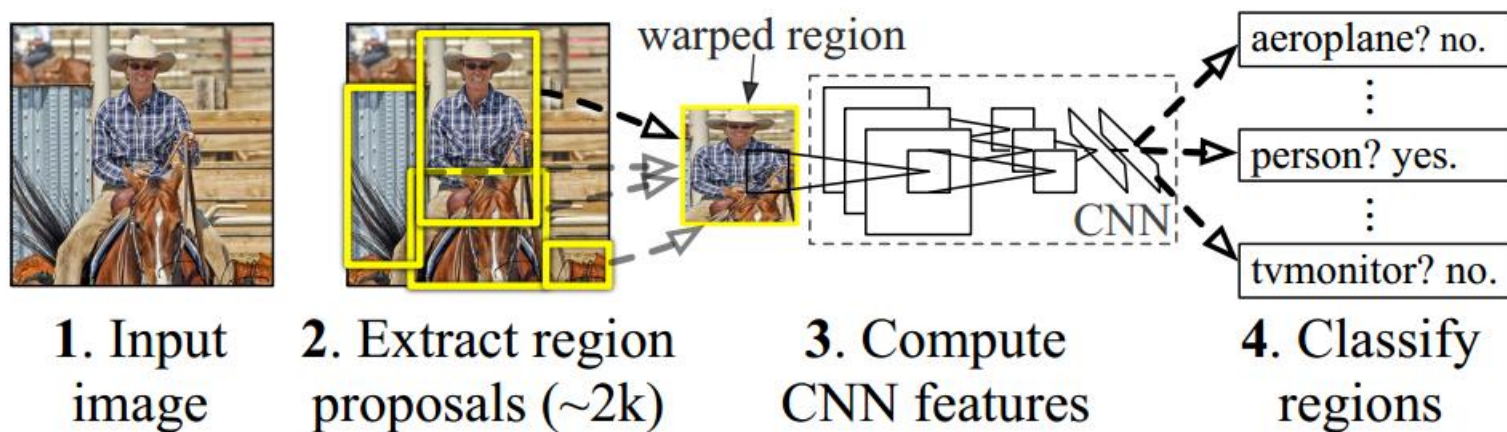
- R-CNN: Girshick R , Donahue J , Darrell T , et al. Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation[C]. 2014.
- Fast R-CNN: Girshick R. Fast R-CNN[J]. Computer Science, 2015.
- Faster R-CNN : Ren S, He K, Girshick R, et al. Faster R-CNN: towards real-time object detection with region proposal networks[C], 2015:91-99.

网络	主要特点	mAP(VOC2012)	单帧检测时间
R-CNN	结合RegionProposal区域提取和CNN特征提取; SVM分类, Bounding Box回归;	53.3%	50 s
Fast R-CNN	提出 ROI Pooling; softmax分类;	65.7%	2 s
Faster R-CNN	使用RPN(Region Proposal Network) 生成候选区域	67.0%	0.2 s

R-CNN

R-CNN的主要步骤:

- **候选区域提取**: 使用Selective Search从输入图片中提取2000个左右候选区域
- **特征提取**: 首先将所有侯选区域裁切缩放为固定大小, 再用 AlexNet(5conv+2FC)提取图像特征
- **线性分类**: 用特定类别的线性SVMs对每个候选区域做分类
- **边界框回归**: 用线性回归修正边界框的位置与大小, 其中每个类别单独训练一个边界框回归器



只有特征提取部分使用了神经网络技术

R-CNN

- 候选区域(Region Proposal)

- Uijlings J R R , K. E. A. van de Sande.... Selective Search for Object Recognition[J]. International Journal of Computer Vision, 2013, 104(2):154-171.
- **意义**：经典的目标检测算法使用滑动窗法依次判断所有可能的区域(穷举)，而 R-CNN 采用 Region Proposal 预先提取一系列较可能是物体的候选区域，之后仅在这些候选区域上提取特征，大大减少了计算量。
- **方法**：带多样性策略的**选择性搜索**(Selective Search)



R-CNN

- 候选区域提取 步骤

Algorithm 1: Hierarchical Grouping Algorithm

Input: (colour) image

Output: Set of object location hypotheses L

Obtain initial regions $R = \{r_1, \dots, r_n\}$ using Felzenszwalb and Huttenlocher (2004) Initialise similarity set $S = \emptyset$;

foreach *Neighbouring region pair* (r_i, r_j) **do**

 Calculate similarity $s(r_i, r_j)$;

$S = S \cup s(r_i, r_j)$;

while $S \neq \emptyset$ **do**

 Get highest similarity $s(r_i, r_j) = \max(S)$;

 Merge corresponding regions $r_t = r_i \cup r_j$;

 Remove similarities regarding r_i : $S = S \setminus s(r_i, r_*)$;

 Remove similarities regarding r_j : $S = S \setminus s(r_*, r_j)$;

 Calculate similarity set S_t between r_t and its neighbours;

$S = S \cup S_t$;

$R = R \cup r_t$;

Extract object location boxes L from all regions in R ;

- 层次化分组算法

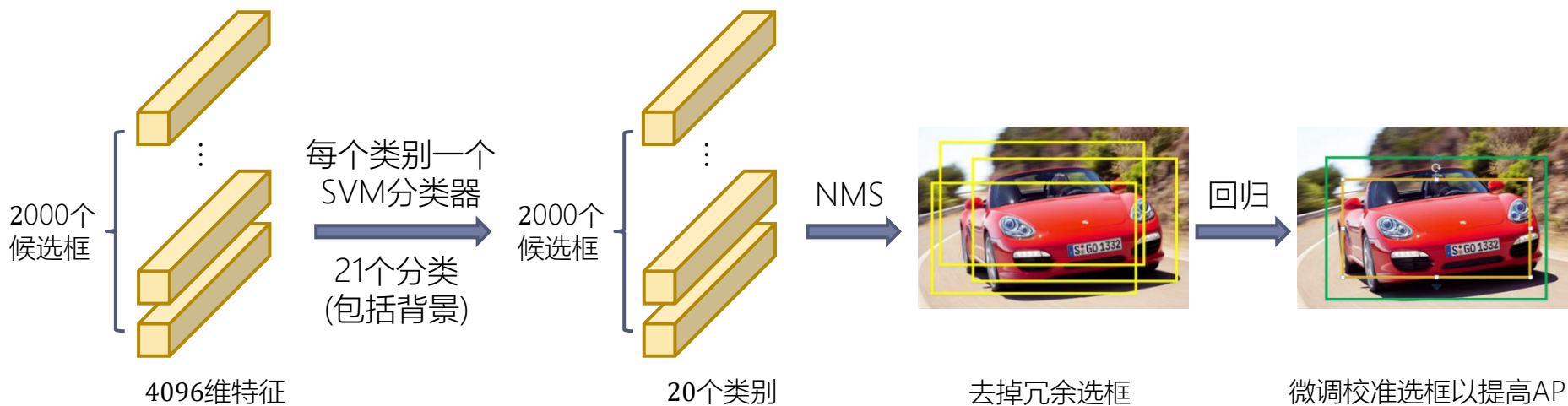
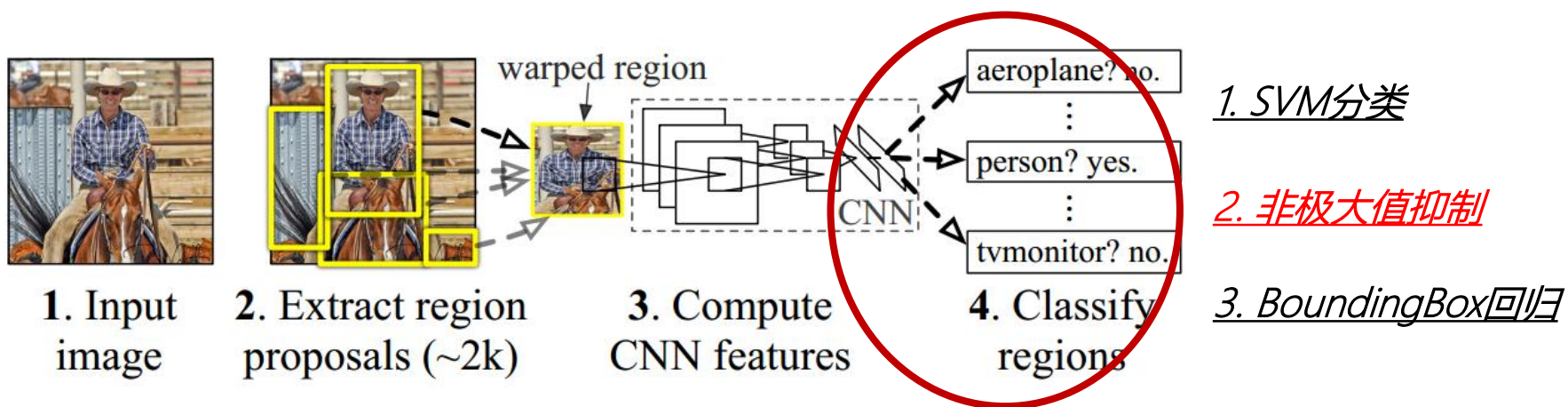
- 用基于图的图像分割方法创建初始区域
- 计算所有相邻区域间的相似度
- 每次合并相似度最高的两个相邻图像区域，并计算合并后的区域与其相邻区域的相似度。重复该过程，直到所有图像区域合并为一张完整图像
- 提取所有图像区域的目标位置框，并按层级排序（覆盖整个图像的区域层级为1）

- 在不同图像分割阈值、不同色彩空间、以及不同的相似度（综合考虑颜色、纹理、大小、重叠度）下，调用层次化分组算法，对所有合并策略下得到的位置框按**层级*随机数排序**。

- 取一定个数的候选区域作为后续卷积神经网络的输入(R-CNN取2000个)

R-CNN

• 分类与回归



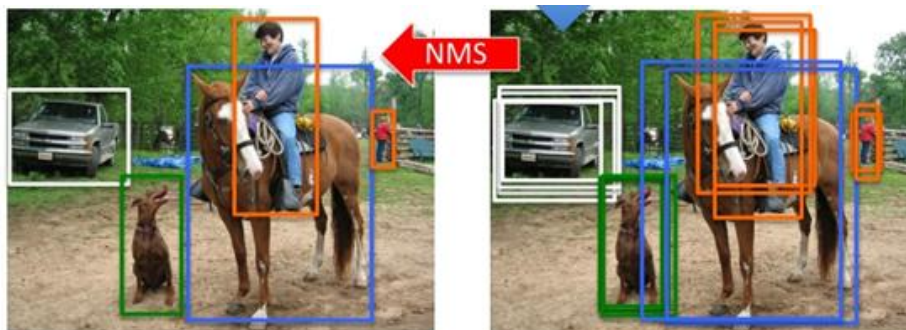
R-CNN

- 非极大值抑制(Non-Maximum Suppression, NMS)
 - 问题：同一目标的位置可能产生多个候选框，而这些候选框之间可能会有重叠
 - 解决思路：
 - 对每个分类，使用贪心NMS选择较优的目标边界框，去除冗余的边界框
 - NMS算法流程：
 1. 按照检测得分对候选框排序
 2. 将分数最高的候选框 b_m 加到最终输出列表中，并将其从候选框列表中删除
 3. 计算 b_m 与其它候选框 b_i 的IoU；如果IoU大于阈值，则从候选框列表中删除 b_i ；
 4. 重复上述步骤，直至候选框列表为空。

R-CNN

R-CNN的缺点:

- 重复计算: 需要对两千个候选框做CNN, 计算量很大, 而且有很多重复计算
 - 下图骑马的人, 三个候选框, 重叠度80%以上, 重复计算过多
- SVM模型: 在标注数据足够的时候不是最好的选择
- 多个步骤: 候选区域提取、特征提取、分类、回归都要单独训练, 大量中间数据需要保存
- 检测速度慢: GPU上处理一张图片需要13秒, CPU上则需要53秒



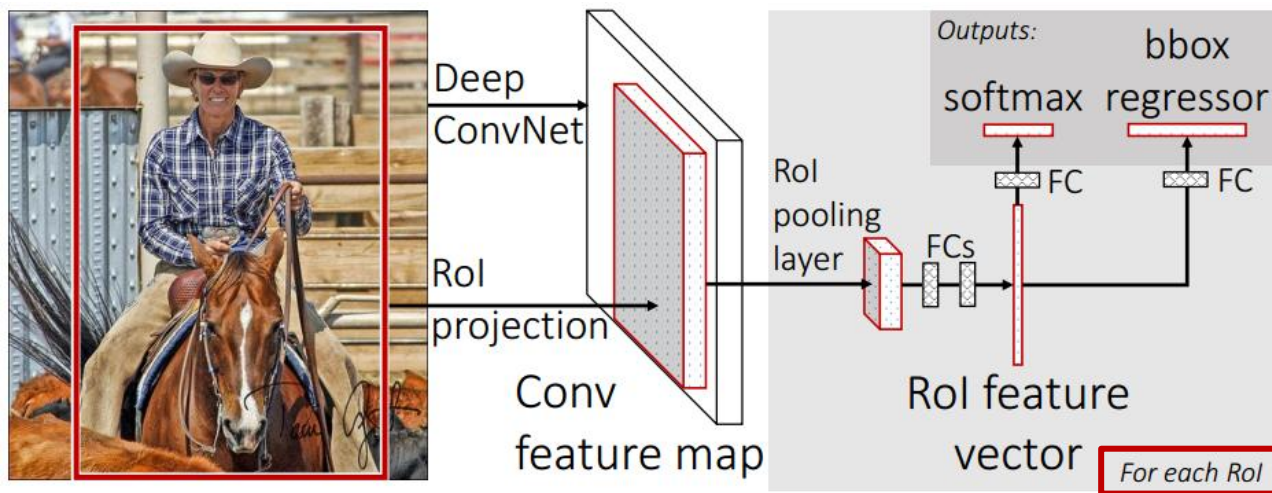
能否避免候选框特征提取过程的重复计算?



Fast R-CNN

Fast R-CNN的主要步骤:

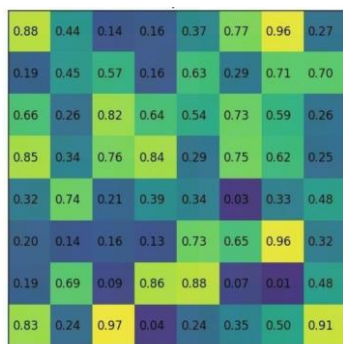
- 候选区域提取: 通过Selective Search从原始图片提取2000个左右区域候选框;
- 特征提取: 原始图像输入CNN网络, 得到特征图; (只做1次, 而不是2000次)
- ROI-Pooling: 根据映射关系, 将不同尺寸的候选框在特征图上的对应区域池化为维度相同的特征图(因为全连接层要求输入尺寸固定);
- 全连接层: 将维度相同的特征图转化为ROI特征向量(ROI feature vector);
- 分类与回归: 经过全连接层, 再用softmax分类器进行识别, 用回归器修正边界框的位置与大小, 最后对每个类别做NMS。



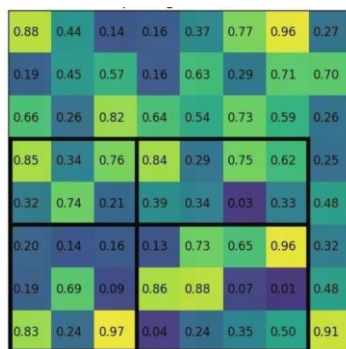
Fast R-CNN

- ROI Pooling

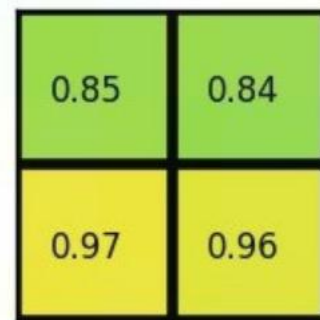
- ROI: regions of interest, 对应前文中经过region proposal得到的候选框;
- 目的: 将不同尺寸的ROI对应的卷积特征图转换为固定大小的特征图。一方面ROI可以复用卷积层提取的特征图, 提高图像处理速度; 另一方面向全连接层提供固定尺寸的特征图。
- 特点: 输出尺寸与输入尺寸无关。对每个特征图通道, 根据输出尺寸(HxW)将输入(hxw)均分成多块($h/H \times w/W$ 大小/块), 取每块的最大值作为输出。



输入特征图



ROI投影到特征图的区域



对每个子网格做最大池化

Fast R-CNN

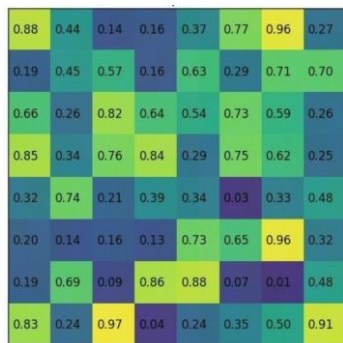
● ROI Pooling具体操作

输入：图像经过CNN之后的特征图feature map，输出尺寸H、W（分类器所需的尺寸）

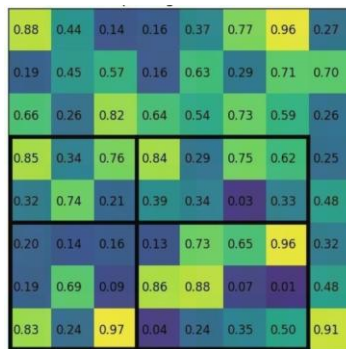
- (1) 根据输入image，将ROI映射到feature map对应位置；
- (2) 将映射后的区域划分为相同大小的sections（sections数量与输出的维度相同）；
- (3) 对每个sections进行max pooling操作；

● ROI Pooling Example

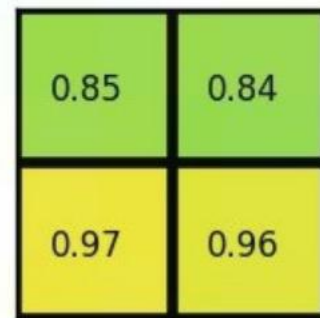
考虑一个8*8大小的feature map，1个ROI，以及输出大小为2*2.



输入特征图



ROI投影到特征图的区域



对每个子网格做最大池化

Fast R-CNN

Fast R-CNN 改进之处：

- 直接对整张图像做卷积，不再对每个候选区域分别做卷积，从而减少大量的重复计算。
- 用ROI pooling对不同候选框的特征进行尺寸归一化。
- 将边界框回归器放进网络一起训练，每个类别对应一个回归器；
- 用softmax代替SVM分类器。

Fast R-CNN 缺点：

- 候选区域提取仍使用selective search，目标检测时间大多消耗在这上面（region proposal 2~3s，而特征分类只需0.32s）；

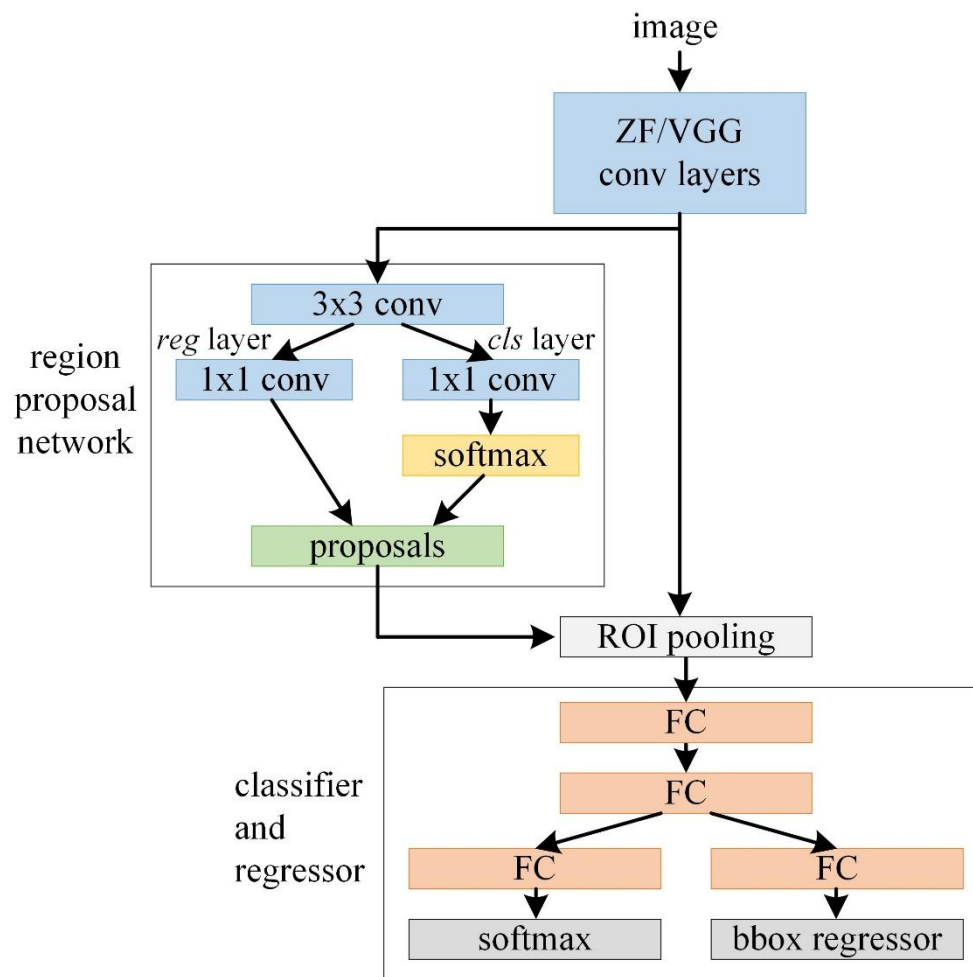
寻找更高效的候选区域生成方法？



Faster R-CNN

Faster R-CNN网络结构:

$\text{Faster R-CNN} = \text{候选区域生成网络 RPN} + \text{Fast R-CNN}$



Faster R-CNN

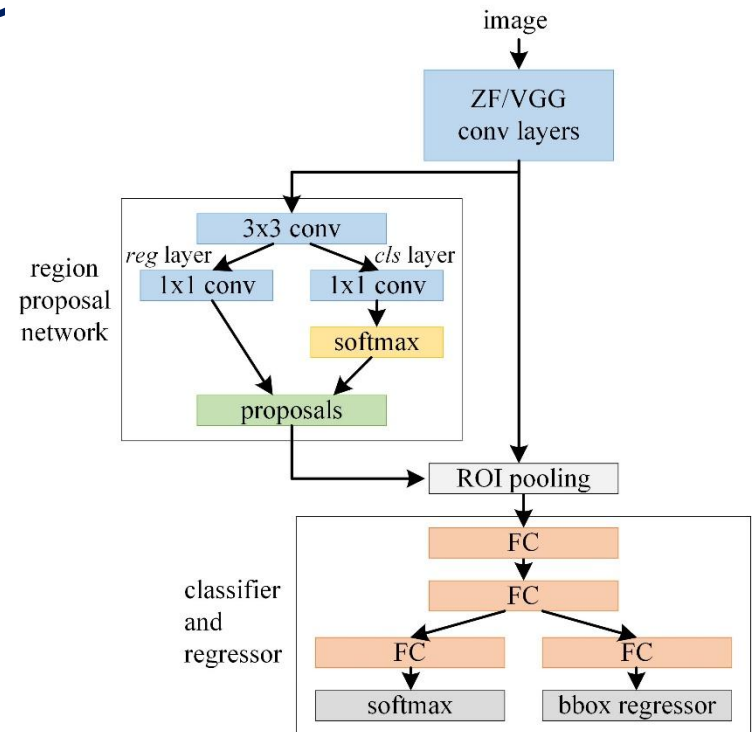
Faster R-CNN主要步骤:

1. **卷积层**: 输入图片经过多层卷积神经网络 (ZF、VGG), 提取出卷积特征图, 供RPN网络和Fast R-CNN使用。RPN网络和Fast R-CNN共享特征提取网络可大大减小计算时间;

2. **RPN层**: 生成候选区域, 并用 softmax 判断候选框是前景还是背景, 从中选取前景候选框并利用 bounding box regression 调整候选框的位置, 得到候选区域;

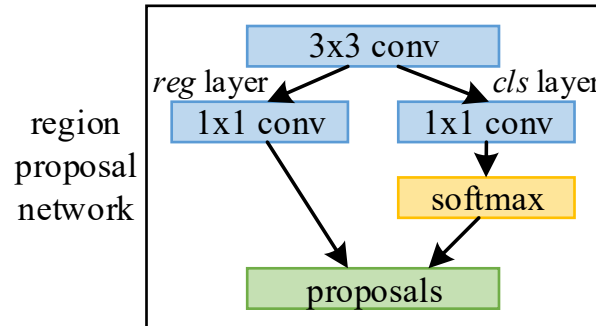
3. **ROI Pooling层**: 同 Fast R-CNN, 将不同尺寸的候选框在特征图上的对应区域池化为维度相同的特征图

4. **分类与回归**: 同 Fast R-CNN, 用softmax分类器判断图像类别, 同时用边界框回归修正边界框的位置和大小



Faster R-CNN

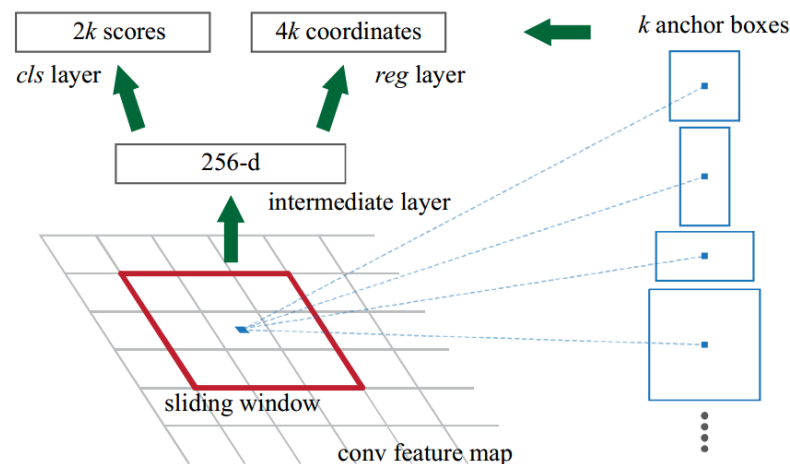
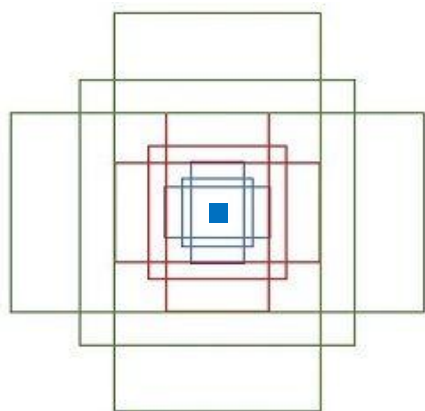
- RPN (region proposal networks)



- **目的：**输入特征图，输出候选区域集合，包括各候选区域属于前/背景的概率、以及位置坐标。RPN采用Anchor机制能够从特征图上直接提取候选区域的特征，相对于selective search大大减少运算量，且整个过程融合到一个网络中，方便训练和测试。
- **步骤及方法：**
 1. 先经过一个3x3卷积，使每一个点对应256维(ZFNet模型) 或512维 (VGG16) 特征向量。
 2. 然后分两路处理：一路用来判断候选框是前景还是背景，先将卷积特征reshape成一维向量，然后做softmax判断是前景还是背景，然后reshape回二维feature map；另一路用bbox regression来确定候选框的位置。
 3. 两路计算结束后，计算得到前景候选框（因为物体在前景中），再用NMS去除冗余候选框，最后输出候选区域。

Faster R-CNN

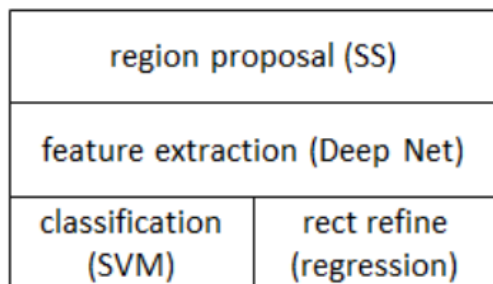
- 关于anchor box



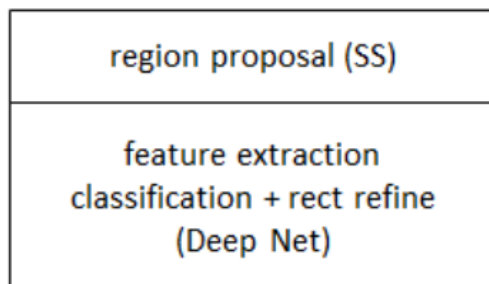
- 对于feature map的每个位置，考虑9个可能的候选框：三种面积分别是 128×128 , 256×256 , 512×512 ，每种面积又分成3种长宽比，分别是2:1, 1:2, 1:1，这些候选框称为anchors。
- 在RPN中，feature map每个位置输出 $2k$ （ k 为anchor box的数目）个得分，分别表示该位置的 k 个anchor为前景/背景的概率；同时每个位置输出 $4k$ 个框位置参数，用 $[x, y, w, h]$ 四个坐标来表示 anchor的位置。

R-CNN 系列

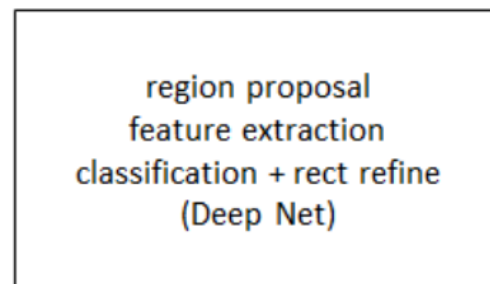
● R-CNN系列



RCNN



fast RCNN



faster RCNN

网络	mAP (VOC2012)	单帧检测 时间
R-CNN	53.3%	50 s
Fast R-CNN	65.7%	2 s
Faster R-CNN	67.0%	0.2 s

- 从R-CNN到Fast R-CNN，再到Faster R-CNN，目标检测的四个基本步骤（候选区域生成，特征提取，分类，位置调整）终于被统一到一个深度网络框架之内，大大提高了运行速度。

拓展：R-CNN训练相关，Ross Girshick在ICCV15的演讲，[Training R-CNNs of various velocities\(Slow, fast, and faster\)](http://arxiv.org/abs/1506.01497)

基于CNN的图像检测算法

- R-CNN系列
 - YOLO
 - SSD
- } One-stage

目前，基于深度学习的目标检测算法大致分为两类：

- 1.两阶段 (two-stage) 算法：基于候选区域方法，先产生边界框，再做CNN分类(R-CNN系列)
- 2.一阶段 (one-stage) 算法：对输入图像直接处理，同时输出定位及其类别 (YOLO系列)

YOLO

- YOLO(v1): Redmon J, Divvala S, Girshick R, et al. You Only Look Once: Unified, Real-Time Object Detection[J]. 2015.
- **主要思想**: 将目标检测问题转换为直接从图像中提取bounding boxes和类别概率的**单回归问题**, 只需看一眼(you only look once, YOLO) 就可以检测出目标的类别和位置。
YOLO算法开创了one-stage检测的先河, 将目标分类和边界框定位合二为一, 实现了端到端的目标检测。YOLO的运行速度非常快, 达到45帧/秒, 满足实时性要求。

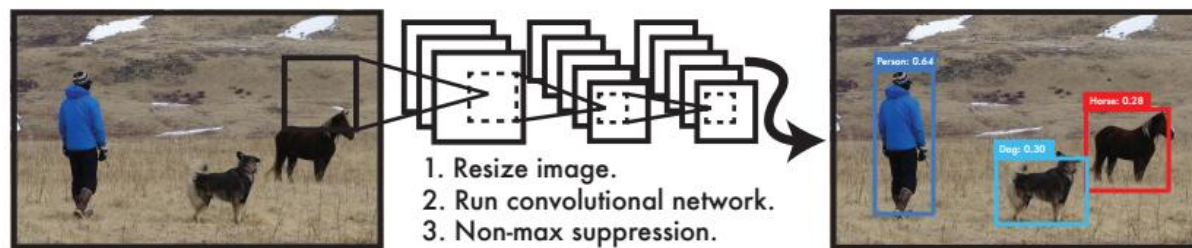


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

YOLO

- 统一检测(Unified Detection)具体实现

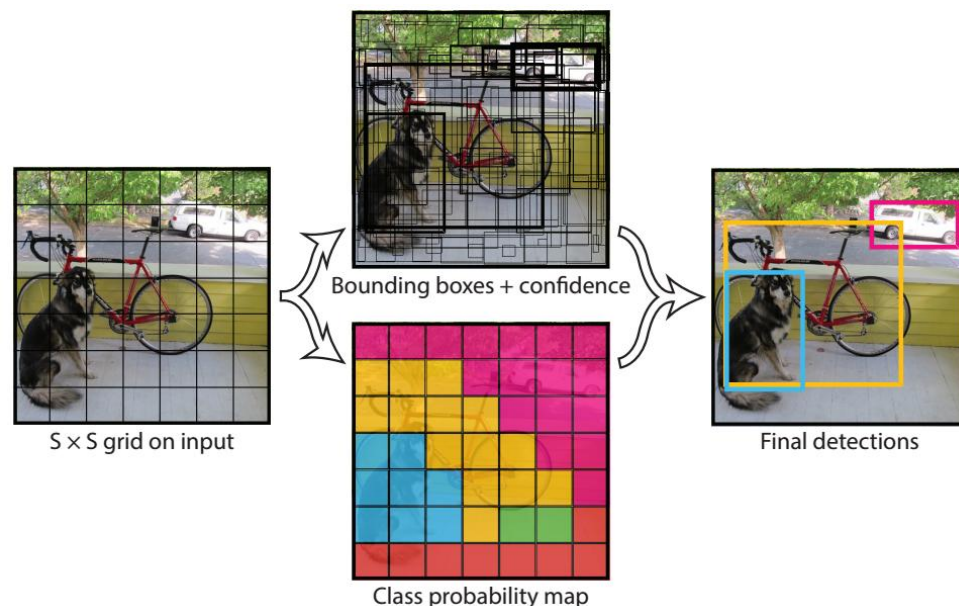
- 将输入图像分为 $S \times S$ 个格子，每个格子都预测B个Bounding box，每个bbox包含五个预测值：x, y, w, h和confidence;
- x, y, w, h用于表示bbox的位置和大小，且都被归一化到(0, 1);
- confidence(置信度分数) 综合考虑了当前bbox内存在目标的可能性 $\Pr(\text{Object})$ 以及预测目标位置的准确性 $\text{IOU}(\text{pred}|\text{truth})$ ，定义为：

$$\text{confidence} = \Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}, \quad \Pr(\text{Object}) = \begin{cases} 1, & \text{object exists in that cell} \\ 0, & \text{no object exists in that cell} \end{cases}$$

- 每个格子还要预测分别属于C种类别的条件概率 $\Pr(\text{Class}_i|\text{Object})$, $i = 0, 1, \dots, C$ ，其中C为数据集物体类别数量；在测试时，属于该格子的B个bbox共享这C个类别的条件概率（B个bbox属于某类别的概率求和）。某个bbox的类别置信度为：

$$\Pr(\text{Class}_i|\text{Object}) * \Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}} = \Pr(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}}$$

- 最终输出tensor的维度为： $S \times S \times (B \times 5 + C)$

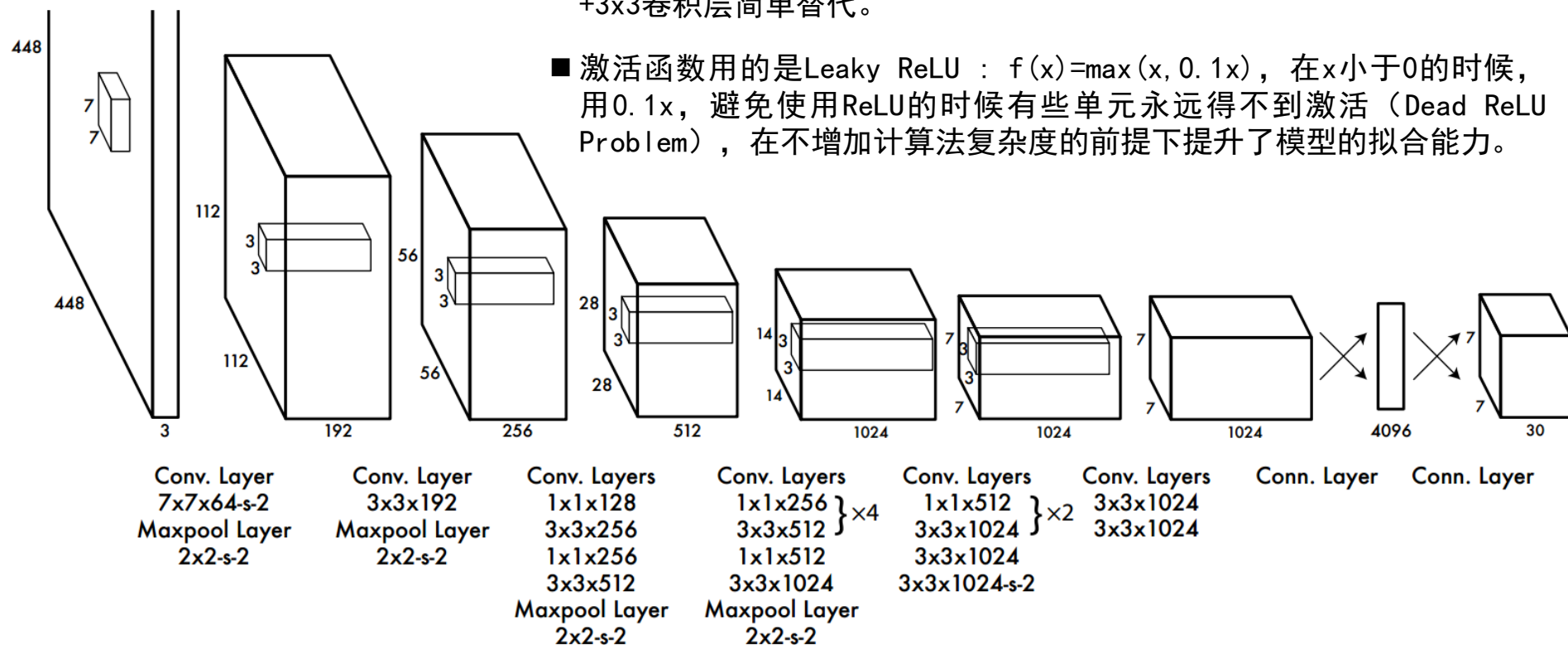


YOLO

● 网络结构

■ 网络结构基于GoogleNet，不同的是，YOLO未使用inception module，而是使用1x1卷积层（此处1x1卷积层的存在是为了跨通道信息整合）+3x3卷积层简单替代。

■ 激活函数用的是Leaky ReLU： $f(x) = \max(x, 0.1x)$ ，在 x 小于0的时候，用 $0.1x$ ，避免使用ReLU的时候有些单元永远得不到激活（Dead ReLU Problem），在不增加计算复杂度的前提下提升了模型的拟合能力。



- 对于PASCAL VOC数据集，采用 $S=7$ ， $B=2$ ， $C=20$ ，最终输出tensor维度为 $7 \times 7 \times 30$ (其中 $30=B*5+C$)。

YOLO

YOLO(v1)的优点

- 1、检测速度快：** YOLO将目标检测重建为单一回归问题，对输入图像直接处理，同时输出边界框坐标和分类概率，而且每张图像只预测98个bbox，检测速度非常快，在Titan X 的 GPU 上能达到45 FPS，Fast YOLO检测速度可以达到155 FPS。
- 2、背景误判少：** 以往基于滑窗或候选区域提取的目标检测算法，只能看到图像的局部信息，会出现把背景当前景的问题。而YOLO在回归之前做了全连接，在训练和测试时能对整张图片同时进行预测，能利用图片上下文信息避免对背景的误识别。
- 3、泛化性更好：** YOLO能够学习到目标的泛化表示，能够迁移到其它领域。例如，当YOLO在自然图像上做训练，在艺术品上做测试时，YOLO的性能远优于DPM、R-CNN等。YOLO可以学习到高度泛化的特征，从而迁移到其他领域。

YOLO

YOLO(v1) 的缺点

- 1、**邻近物体检测精度低**：YOLO对每个cell只预测两个bbox和一个分类，如果多个物体的中心都在同一cell内，检测精度低。
- 2、**损失函数的设计过于简单**：损失函数包含了坐标误差、IOU误差、分类误差，都用坐标和分类的MSE（均误差）作为损失函数，比较简单粗暴。

$$\begin{aligned} & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \end{aligned}$$

$\mathbb{1}_i^{\text{obj}}$ 表示目标出现在cell i中 $\mathbb{1}_{ij}^{\text{obj}}$ 表示cell i中第j个边框预测目标在该cell中

➤ 检测两个标准：框在哪里？框里是什么分类？

- 3、**训练不易收敛**：直接预测的bbox位置，相较于预测物体的偏移量，模型收敛不稳定。

YOLO

●拓展

- YOLO-v2 : Redmon J, Farhadi A. YOLO9000: Better, Faster, Stronger[C]// IEEE Conference on Computer Vision & Pattern Recognition. 2017.

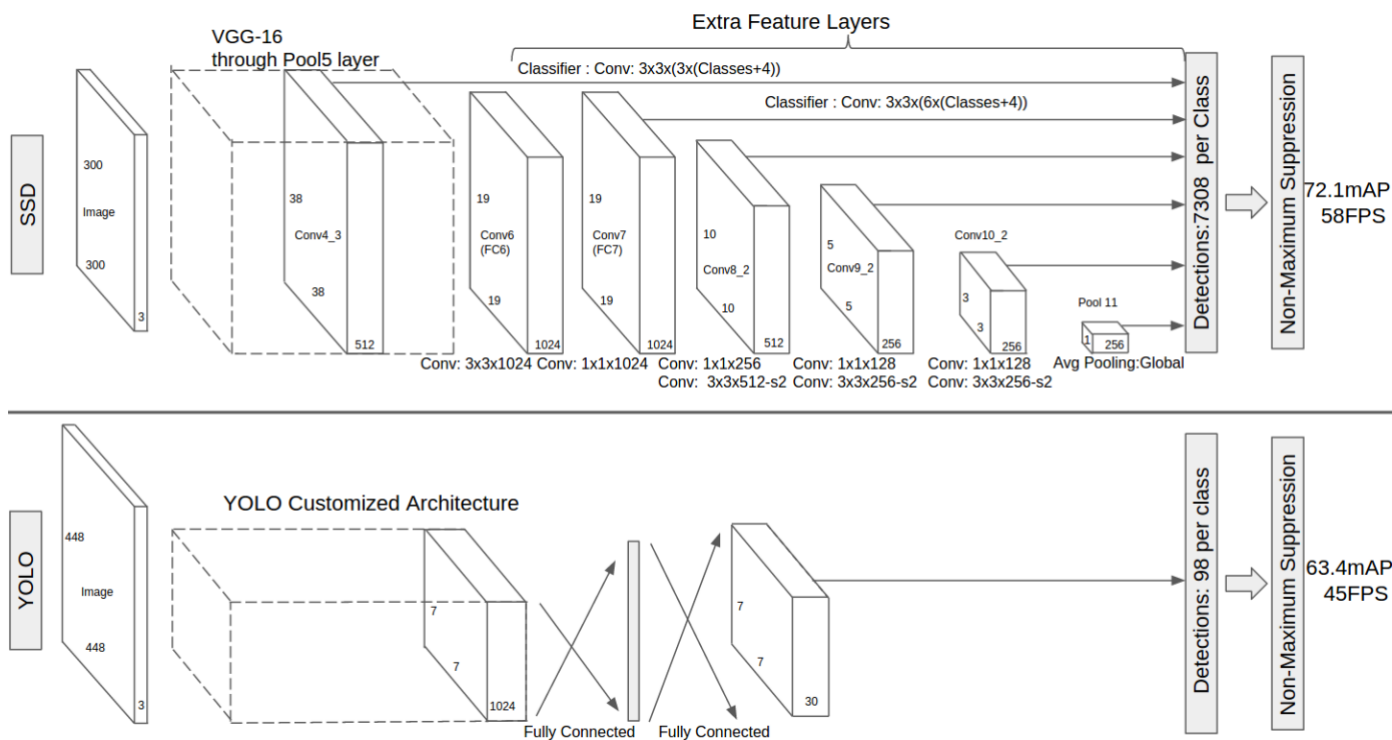
引入了faster rcnn中anchor box的思想; 对网络结构的设计进行了改进(Darknet-19); 输出层使用卷积层替代YOLO的全连接层, 联合使用coco物体检测标注数据和imagenet物体分类标注数据训练物体检测模型。

- YOLO-v3 : Redmon J, Farhadi A. YOLOv3: An Incremental Improvement[J]. 2018.

类似FPN (Feature Pyramid Network)多尺度目标检测方案) 的多尺度预测; 更好的基础分类网络 (Darknet-53, 结合resnet) ; Sigmoid代替Softmax用于多标记分类。

SSD

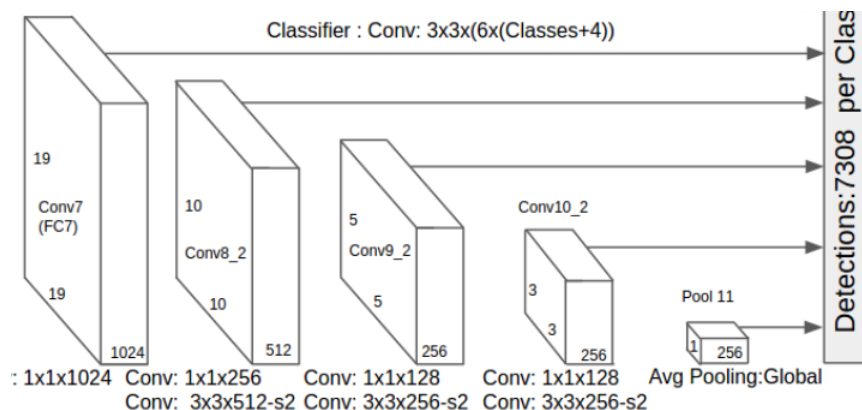
- Liu W, Anguelov D, Erhan D, et al. SSD: Single Shot MultiBox Detector[C]// European Conference on Computer Vision. 2016.
- 主要思想：基于YOLO直接回归 bbox和分类概率的one-stage检测方法，结合Faster R-CNN中的**anchor-box**思想产生先验框，并且采用特征金字塔进行多尺度预测（6个特征图），在满足检测速度快的同时，大大提高了检测准确度。



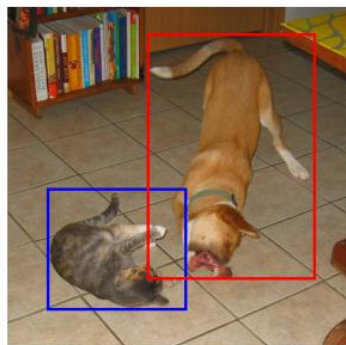
SSD

多尺度其实是一个很常规的方法，为什么SSD能发高水平论文？

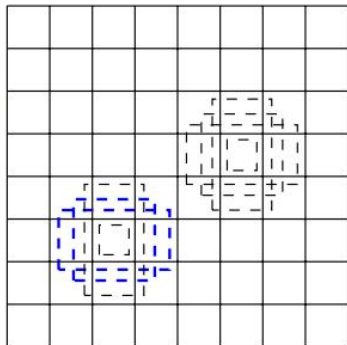
● 多尺度特征图检测



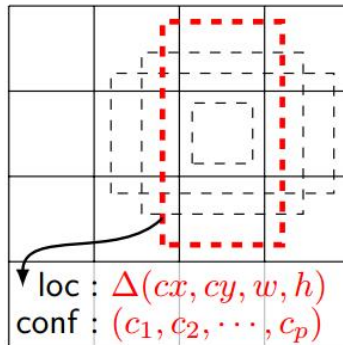
- CNN网络一般前面的特征图比较大，后面会逐渐采用stride=2的卷积或者pool来降低特征图大小，**在大的和小的特征图都提取 anchor box**用来做检测，可找到最合适的 anchor box 尺寸，提高检测准确度。



(a) Image with GT boxes



(b) 8 × 8 feature map



(c) 4 × 4 feature map

- 比较大（浅层）的特征图可以用来检测相对较小的目标，而小的（深层）特征图负责检测大目标，例如左图中8x8的特征图可以划分成更多单元，其每个单元的先验框尺度较小，适合用于检测较小的目标。

SSD

- Anchor box (论文中为default box)

- 个数：每个点对应6个anchor box, 2正方形和4长方形(宽高比 a_r 取1、2、3、1/2、1/3、1);
 - 对于先验框的尺度，其遵守一个线性递增规则：随着特征图大小降低，先验框尺度线性增加
- 第k层scale:

$$s_k = s_{\min} + \frac{s_{\max} - s_{\min}}{m - 1}(k - 1), \quad k \in [1, m]$$

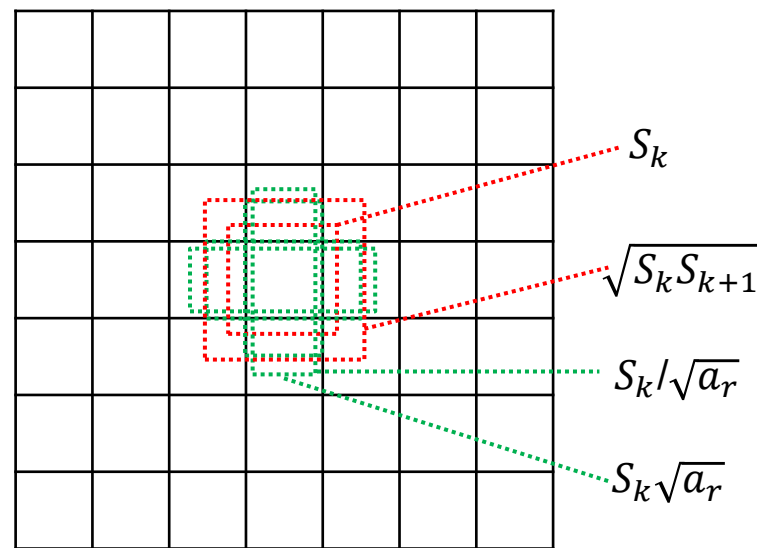
其中： $s_{\min} = 0.2, s_{\max} = 0.9$, m 为生成anchor box进行检测的卷积层总层数（第1层除外）；

宽高比为1的情况，增加一个默认框，其缩放为 $s'_k = \sqrt{s_k s_{k+1}}$

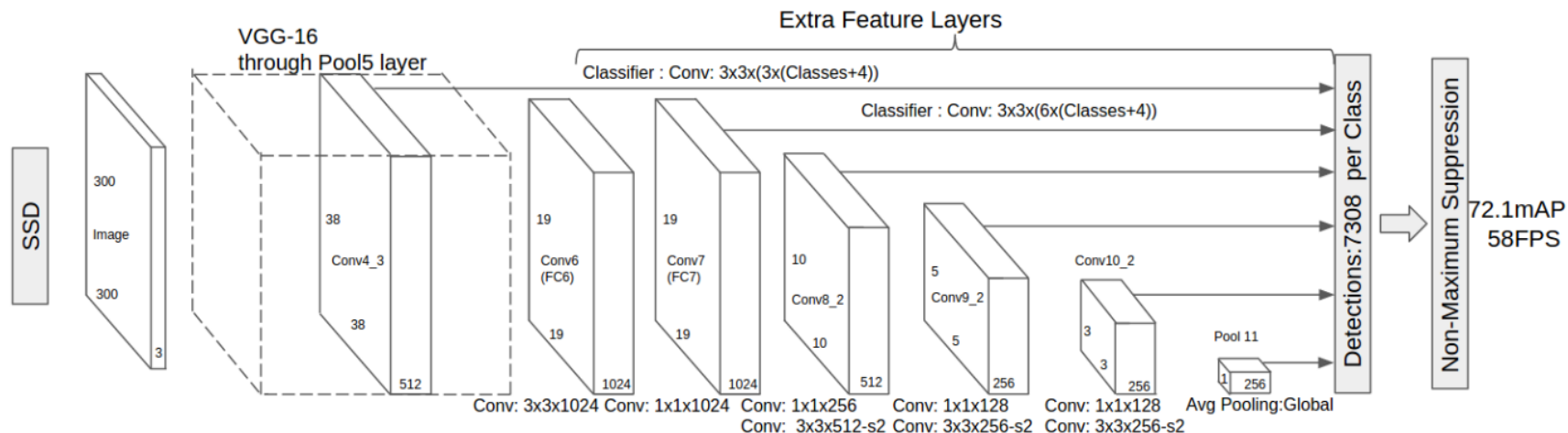
➤ 对于第一个特征图，先验框比例为 $s_{\min}/2=0.1$ ，尺寸为 $300*0.1=30$ 。

- 第k层default box的宽: $w_k^a = s_k \sqrt{a_r}$

$$\text{高: } h_k^a = s_k / \sqrt{a_r}$$



SSD

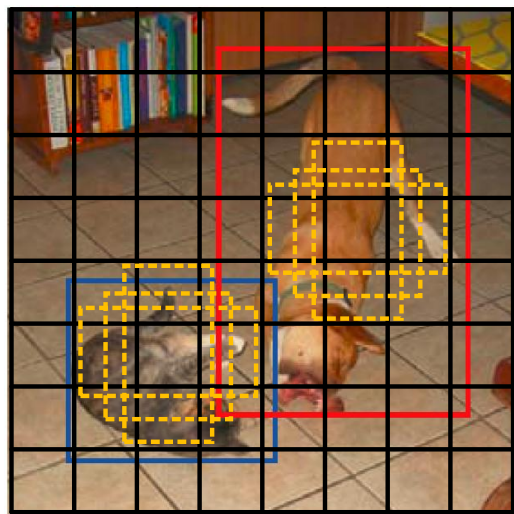


- 在实现时，Conv4_3，Conv10_2和Conv11_2层仅使用4个先验框，它们不使用长宽比为 3,1/3 的先验框。
- 由于每个先验框都会预测一个边界框，所以SSD300一共可以测

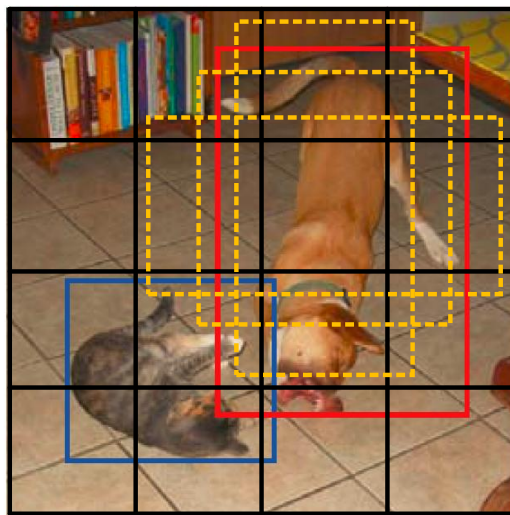
$$38 \times 38 \times 4 + 19 \times 19 \times 6 + 10 \times 10 \times 6 + 5 \times 5 \times 6 + 3 \times 3 \times 4 + 1 \times 1 \times 4 = 8732$$
- 这是一个相当庞大的数字，所以说SSD本质上是密集采样。

SSD

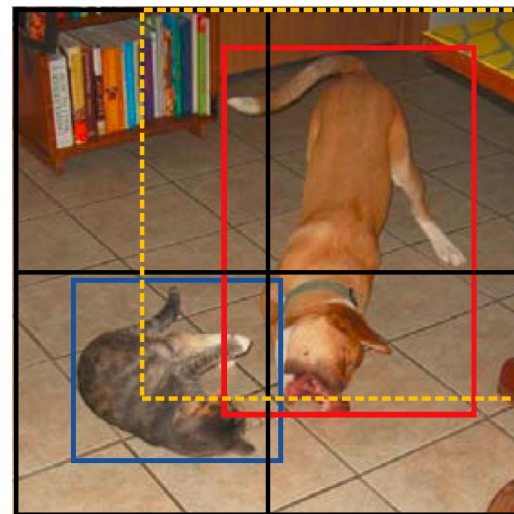
- 同时对多层特征图上的默认框计算IOU，可以找到与真实框大小和位置最接近（即IOU最大）的框，在训练时能达到最好的精度。
- SSD的好处就是在不同层级的特征图上，都做检测。这样比较容易找到跟真实框大小位置最接近。



较低层级的特征图



中间层级的特征图

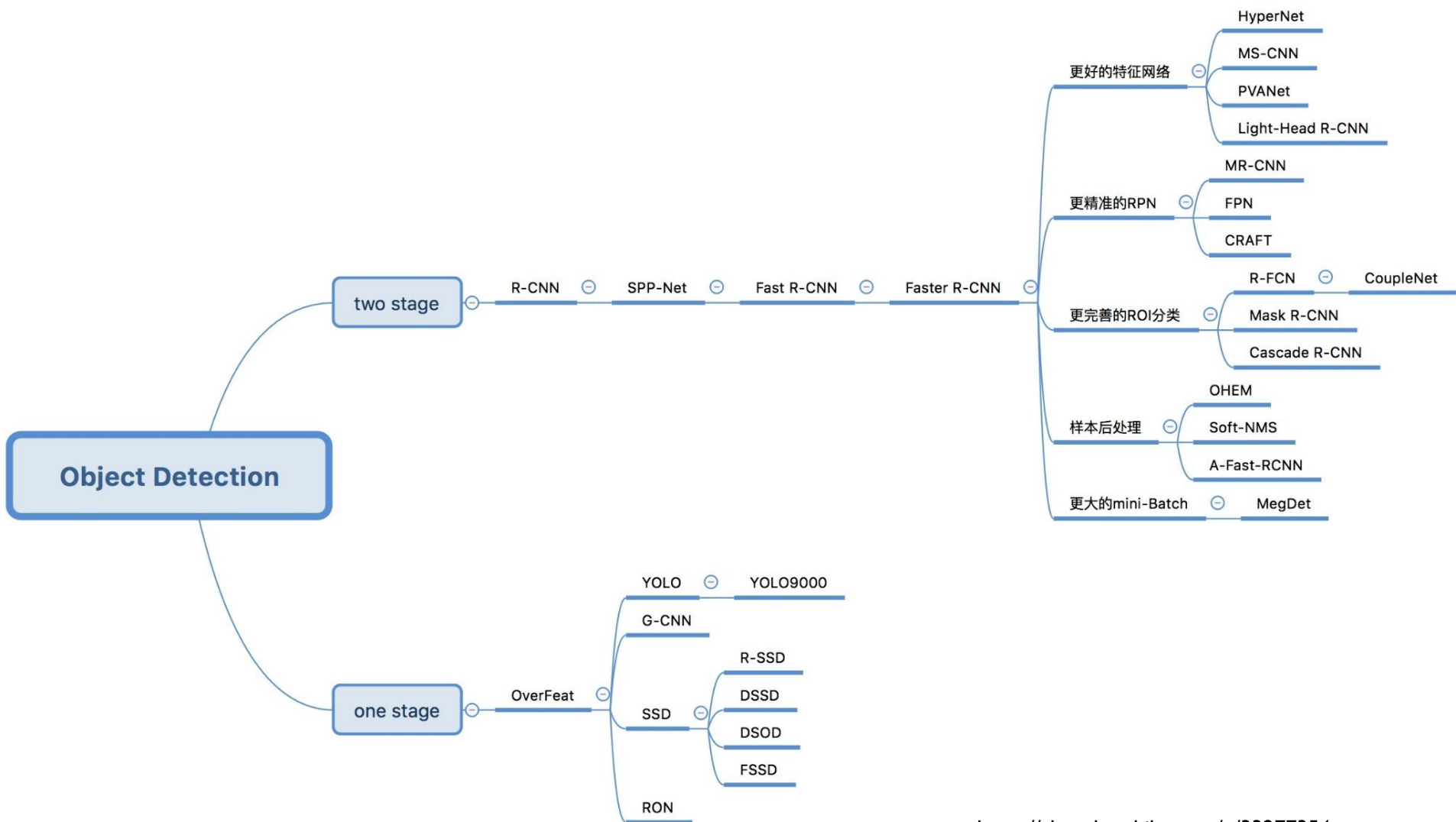


较高层级的特征图

SSD训练过程

- ▶ <https://zhuanlan.zhihu.com/p/33544892>
- ▶ one-stage方法，如Yolo和SSD，其主要思路是均匀地在图片的不同位置进行密集抽样，抽样时可以采用不同尺度和长宽比，然后利用CNN提取特征后直接进行分类与回归，整个过程只需要一步，所以其优势是速度快，但是均匀的密集采样的一个重要缺点是训练比较困难，这主要是因为正样本与负样本（背景）极其不均衡，导致模型准确度稍低。
- ▶ SSD的先验框与ground truth的匹配原则主要有两点。
 - ▶ 原则1：对于图片中每个ground truth，找到与其IOU最大的先验框，该先验框与其匹配，保证每个ground truth一定与某个先验框匹配。与ground truth匹配的先验框为正样本，反之，若一个先验框没有与任何ground truth进行匹配，就是负样本。一个图片中ground truth:非常少，而先验框却很多，如果仅按第一个原则匹配，很多先验框会是负样本，正负样本极其不平衡，所以需要第二个原则。
 - ▶ 原则2：对于剩余的未匹配先验框，若某个ground truth的 IOU 大于某个阈值（一般是0.5），那么该先验框也与这个ground truth进行匹配。这意味着某个ground truth可能与多个先验框匹配。但是反过来却不可以，因为一个先验框只能匹配一个ground truth，如果多个ground truth与某个先验框 IOU 大于阈值，那么先验框只与IOU最大的那个ground truth进行匹配。
- ▶ 为了保证正负样本尽量平衡，SSD采用了hard negative mining，就是对负样本进行抽样，抽样时按照置信度误差（预测背景的置信度越小，误差越大）进行降序排列，选取误差的较大的top-k作为训练的负样本，以保证正负样本比例接近1:3。（核心是选难的样本）

图像检测算法



<https://zhuanlan.zhihu.com/p/33277354>



谢谢大家！